Bios 740- Chapter 6. Deep Generative Models

Acknowledgement: Many thanks to Mr. Xiaoqi Liu for preparing some of these slides, and to Drs. Mingxia Liu, Gang Li, Xiao Wang, and Jian Huang for sharing their slides. I also drew on material from the lecture presentations of Stanford CS224w and StanfordCS231n, Dr. Liu's presentation, and Dr. Huang's lecture, as well as content generated by ChatGPT.

Content

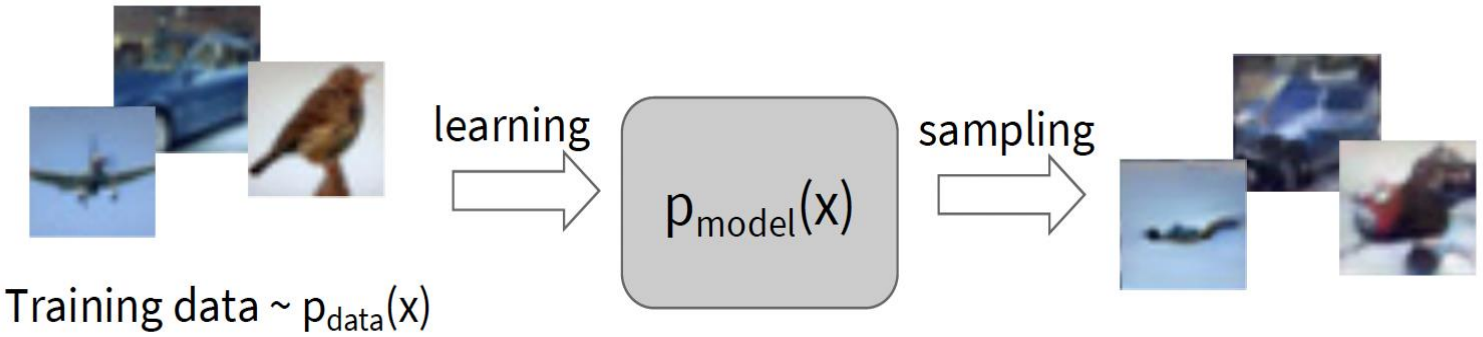
1 Motivating Applications
2 Understanding Deep Generative Models
3 PixelDNN and Variational Autoencoder
4 GANs and their Architectures
5 Normalized Flow
6 Diffusion Models
7 Score Matching
8 Applications
9 Theoretical Properties

Content

1 Motivating Applications

What are Generative Models?

Definition: Generative models learn to generate new data samples resembling a given dataset.



Objectives:

1. Learn $p_{model}(x)$ that approximates $p_{data}(x)$
2. Sampling new x from $p_{model}(x)$

Major Generative Models:

- **Explicit Density Models:** Estimate probability distributions (e.g., Gaussian Mixture Models, VAEs).
- **Implicit Density Models:** Generate samples without explicit density estimation (e.g., Generative Adversarial Network (GAN)s, Diffusion Models).

Explicitly Density Models

A simple form of generative learning is to learn a deterministic function

- Generate $\eta \sim N(\mathbf{0}, \mathbf{I}_m), m \geq 1$.
- Estimate a generator function G such that

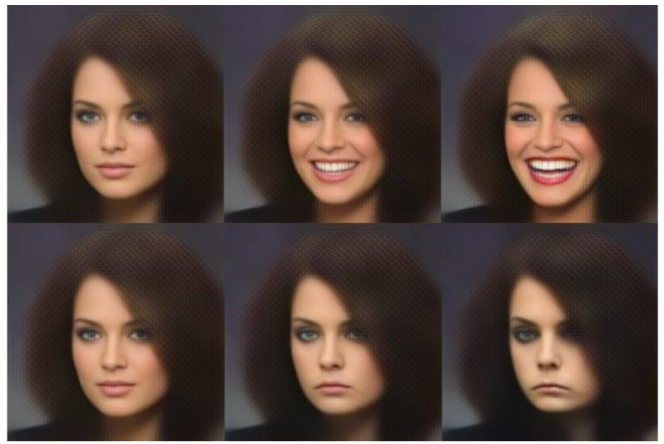
$$G(\eta) \sim P_X.$$

- Let $X = G_\theta(Z)$ with $Z \sim p_Z$. The distribution function of X is

$$P(X \leq \boldsymbol{x}) = P(G_\theta(Z) \leq \boldsymbol{x}) = \int_{G_\theta(\boldsymbol{z}) \leq \boldsymbol{x}} p_Z(\boldsymbol{z}) d\boldsymbol{z}$$

with density function

$$p_{G_\theta}(\boldsymbol{x}) = \frac{\partial}{\partial \boldsymbol{x}} \int_{G_\theta(\boldsymbol{z}) \leq x} p_Z(\boldsymbol{z}) dz.$$



If $G_\theta(\boldsymbol{z})$ is invertible, then we have

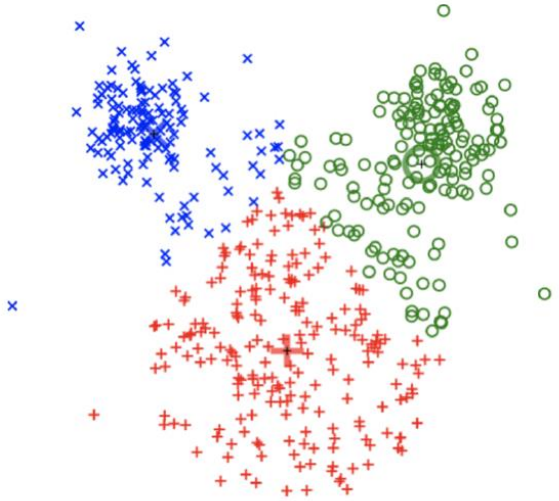
$$p_{G_\theta}(\boldsymbol{x}) = \pi(G_\theta^{-1}(\boldsymbol{x}))|\det(\nabla_x G_\theta^{-1}(\boldsymbol{x}))|.$$

How?

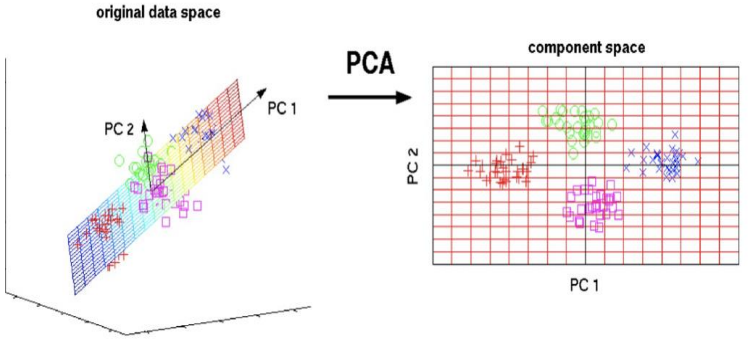
It is still difficult to calculate the inverse G_θ^{-1}.

Unsupervised Learning

- **Data:** X — Just data, no labels
- **Goal:** Learn some underlying hidden structure or distribution of the data
- **Examples:** clustering, dimension reduction, feature learning, density estimation, etc.
- **Generative models are a subset of unsupervised learning, but not all unsupervised learning techniques are generative (e.g., k-means, PCA)**



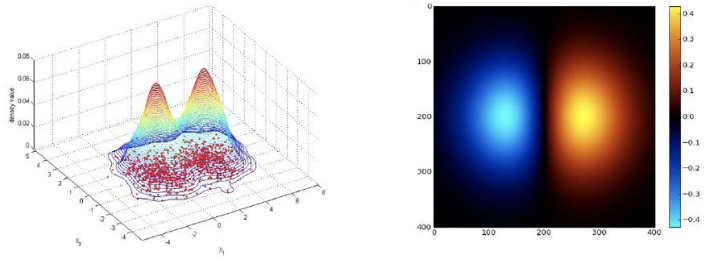
K-means clustering



PCA

Figure copyright Ian Goodfellow, 2016. Reproduced with permission.

1-d density estimation



2-d density estimation

Modeling p(x)

2-d density images left and right are CC0 public domain

Why GANs and Other Generative Models?

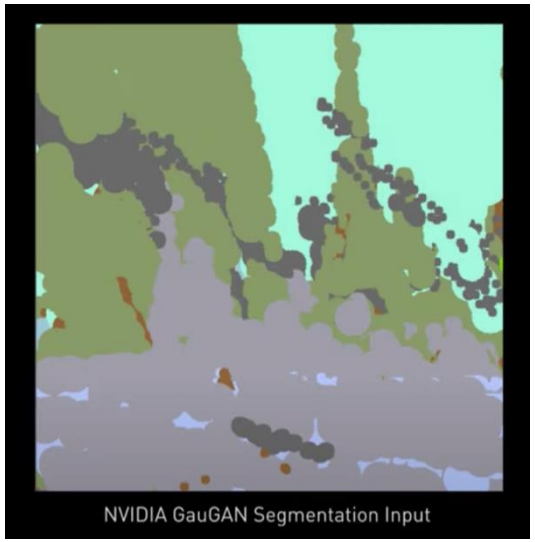
Goodfellow, Ian, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. "Generative adversarial nets." *Advances in neural information processing systems* 27 (2014).

GANs Progression on Face Generation

- Better Quality
- High Resolution

1024*1024 Images generated by a GAN created by NVIDIA. (source, 2018)

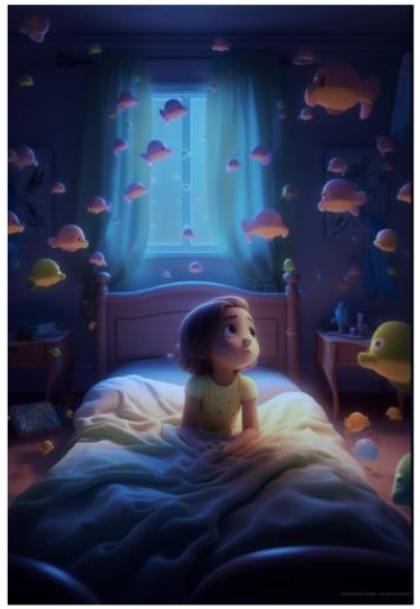
Why Gans?



Realistic samples for artwork, super-resolution, colorization, etc.

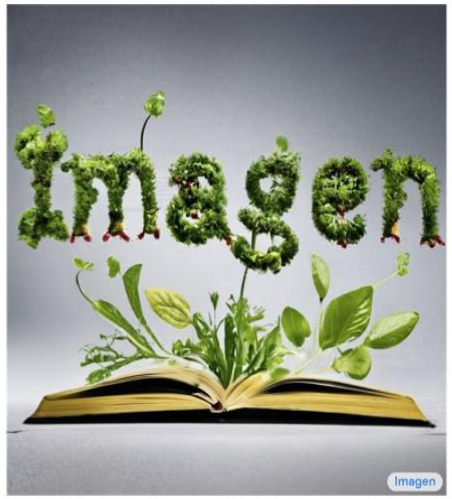
- Learn useful and subtle features for downstream tasks such as classification and object detection.
- Getting insights from high-dimensional data (physics, medical imaging, etc.)
- Modeling physical world for simulation and planning (robotics and reinforcement learning applications)
- Many more ...

AI Art



Midjourney v5

Emerging Generative Models in 2022-



alphacode
it solutions

Parti
Pathways Autoregressive Text-to-Image Model

DESIGNS.AI

DALL·E 2

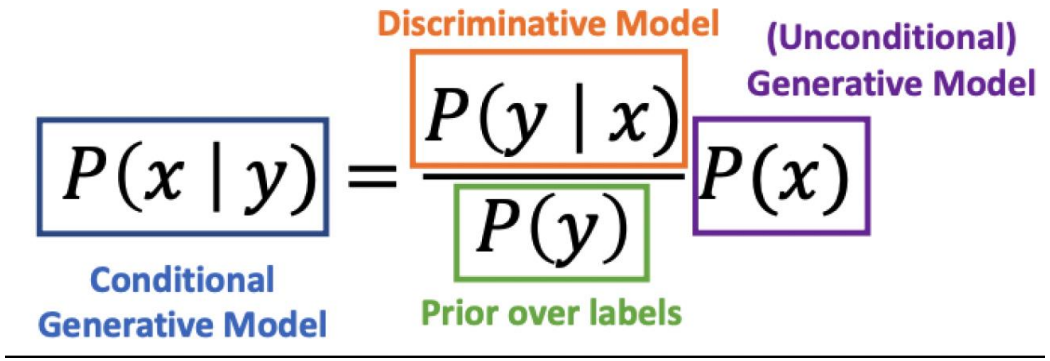
Stable Diffusion

What are Conditional Generative Models?

Definition: A **conditional generative model** is a type of generative model that generates new data samples based on additional context or conditioning information. Mathematically, it models the conditional probability distribution,

Applications: Image-to-Image translation, Text-to-Image generation, Super-Resolution, etc.

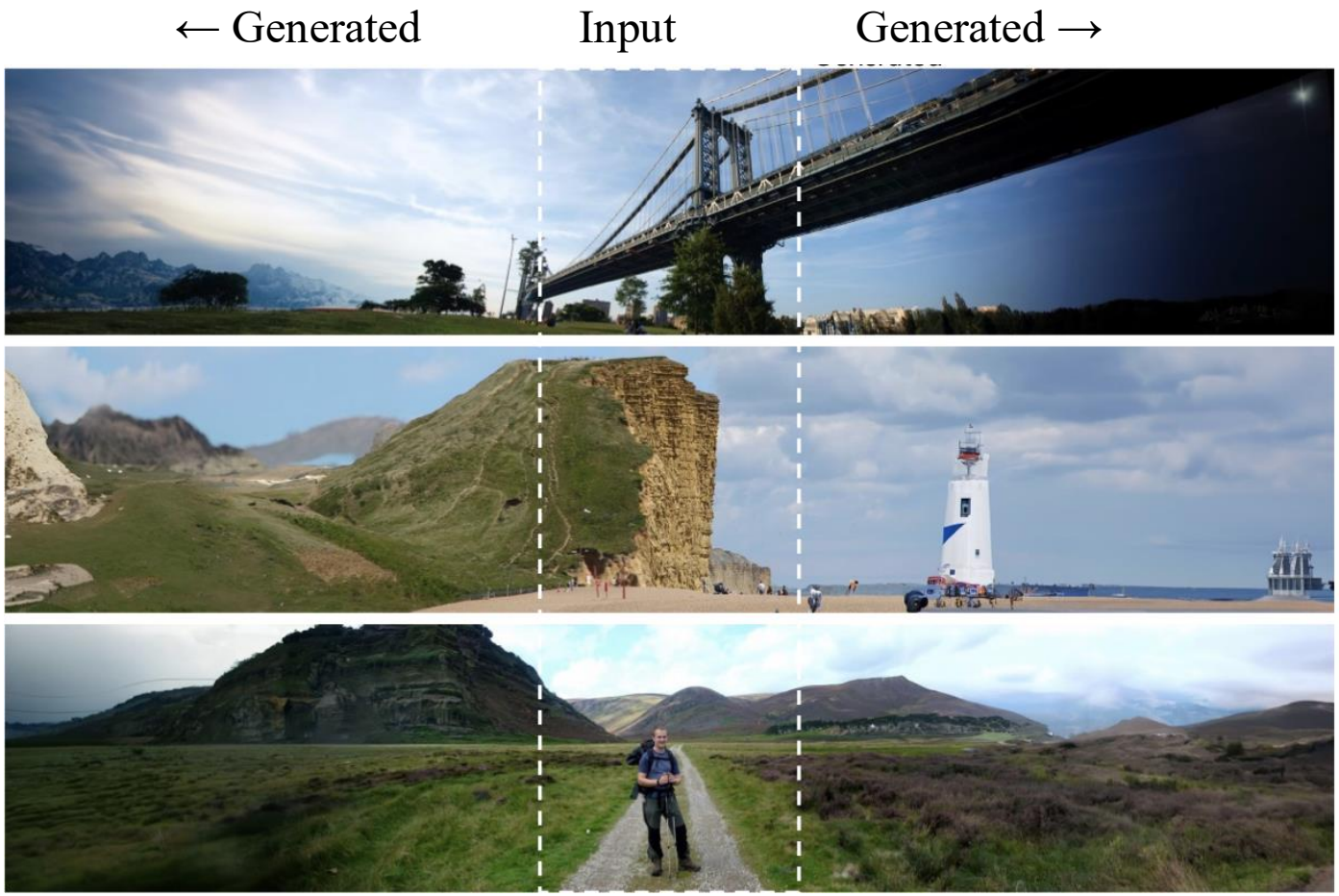
$$P(x \mid y) = \frac{P(y \mid x)}{P(y)} P(x)$$



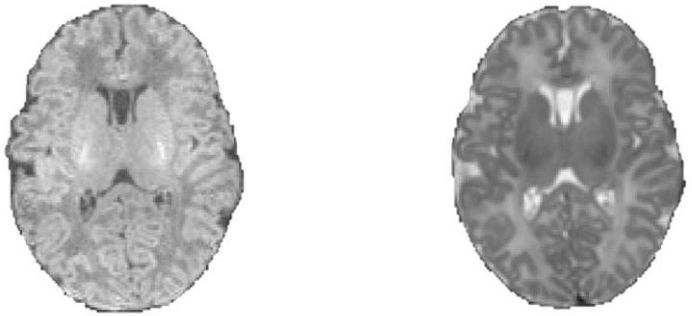
We can build a conditional generative model from other components!

Why Conditional Generative Models?

Outfilling/Missing Data Imputation



T1w<---------→T2w



- **Infant brain MR images (T1w/T2w)**
  - **Low** tissue contrast and **dynamic** change in appearance

Additional Applications

Text to Image

Fashion Design

Content

2 Understanding Deep Generative Models

Deep Generative Models

Deep generative models are neural network-based models designed to learn complex data distributions and generate realistic synthetic samples that resemble the original training data. These models leverage deep learning to approximate the true underlying data distribution.

Let $X \sim P_X$, where P_X is the distribution of X. Let its density function be p_X. There are two ways to learn the distribution of X:

- The explicit modeling approach assumes $p_X \in \mathcal{P}_\Theta$, or estimates p_X directly nonparamtrically.
- Generative models learn a generator function $G : \mathbb{R}^m \to \mathbb{R}^p$ such that $G(\eta) \sim P_X$, where $\eta \sim P_\eta$, a known reference distribution.
  - If a generator function G is known, then we know everything about P_X, since we can first sample $\eta \sim P_\eta$, then $G(\eta) \sim P_X$.
  - We usually take the reference distribution to be $N(\mathbf{0}, \mathbf{I}_m)$ or uniform distribution on $[0, 1]^m$.

Common DGMs:

- Deep Belief Networks (DBNs)
- Deep Boltzmann Machines (DBMs)
- Denoising Autoencoders (DAEs)
- Generative Stochastic Networks (GSNs)
- PixelRNN/PixelCNN
- Generative Adversarial Network (GANs)
- Variational Autoencoder

Taxonomy of Deep Generative Models

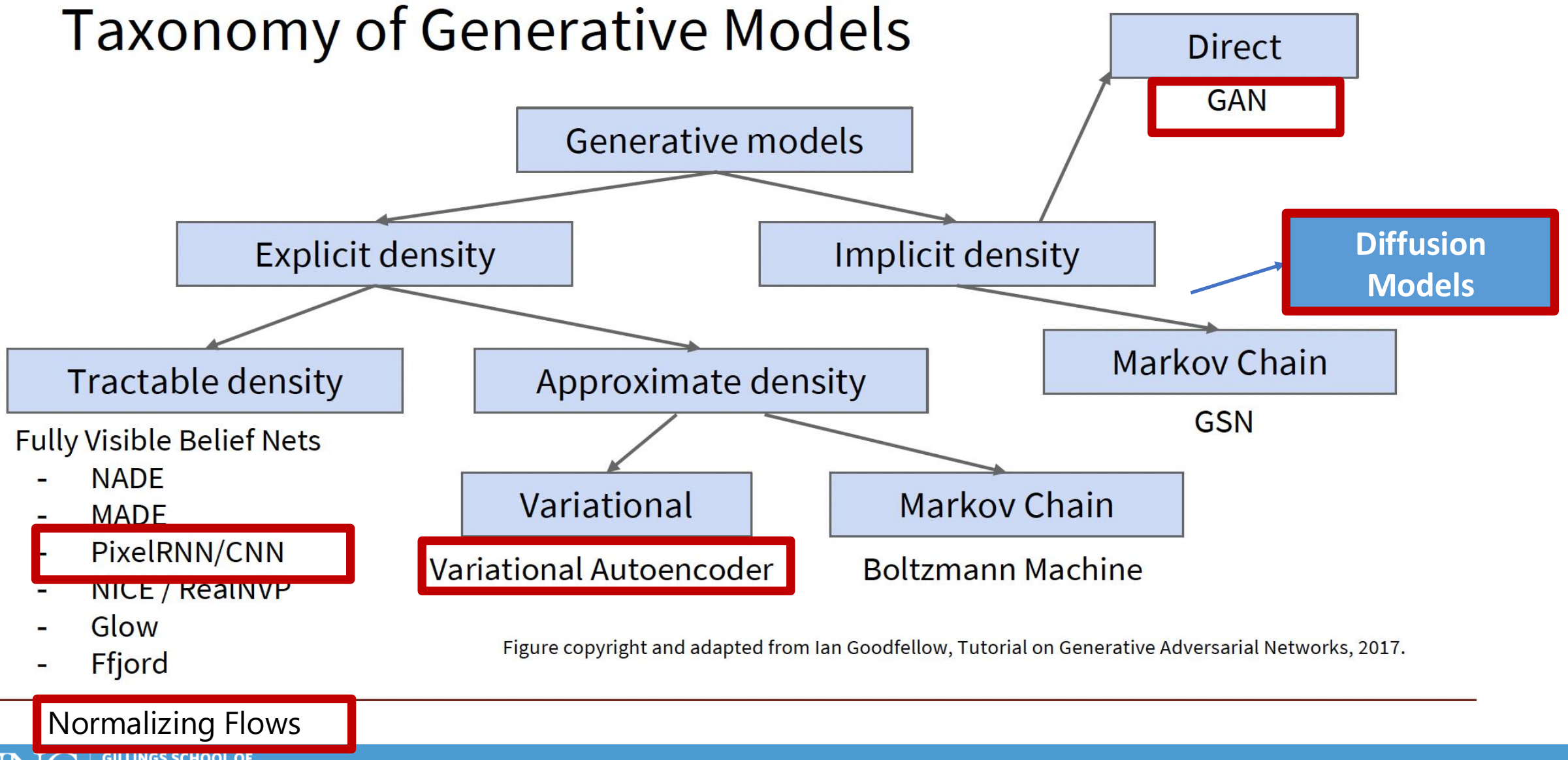
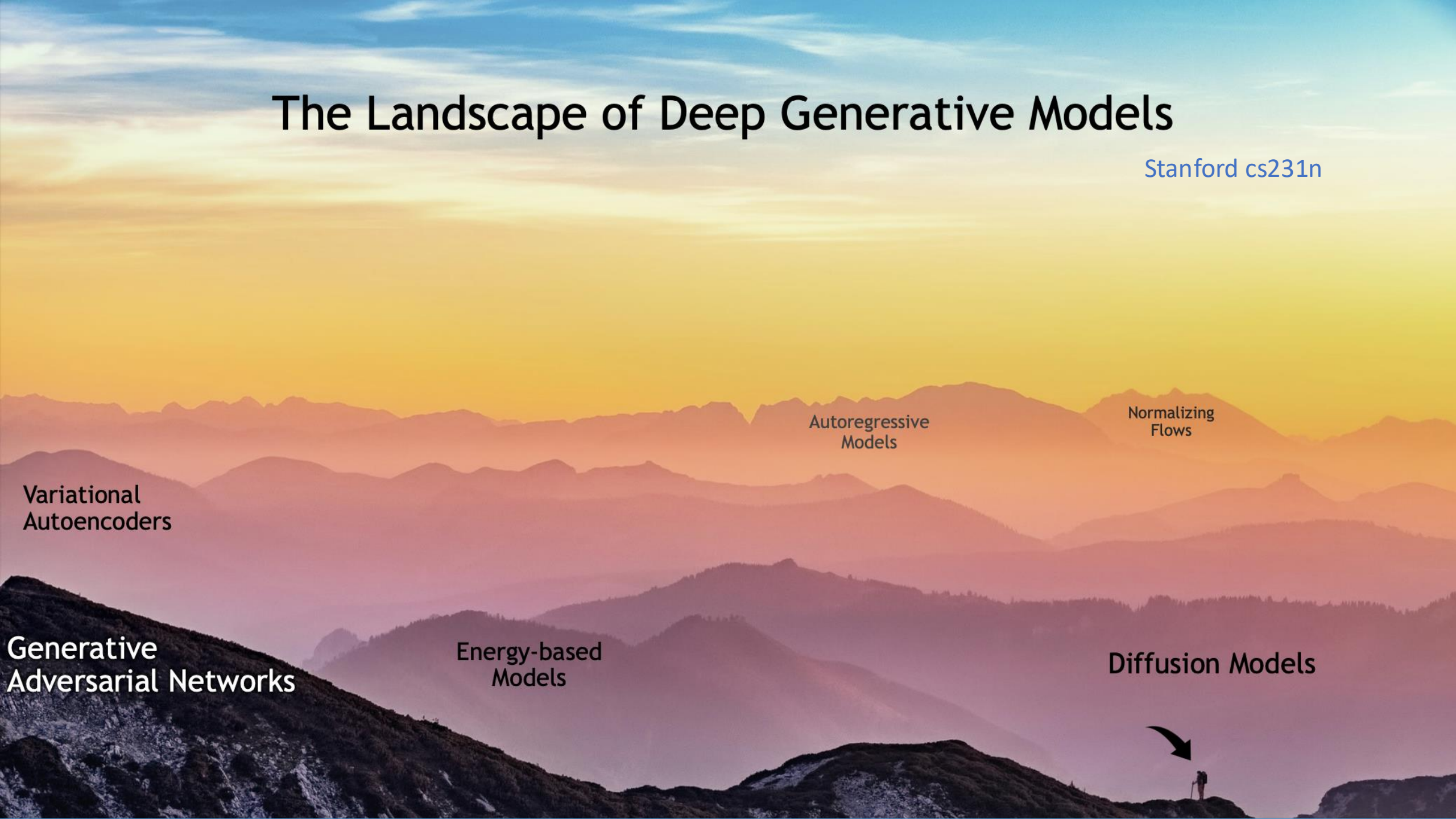
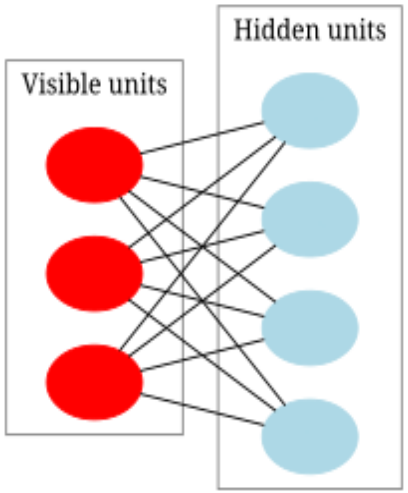


Figure copyright and adapted from Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017.


The Landscape of Deep Generative Models
Stanford cs231n
Autoregressive Models
Normalizing Flows
Variational Autoencoders
Generative Adversarial Networks
Energy-based Models
Diffusion Models

DBN and DBM

Restricted Boltzmann Machines (RBMs)



- **Energy Function:**

$$E(v,h) = -\sum_i a_i v_i - \sum_j b_j h_j - \sum_{i,j} v_i W_{ij} h_j$$

- **Probability Distribution:**

$$P(v,h) = \frac{e^{-E(v,h)}}{Z}, \quad Z = \sum_{v,h} e^{-E(v,h)}$$

$$P(v) = \frac{1}{Z} \sum_{\{h\}} e^{-E(v,h)}$$

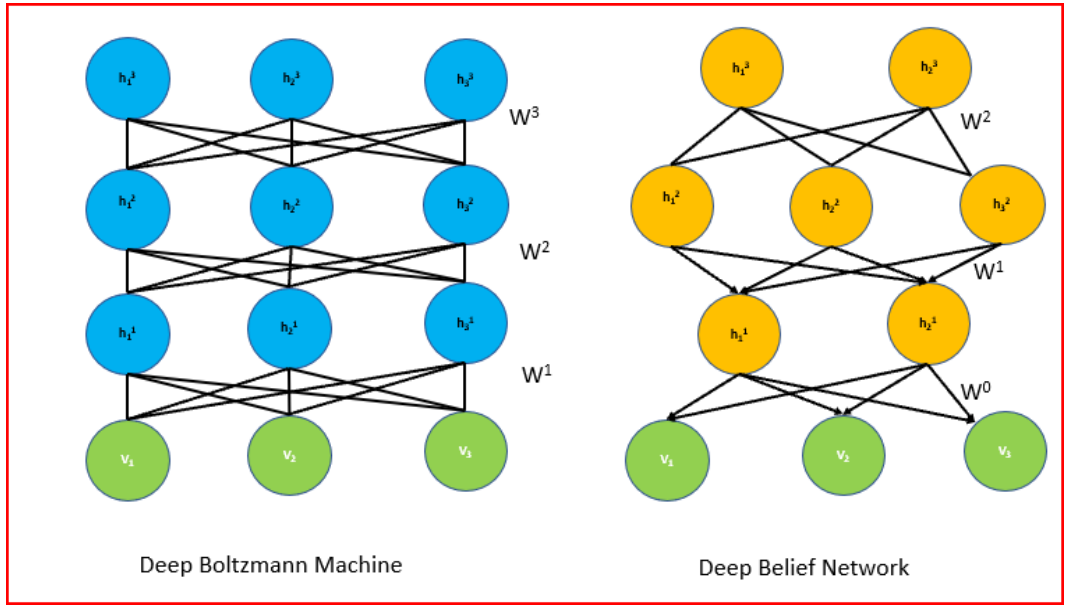
DBNs stack multiple RBMs to form a deep probabilistic model.

$$P(v, h^1, h^2, \ldots, h^L) = P(h^L) \prod_{k=1}^{L-1} P(h^k | h^{k+1}) P(v | h^1)$$

DBMs extend RBMs into a multi-layer structure, allowing deeper hierarchical representations.

$$E(v, h^1, h^2) = -\sum_i a_i v_i - \sum_j b_j h_j^1 - \sum_k c_k h_k^2$$

$$- \sum_{ij} v_i W_{ij} h_j^1 - \sum_{jk} h_j^1 U_{jk} h_k^2$$



Denoising Autoencoders (DAE)

A variant of autoencoders that learns robust representations by reconstructing clean inputs from noisy versions.

- ❖ The goal is to force the network to capture meaningful structure while ignoring noise.
- ❖ Used in image denoising, feature learning, and semi-supervised learning.

- A corrupted version is generated as $\tilde{\mathbf{x}} \sim q_D(\tilde{\mathbf{x}}|\mathbf{x})$, where noise is added.
- The encoder maps the noisy input to a latent representation:

$$\mathbf{h} = f_\theta(\tilde{\mathbf{x}}) = \sigma(W\tilde{\mathbf{x}} + b)$$

- The decoder reconstructs the original input:

$$\mathbf{r} = g_\phi(\mathbf{h}) = \sigma(W'\mathbf{h} + b')$$

- **Noise Injection:** The corruption process $q_D(\tilde{\mathbf{x}}|\mathbf{x})$ can be:
  - Gaussian noise: $\tilde{\mathbf{x}} = \mathbf{x} + \mathcal{N}(0, \sigma^2 I)$.
  - Masking: Randomly setting input values to zero.
  - Salt-and-pepper noise.
- The reconstruction error is minimized:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{x}\sim p_{data}, \tilde{\mathbf{x}}\sim q_D}\left[L(\mathbf{x}, g_\phi(f_\theta(\tilde{\mathbf{x}})))\right]$$

where $L(\mathbf{x}, \mathbf{r}) = \|\mathbf{x} - \mathbf{r}\|_2^2$.

- DAEs generate samples by reconstructing corrupted inputs:

$$\tilde{X} \sim q(\tilde{X}|X), \quad h = f_\theta(\tilde{X}), \quad \hat{X} = g_\phi(h)$$

Generative Stochastic Networks (GSNs)

•**Definition**: GSNs are a class of generative models that learn to transform noise into structured data by learning a Markov transition function. GSNs sidestep intractable partition functions.

- ❖ They generalize Denoising Autoencoders (DAEs) by introducing latent variables into the learning process.
- ❖ Training involves learning a transition function that converges to the data distribution

GSNs use the denoising autoencoder to define a **Markov chain** that generates samples. The Markov chain is defined by a transition operator $T(x' \mid x)$.

- Let x_t be the state of the Markov chain at step t
- The next state x_{t+1} is generated by:
  1. Corrupting x_t with noise: $\tilde{x}_t \sim q(\tilde{x} \mid x_t)$.
  2. Applying the denoising autoencoder to obtain $x_{t+1} = f_\theta(\tilde{x}_t)$.
- The transition operator $T(x_{t+1} \mid x_t)$ can be written as:

Iterative Markov chain with noise injection:

$$X_{t+1} = f_\theta(X_t, Z_t), \quad Z \sim p(Z)$$

$$T(x_{t+1} \mid x_t) = \int q(\tilde{x} \mid x_t)\, \delta(x_{t+1} - f_\theta(\tilde{x}))\, d\tilde{x}$$

where $\delta(\cdot)$ is the Dirac delta function.

Generative Stochastic Networks (GSNs)

To generate new samples from the GSN:

1. Start with an initial state x_0 (e.g., random noise).
2. Iteratively apply the transition operator $T(x_{t+1} \mid x_t)$ for T steps:

$$x_{t+1} = f_\theta(\tilde{x}_t), \quad \tilde{x}_t \sim q(\tilde{x} \mid x_t)$$

3. After sufficient steps, x_T will be a sample from $p_{\text{model}}(x)$.

$$\theta^* = \arg\min_\theta \mathbb{E}_{x \sim p_{\text{data}}, \tilde{x} \sim q(\tilde{x}|x)} \left[||x - f_\theta(\tilde{x})||^2 \right]$$

- p_{data} represents the true data distribution.
- $q(\tilde{x}|x)$ defines the corruption process introducing noise.
- $f_\theta(\tilde{x})$ is the model that reconstructs x from corrupted input $\tilde{x}$.

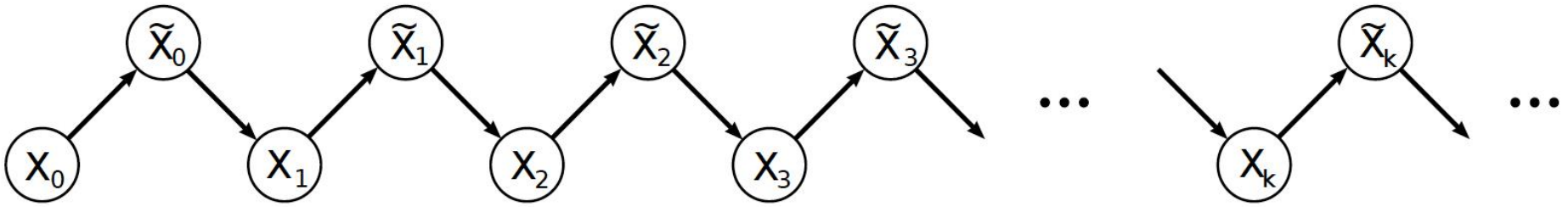
The training objective of GSNs is to minimize the reconstruction error of a denoising autoencoder while ensuring that the Markov chain converges to the desired distribution. Additionally, regularization techniques or constraints can be applied to stabilize the Markov chain during training.

Generative Stochastic Networks (GSNs)

Proposition 2 *Let $P(X)$ be the training distribution for which we only have empirical samples. Let $\pi(X)$ be the implicitly defined asymptotic distribution of the Markov chain alternating sampling from $P_\theta(X|\tilde{X})$ and $\mathcal{C}(\tilde{X}|X)$, where $\mathcal{C}$ is the original local corruption process.*

If we assume that $P_\theta(X|\tilde{X})$ has sufficient capacity and that the walkback algorithm converges (in terms of being stable in the updates to $P_\theta(X|\tilde{X})$), then $\pi(x) = P(X)$.

That is, the Markov chain defined by alternating $P_\theta(X|\tilde{X})$ and $\mathcal{C}(\tilde{X}|X)$ gives us samples that are drawn from the same distribution as the training data.



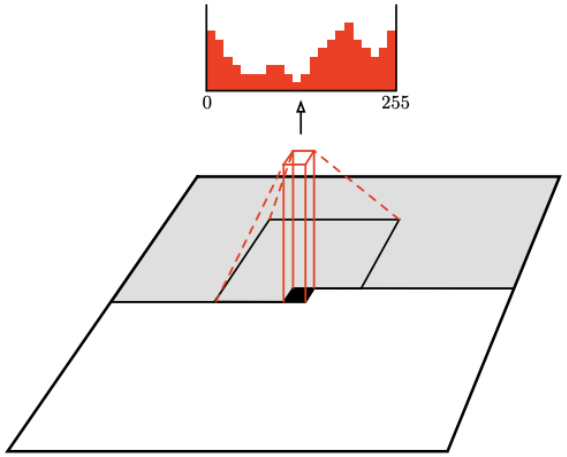
Alain, Guillaume, Yoshua Bengio, Li Yao, Jason Yosinski, Eric Thibodeau-Laufer, Saizheng Zhang, and Pascal Vincent. "GSNs: generative stochastic networks." *Information and Inference: A Journal of the IMA* 5, no. 2 (2016): 210-249.

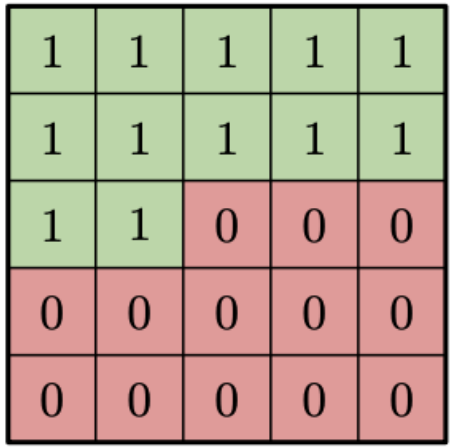
PixelRNN/PixelCNN

Autoregressive models predict the probability of an image as a sequential product of pixel conditionals:

- Each pixel is generated sequentially, conditioned on all previously generated pixels.
- PixelRNN and PixelCNN differ in how they model these dependencies.

$$p(\mathbf{x}) = \prod_{i=1}^{N} p(x_i | x_1, x_2, \ldots, x_{i-1})$$





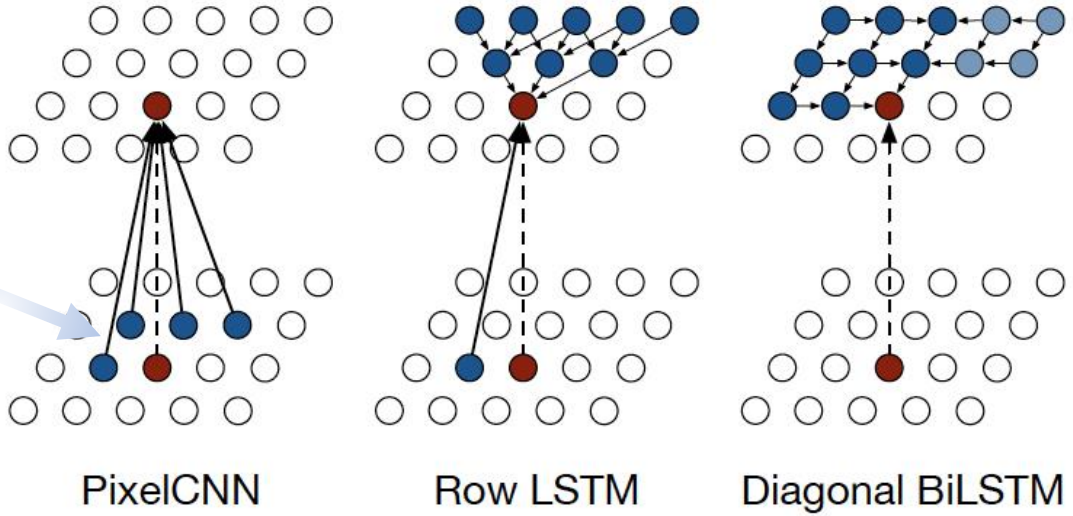
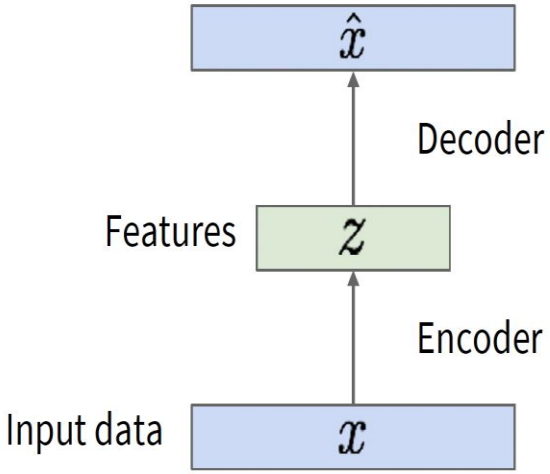


Figure 4. Visualization of the input-to-state and state-to-state mappings for the three proposed architectures.

Variational Autoencoder

Key Idea:

- Encoder: $q(z \mid x) \approx$ posterior.
- Decoder: $p(x \mid z)$.
- Prior: $p(z) = \mathcal{N}(0, I)$.



Objective (ELBO):

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q(z|x)}[\log p(x \mid z)] - \text{KL}(q(z \mid x) \| p(z))$$

Training:

- Maximize ELBO using stochastic gradient descent.

Encoder/Decoder:

$$q(z \mid x) = \mathcal{N}(z; \mu(x), \sigma^2(x)), \quad p(x \mid z) = \mathcal{N}(x; \mu(z), \sigma^2(z))$$

KL Divergence:

$$\text{KL}(q(z \mid x) \| p(z)) = \frac{1}{2}\left(\text{tr}(\sigma^2(x)) + \|\mu(x)\|^2 - d - \log\det(\sigma^2(x))\right)$$

Reparameterization Trick:

$$z = \mu(x) + \sigma(x) \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

Generative Adversarial Network (GANs)

Key Idea:

- Two networks compete:
  - Generator G: Maps noise $z \sim p_z(z)$ to $G(z)$.
  - Discriminator D: Outputs $D(x) \in [0, 1]$.

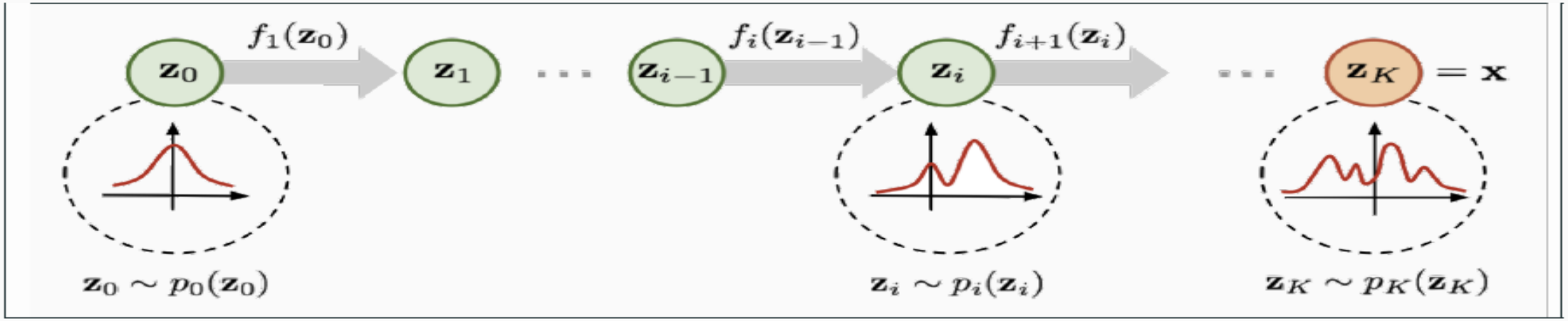
Objective:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z \sim p_z}[\log(1 - D(G(z)))]$$

Training:

1. Update D to maximize $V(D, G)$.
2. Update G to minimize $V(D, G)$.

Normalizing Flow



- **A flow-based generative model** takes samples of x and learns $f_1^{-1}, f_2^{-1}, \cdots f_k^{-1}$ such that

$$z_0 = f_1^{-1} \circ f_2^{-1} \circ \cdots \circ f_K^{-1}(x)$$

- Once the functions are learned, it uses

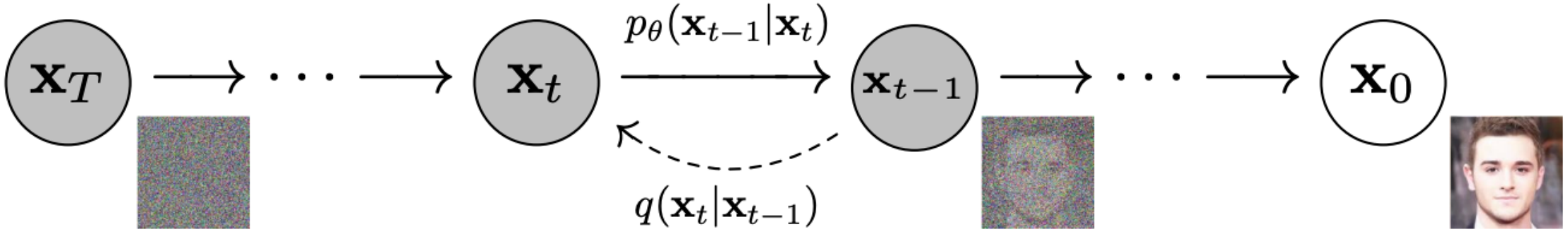
$$x = f_k \circ f_{k-1} \circ \cdots \circ f_1(z_0)$$

to approximate the distribution of x

A normalizing flow learns a sequence of invertible maps so that data can be transformed into simple Gaussian noise (for training and likelihood), and Gaussian noise can be transformed back into realistic samples (for generation).

Discrete time diffusion models

- Forward diffusion process $q(x_t|x_{t-1})$, gradually adds Gaussian noise to an image, until you end up with pure noise.



- Reverse denoising diffusion process $p_\theta(x_{t-1}|x_t)$, where a neural network is trained to gradually denoise an image starting from pure noise, until you end up with an actual image.

Score Matching Langevin Dynamics

SMLD solves this by learning a neural approximation to the score and plugging it back into Langevin dynamics.

Unknown quantity

$$\nabla_x \log p(x)$$

The target density p(x) is inaccessible, so the exact score is also inaccessible.

Learn a neural score model

$$s_\theta(x) \approx \nabla_x \log p(x)$$

Train s_θ directly from samples rather than modeling a normalized density explicitly.

Sampling recursion after training

$$x_{t+1} = x_t + \tau s_\theta(x_t) + \sqrt{2\tau}\, z$$

This is the practical SMLD pipeline: learn the score first, then sample by stochastic updates.

One-slide summary of the pipeline

Training samples from data distribution → **Train score network $s_\theta(x)$** → **Plug score into Langevin update** → **Generate samples from noise**

- It compares three routes: **explicit score matching**, **implicit score matching**, **and denoising score matching (DSM)**.
- The theorems later in the chapter explain why DSM becomes the most interpretable and practical route.

Comparisons among DGMs

Model	Quality	Diversity	Pros	Cons
DBN	Moderate	Moderate	Efficient pretraining, useful for feature extraction	Not a true generative model, lacks flexibility
DBM	High	High	Captures complex dependencies, deep representation learning	Hard to train, requires MCMC sampling
DAE	Moderate	Low	Effective for representation learning, robust to corruption	Not a true generative model, lacks explicit sampling mechanism
GSN	High	High	No intractable partition function, stable training	Requires well-tuned Markov transitions
PixelRNN / PixelCNN	Very High	Low	High-quality samples, avoids adversarial training issues	Slow sampling (PixelRNN), limited long-range dependencies (PixelCNN)
GAN	Very High	Low	Generates highly realistic images, fast sampling	Mode collapse, unstable training
VAE	Moderate	High	Well-defined latent space, meaningful representations	Blurry image samples, over-regularized latent space

- GANs and Pixel-based models achieve the highest quality but can struggle with diversity.
- VAEs and GSNs offer high diversity but often produce lower quality outputs.
- DBMs and GSNs allow complex feature learning but are harder to train than DBNs.
- PixelCNN is faster and more stable than PixelRNN due to parallel processing.
- DAEs and VAEs are better for representation learning but are not ideal for direct sample generation.

Comparisons among DGMs

Model Type	Sample Quality	Diversity	Pros	Cons
Diffusion Models	★★★★★ (state-of-the-art)	★★★★★	Stable training, high fidelity, strong conditioning	Slow sampling, heavy compute
Score-Based Models	★★★★★	★★★★★	Flexible SDEs, unified theory, strong coverage	Slow, complex training
Normalizing Flows	★★★	★★★	Tractable likelihood, fast sampling, invertible	Architecturally constrained, lower fidelity

Evaluation Metrics

Metric	Measures	Best for	Limitations
Inception Score (IS)	Quality & diversity	Image GANs	Doesn't compare to real data
Fréchet Inception Distance (FID)	Realism & diversity	Image GANs	Requires feature extraction
Precision & Recall	Fidelity & coverage	Any model	Computationally expensive
Log-Likelihood	Probability assignment	VAEs, Flows	Doesn't match human perception
Human Evaluation	Subjective quality	Any model	Expensive and subjective
Downstream Task Performance	Utility in real tasks	Task-driven models	Domain-dependent

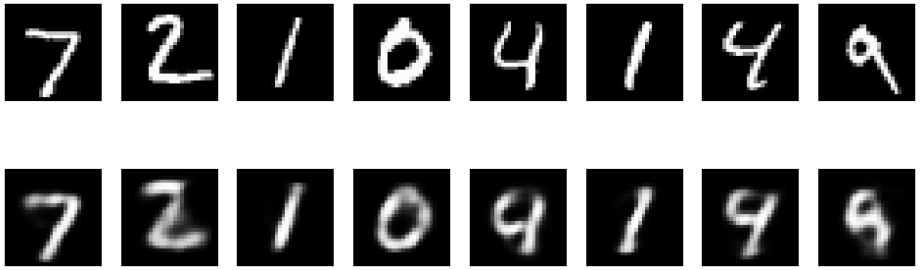
Applications of Generative Models in AI

- **Understanding Probability Distributions**
  - Generative models help represent and manipulate high-dimensional probability distributions across various fields.
- **Role in Reinforcement Learning (RL)**
  - Used in model-based RL to simulate possible futures for planning & decision-making.
  - Enables learning in imaginary environments, reducing risks of real-world errors.
  - Guides exploration by tracking visited states & attempted actions.
  - Supports inverse RL for learning from expert demonstrations.
- **Handling Missing Data & Semi-Supervised Learning**
  - Can train with missing data and predict missing inputs.
  - Enables semi-supervised learning, reducing the need for labeled data.
- **Multi-Modal Learning & Sample Generation**
  - Allows multiple correct outputs for a single input (e.g., video frame prediction).
  - GANs excel in generating realistic samples for various AI applications.

Model Results

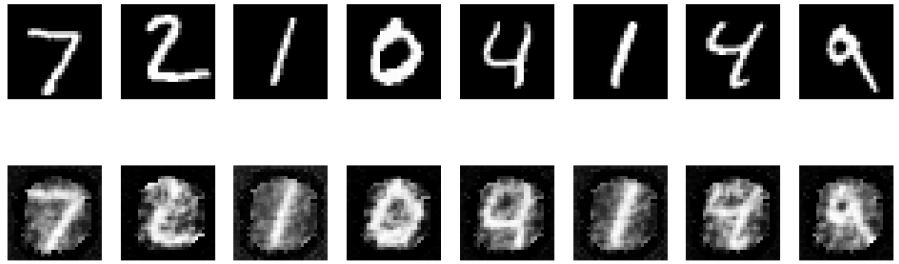
Restricted Boltzmann Machine

Real (Top) vs. RBM Generated (Bottom)



Graph-Structured Network

Real (Top) vs. GSN Generated (Bottom)

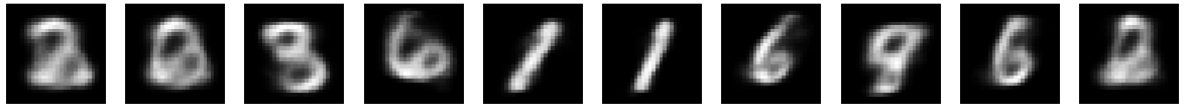


Denoising Autoencoder

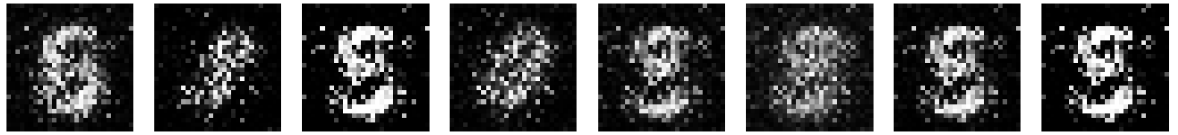
Real (Top) vs. DAE Generated (Bottom)

VAE (generated from noise)

VAE Generated Samples



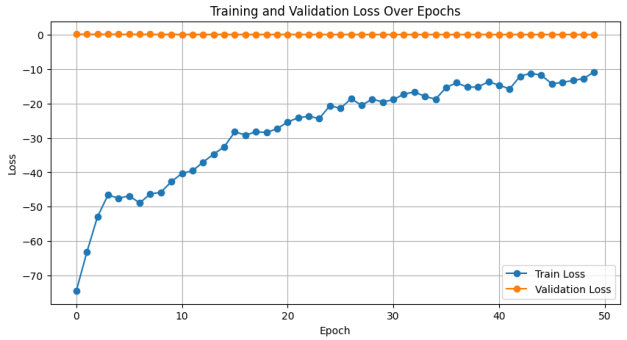
GAN (generated from noise)



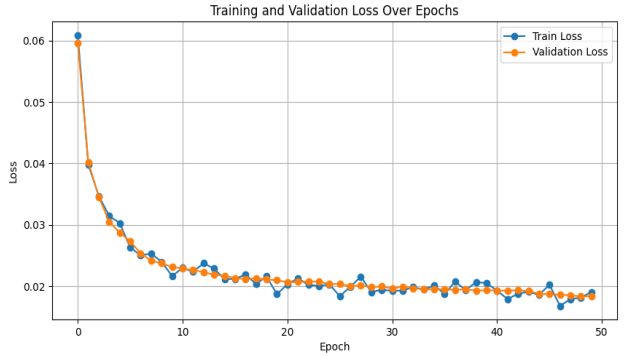
Training & Validation Loss

train_loss =
contrastive-divergence
free-energy difference

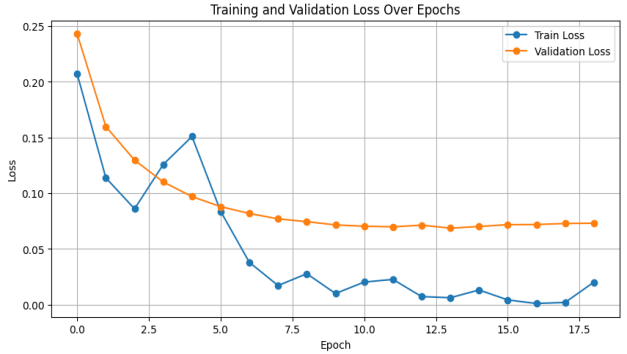
Restricted Boltzmann Machine



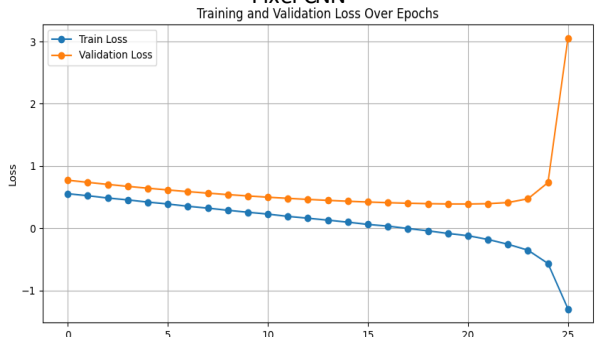
Graph-Structured Network



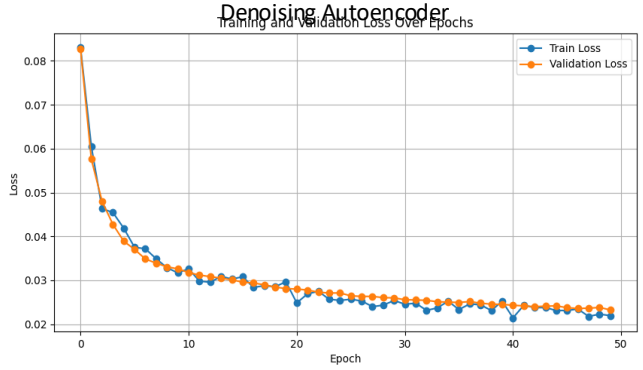
Deep Belief Network (multi-layer RBM - classifier only)



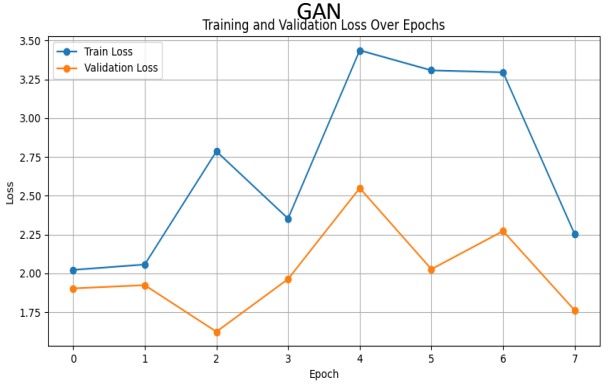
Pixel CNN



Denoising Autoencoder



GAN



Model	Test_loss
RBM	0.025985
DBN	0.064641
DAE	0.010321
GSN	0.599898
PIXELCNN	2.370674

Content

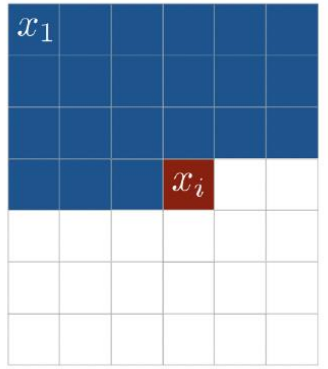
3 PixelDNN and Variational Autoencoder

Fully visible belief network (FVBN)

Definition: A Fully Visible Belief Network (FVBN) is a directed probabilistic model where the joint probability of observed variables is factorized using the chain rule of probability.

$$p(x) = p(x_1, x_2, \ldots, x_n) = \prod_{i=1}^{n} p(x_i | x_1, \ldots, x_{i-1})$$

Probability of the i-th pixel value given all previous pixels



Complex distribution over pixel values => Express using a neural network!

Key Features

- ✅ **Autoregressive structure**: Each variable is modeled sequentially.
- ✅ **Exact likelihood estimation**: Unlike GANs, FVBNs provide explicit probability distributions.
- ✅ **No latent variables**: Unlike VAEs or DBNs, FVBNs do not rely on hidden representations.

Applications

- 📌 **Generative Modeling** – Used in density estimation tasks.
- 📌 **Sequential Data** – Applied in speech and language models.
- 📌 **Autoregressive Image Models** – Used in PixelCNN-like architectures.

PixelRNN

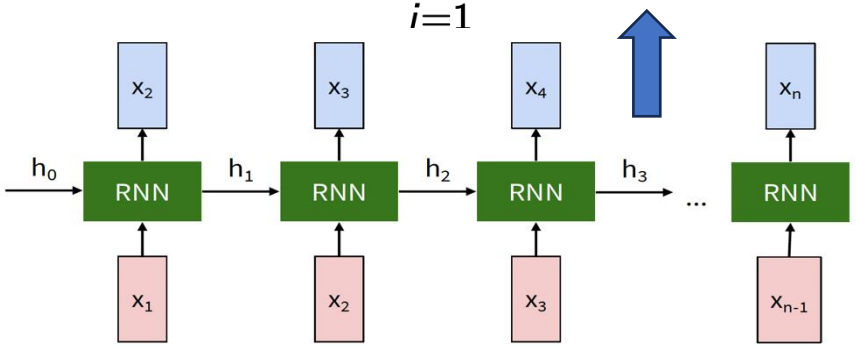
van den Oord et al. (2016).

What is PixelRNN?

- A generative autoregressive model that predicts pixels sequentially along spatial dimensions.
- Used for image generation, inpainting, and density estimation.

Pixel-Level Autoregressive Model

$$P(X) = \prod_{i=1}^{n^2} P(X_i | X_{1:i-1})$$



Color Image

$$P(X_i) = P(X_{i,R}|X_{<i})P(X_{i,G}|X_{<i}, X_{i,R})P(X_{i,B}|X_{<i}, X_{i,R}, X_{i,G})$$

Architecture of PixelRNN

Recurrent Layers (LSTM)

- Row LSTM: Processes the image row-by-row.
- Diagonal BiLSTM: Processes diagonal bands for improved efficiency.

Output Layer

- Uses a softmax layer to predict pixel intensities.

PixelRNN

- Uses LSTMs to process images sequentially.
- Captures long-range dependencies.
- Computationally expensive

Row LSTM

$$h_{i,j} = \text{LSTM}(x_{i,j}, h_{i-1,j})$$

Diagonal BiLSTM

$$h_{i,j} = \text{LSTM}(x_{i,j}, h_{i-1,j-1}, h_{i,j-1}, h_{i-1,j})$$

PixelRNN

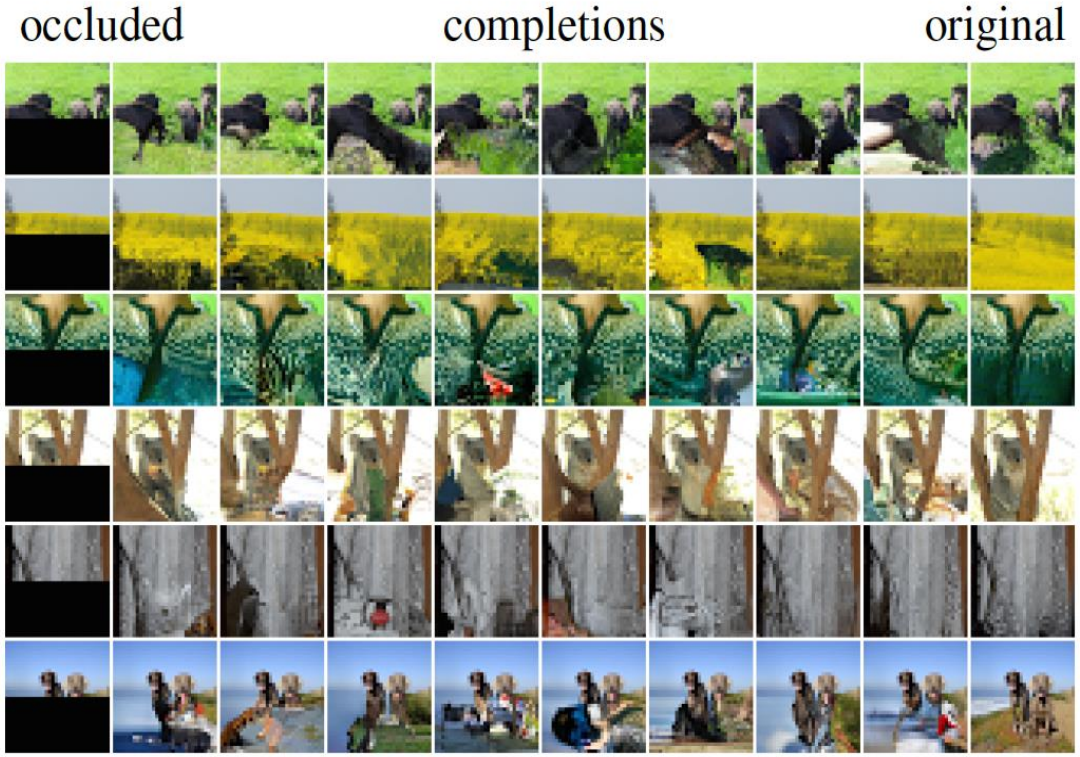
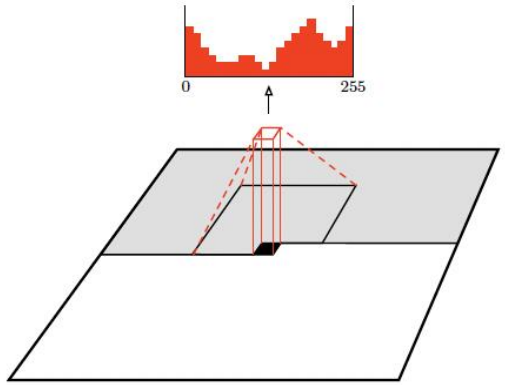


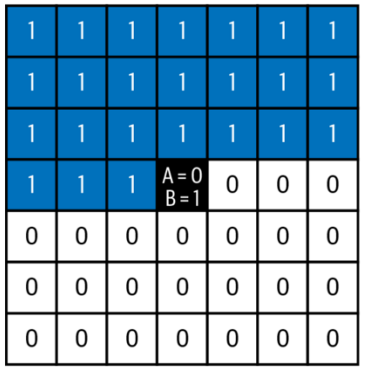
Image completions sampled from a model that was trained on 32x32 ImageNet images (van den Oord et al. (2016)).

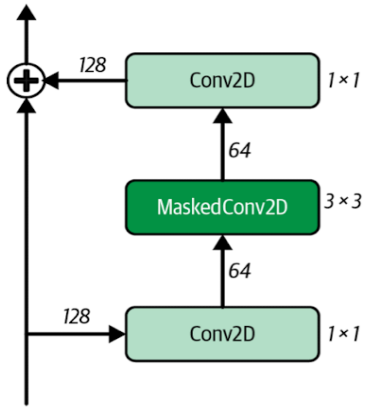
PixelCNN

What is PixelCNN?

- A convolution-based generative model for image synthesis.
- Trains using autoregressive likelihood estimation.
- Faster than PixelRNN due to convolutional structure.
- Efficient for real-world image generation tasks.







- Mask A: Ensures pixels don't see themselves.
- Mask B: Allows flow of information across layers.

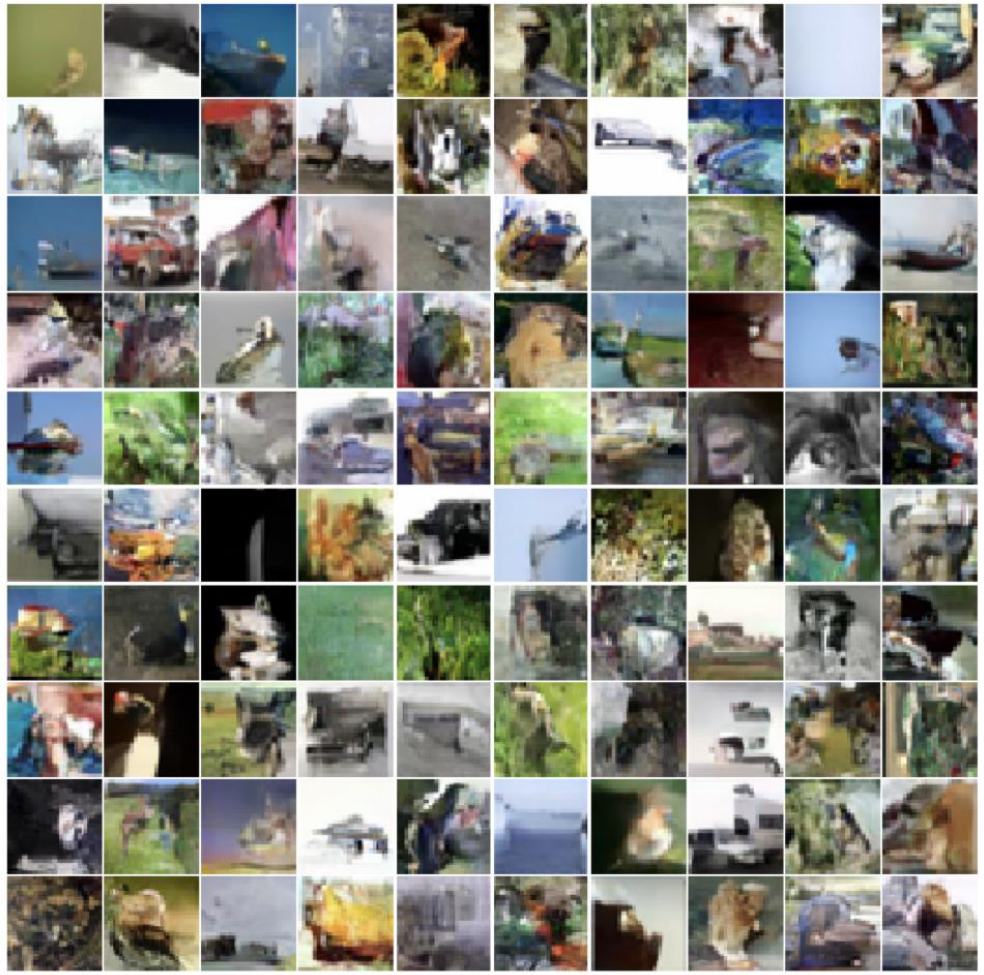
Architecture of PixelCNN

```
Model: "model"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
input_1 (InputLayer)         [(None, 28, 28, 1)]       0
pixel_conv_layer (PixelConv  (None, 28, 28, 128)       6400
Layer)
residual_block (ResidualBlo  (None, 28, 28, 128)       98624
ck)
residual_block_1 (ResidualB  (None, 28, 28, 128)       98624
lock)
residual_block_2 (ResidualB  (None, 28, 28, 128)       98624
lock)
residual_block_3 (ResidualB  (None, 28, 28, 128)       98624
lock)
residual_block_4 (ResidualB  (None, 28, 28, 128)       98624
lock)
pixel_conv_layer_6 (PixelCo  (None, 28, 28, 128)       16512
nvLayer)
pixel_conv_layer_7 (PixelCo  (None, 28, 28, 128)       16512
nvLayer)
conv2d_18 (Conv2D)           (None, 28, 28, 1)         129
=================================================================
Total params: 532,673
Trainable params: 532,673
Non-trainable params: 0
_________________________________________________________________
```

Mask A

Mask B

PixelCNN+++



Model	Bits per sub-pixel
Deep Diffusion (Sohl-Dickstein et al., 2015)	5.40
NICE (Dinh et al., 2014)	4.48
DRAW (Gregor et al., 2015)	4.13
Deep GMMs (van den Oord & Dambre, 2015)	4.00
Conv DRAW (Gregor et al., 2016)	3.58
Real NVP (Dinh et al., 2016)	3.49
PixelCNN (van den Oord et al., 2016b)	3.14
VAE with IAF (Kingma et al., 2016)	3.11
Gated PixelCNN (van den Oord et al., 2016c)	3.03
PixelRNN (van den Oord et al., 2016b)	3.00
PixelCNN++	**2.92**

What are Autoencoders?

Autoencoders are neural networks designed for dimensionality reduction and feature extraction by compressing and reconstructing data.

They consist of two main components:

- **Encoder (e)**: Maps input x to a **low-dimensional latent space** z, where similar inputs have similar latent representations.

  e: X→Z, z=e(x) with dim(X)≫dim(Z)

- **Decoder (d)**: Reconstructs x from its latent representation z, mapping back to the original input space.

  d: Z→X and $\hat{x} = d(z) = d(e(x))$.

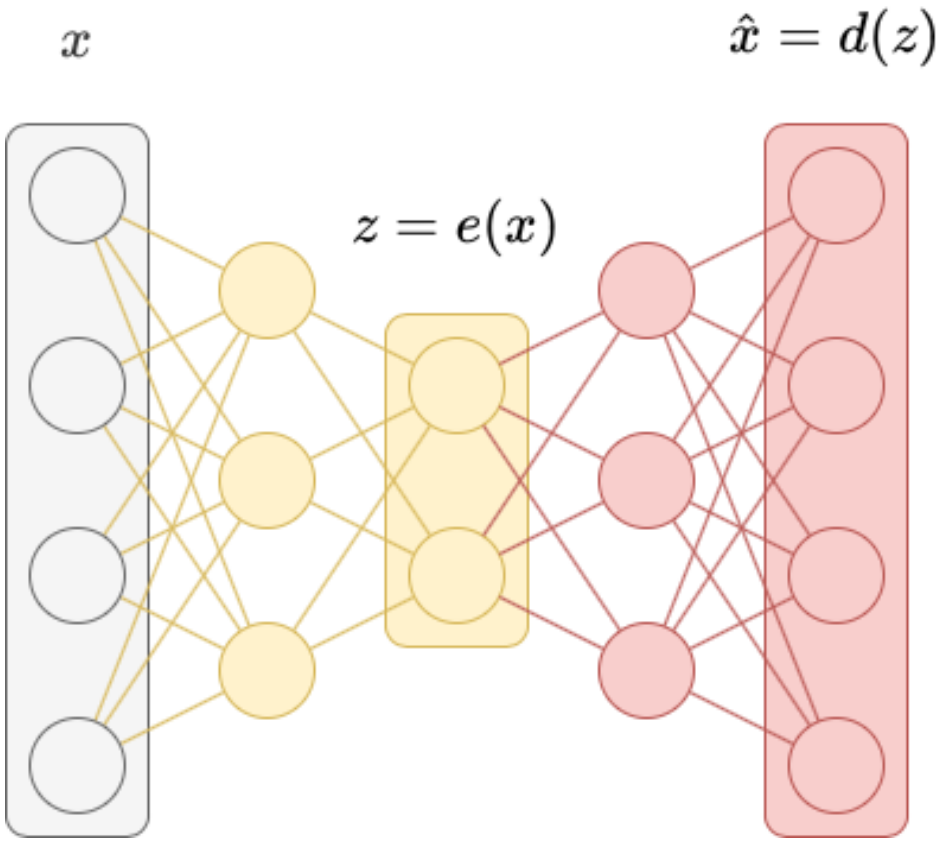
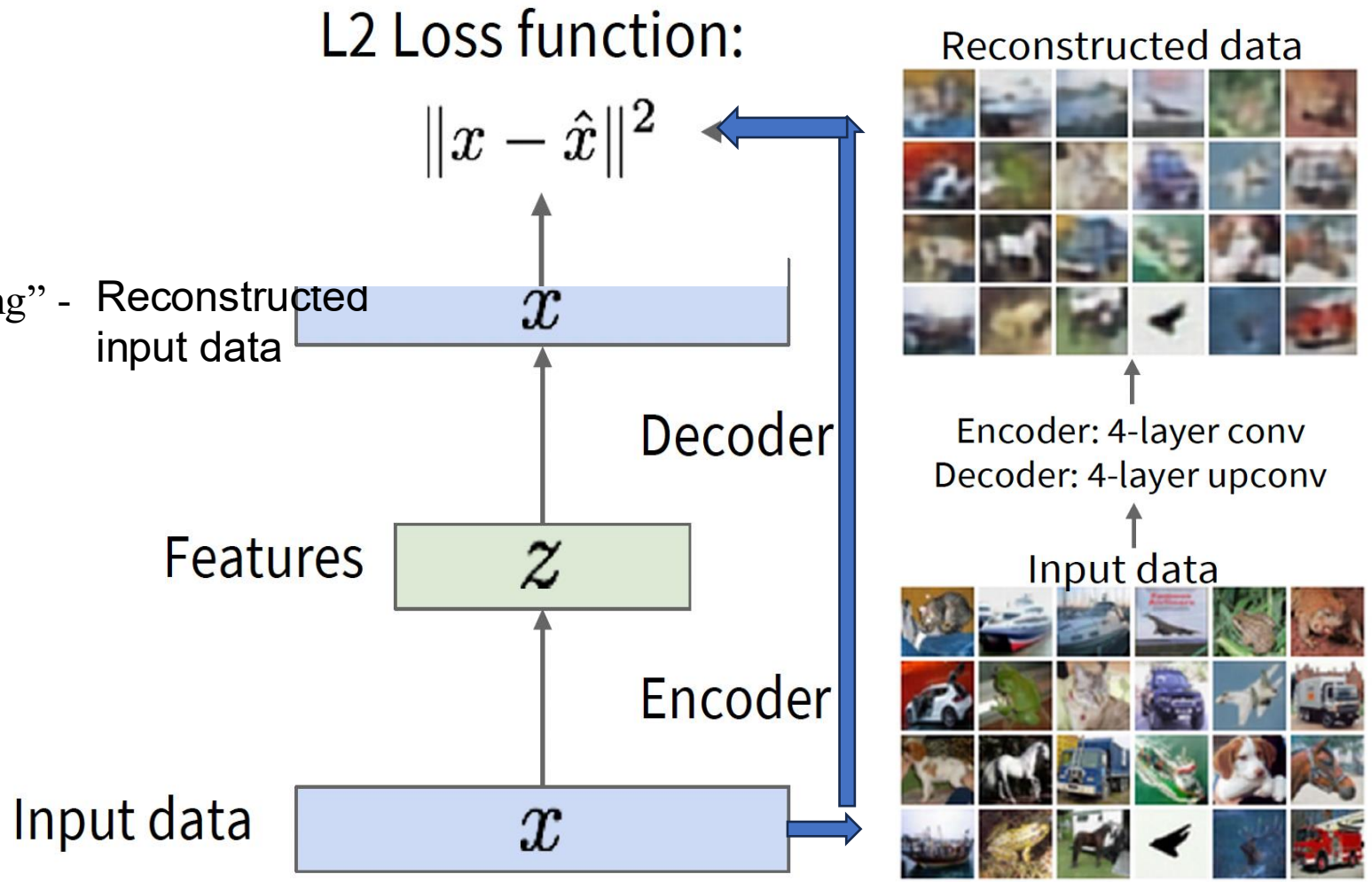


Illustration of autoencoder (source)

What are Autoencoders?

Train such that features can be used to reconstruct original data "Autoencoding" - encoding input itself

Want features to capture meaningful factors of variation in data



Autoencoder Latent Space and Its Limitations

- **Trained on MNIST**, the autoencoder clusters similar digits in the **latent space**.
- **Decoder can reconstruct images** from latent vectors, but gaps in the latent space cause issues.
- **Generative models** aim to produce new samples, but **disjoint latent spaces** in autoencoders make some sampled latent vectors meaningless.
- **Illustration:** In the **top-left corner of the latent space**, unseen regions result in unrealistic reconstructions.
- **Solution: Variational Autoencoders (VAEs)** introduce structured latent spaces to ensure continuity and improve generative performance.

📌 **Key Issue:** Autoencoders are great for representation learning but struggle as generative models due to fragmented latent spaces.

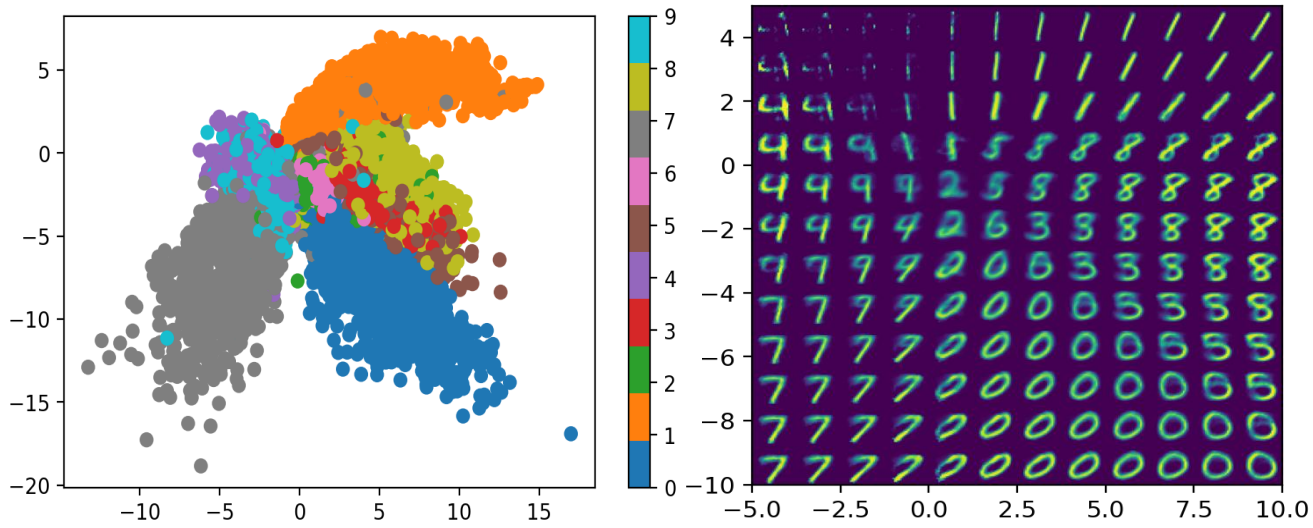


Illustration of example latent vectors using the MNIST dataset (source)

What is a Variational Autoencoder?

📌 **VAE = Autoencoder + Generative Modeling**

- **Same structure as a traditional autoencoder:**
  - ❖ **Encoder:** Compresses input into a latent space representation, but instead of a single point, outputs a probability distribution (Gaussian).
  - ❖ **Decoder:** Samples from this distribution and reconstructs the input.

📌 **Key Difference from Traditional Autoencoders**

❖ Traditional autoencoders map inputs deterministically to a single latent vector z=e(x).

❖ VAEs introduce probabilistic encoding, ensuring smooth and structured latent spaces for better generative performance.

✨ Benefit: Enables meaningful interpolation and sampling for generating new data! 🚀

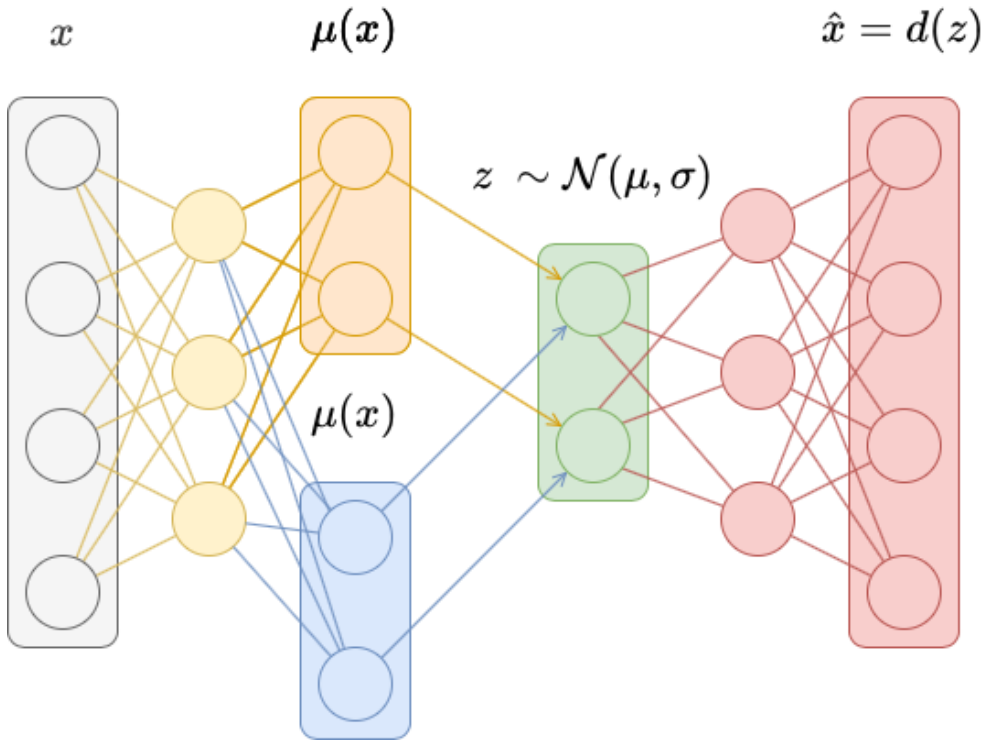


Illustration of VAE (source)

Variational Autoencoder as a DGM

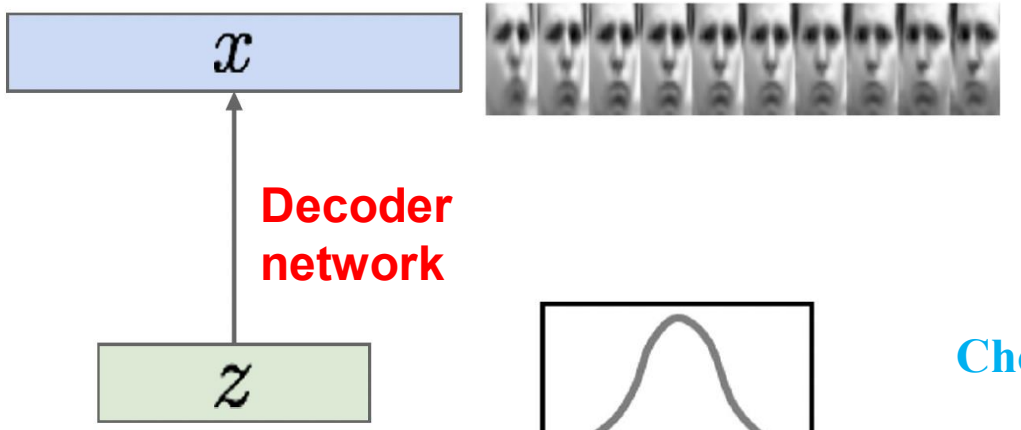
VAEs define an intractable density function with latent

$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$$

Probabilistic spin on autoencoders - will let us sample from the model to generate data!

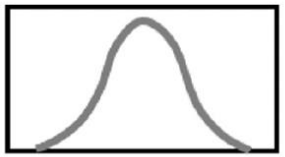
Assume training data $\{x^{(i)}\}_{i=1}^{N}$ is generated from the distribution of unobserved (latent) representation z

Sample from true conditional $p_{\theta^*}(x \mid z^{(i)})$



Conditional p(x|z) is complex (generates image) => represent with neural network

Sample from true prior $z^{(i)} \sim p_{\theta^*}(z)$



Choose prior p(z) to be simple, e.g. Gaussian.

How to train VGE?

Learn model parameters to maximize likelihood of training data

We want to estimate the true parameters of this generative model given training data $\{x^{(i)}\}_{i=1}^{N}$

Q: What is the problem with this? Intractable!

Data Likelihood

$$p_\theta(x) = \int p_\theta(z) p_\theta(x|z) dz$$

$$\log p(x) \approx \log \frac{1}{k} \sum_{i=1}^{k} p(x|z^{(i)}), \text{where } z^{(i)} \sim p(z)$$

Intractable to compute p(x|z) for every z!

Monte Carlo estimation is too high variance

Posterior distribution

$$p_\theta(z|x) = p_\theta(x|z) p_\theta(z) / p_\theta(x)$$

Solution: In addition to decoder network modeling $p_\theta(x|z)$, define additional encoder network

$$q_\theta(z|x) \approx p_\theta(z|x)$$

Will see that this allows us to derive a lower bound on the data likelihood that is tractable, which we can optimize.

How to approximate VGE?

$$\log p_\theta(x^{(i)}) = \mathbf{E}_{z\sim q_\phi(z|x^{(i)})}\left[\log p_\theta(x^{(i)})\right] \quad (p_\theta(x^{(i)}) \text{ Does not depend on } z)$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Bayes' Rule)}$$

$$= \mathbf{E}_z\left[\log \frac{p_\theta(x^{(i)} \mid z)p_\theta(z)}{p_\theta(z \mid x^{(i)})}\frac{q_\phi(z \mid x^{(i)})}{q_\phi(z \mid x^{(i)})}\right] \quad \text{(Multiply by constant)}$$

$$= \mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z)}\right] + \mathbf{E}_z\left[\log \frac{q_\phi(z \mid x^{(i)})}{p_\theta(z \mid x^{(i)})}\right] \quad \text{(Logarithms)}$$

$$= \mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z)) + D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z \mid x^{(i)}))$$

Decoder network gives pθ(x|z), can compute estimate of this term through sampling (need some trick to differentiate through sampling).

This KL term (between Gaussians for encoder and z prior) has nice closed-form solution!

pθ(z|x) intractable (saw earlier), can't compute this KL term. But we know KL divergence always >= 0.

How to approximate VGE?

Decoder: reconstruct the input data

Encoder: make approximate posterior distribution. close to prior

$$\log p_\theta(x^{(i)}) = \underbrace{\mathbf{E}_z\left[\log p_\theta(x^{(i)} \mid z)\right] - D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z))}_{\mathcal{L}(x^{(i)}, \theta, \phi)} + \underbrace{D_{KL}(q_\phi(z \mid x^{(i)}) \,||\, p_\theta(z \mid x^{(i)}))}_{\geq 0}$$

We want to maximize the data likelihood.

Tractable lower bound which we can take gradient of and optimize!
($p\theta(x|z)$ differentiable, KL term differentiable)

Variational evidence lower bound (ELBO):

$$\mathcal{L}(x, \theta, \phi) \leq \log p_\theta(x)$$

Training: Maximize lower bound

$$\hat{\theta}, \hat{\phi} = \arg\max_{\theta,\phi} \sum_{i=1}^{n} \mathcal{L}(x_i, \theta, \phi)$$

Stochastic Optimization of ELBO

Algorithm 1: Stochastic optimization of the ELBO. Since noise originates from both the minibatch sampling and sampling of $p(\boldsymbol{\epsilon})$, this is a doubly stochastic optimization procedure. We also refer to this procedure as the *Auto-Encoding Variational Bayes* (AEVB) algorithm.

Data:

$\mathcal{D}$: Dataset

$q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})$: Inference model

$p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z})$: Generative model

Result:

$\boldsymbol{\theta}, \boldsymbol{\phi}$: Learned parameters

$(\boldsymbol{\theta}, \boldsymbol{\phi}) \leftarrow$ Initialize parameters

while *SGD not converged* **do**

  $\mathcal{M} \sim \mathcal{D}$ (Random minibatch of data)

  $\boldsymbol{\epsilon} \sim p(\boldsymbol{\epsilon})$ (Random noise for every datapoint in $\mathcal{M}$)

  Compute $\tilde{\mathcal{L}}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathcal{M}, \boldsymbol{\epsilon})$ and its gradients $\nabla_{\boldsymbol{\theta},\boldsymbol{\phi}} \tilde{\mathcal{L}}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathcal{M}, \boldsymbol{\epsilon})$

  Update $\boldsymbol{\theta}$ and $\boldsymbol{\phi}$ using SGD optimizer

end

$$\mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathbf{x}) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) - \log q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})\right]$$

$$\nabla_{\boldsymbol{\theta}} \mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathbf{x}) = \nabla_{\boldsymbol{\theta}} \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) - \log q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})\right]$$

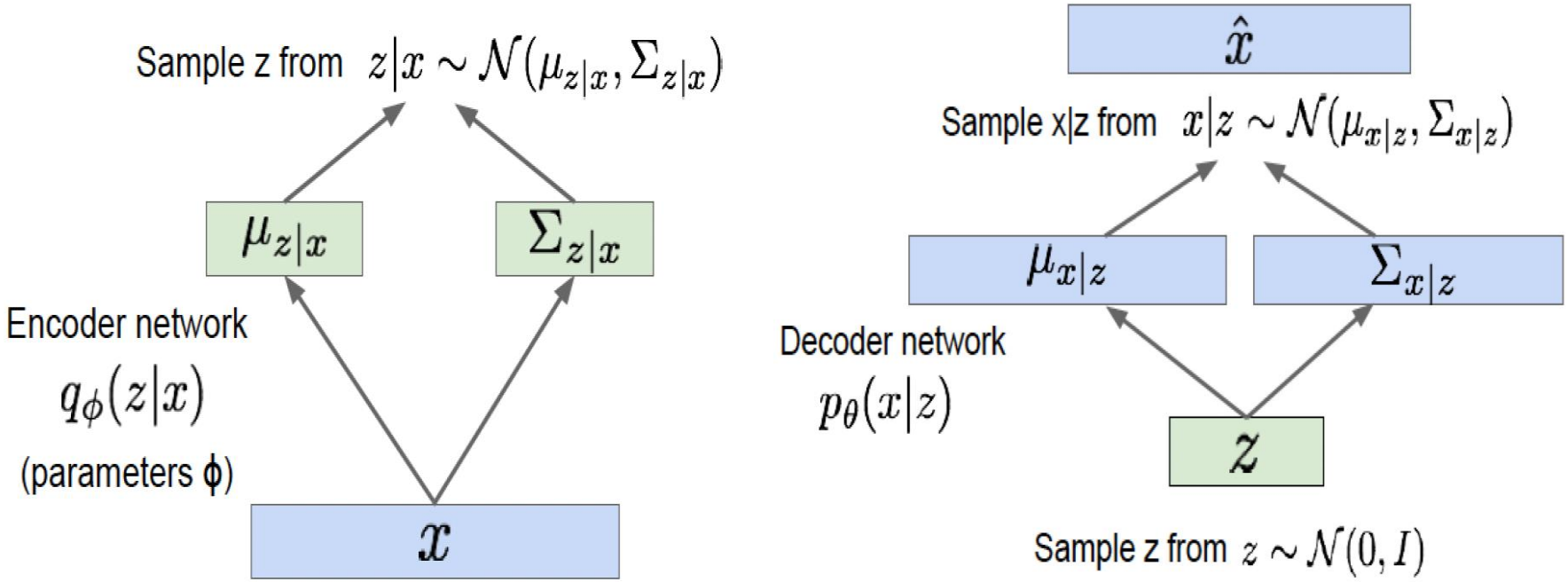
$$\nabla_{\boldsymbol{\phi}} \mathcal{L}_{\boldsymbol{\theta},\boldsymbol{\phi}}(\mathbf{x}) = \nabla_{\boldsymbol{\phi}} \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[\log p_{\boldsymbol{\theta}}(\mathbf{x}, \mathbf{z}) - \log q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})\right]$$

Reparametrization Trick: $\mathbf{z} = \mathbf{g}(\boldsymbol{\epsilon}, \boldsymbol{\phi}, \mathbf{x})$

$$\mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[f(\mathbf{z})\right] = \mathbb{E}_{p(\boldsymbol{\epsilon})}\left[f(\mathbf{z})\right]$$

$$\begin{aligned}\nabla_{\boldsymbol{\phi}} \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{z}|\mathbf{x})}\left[f(\mathbf{z})\right] &= \nabla_{\boldsymbol{\phi}} \mathbb{E}_{p(\boldsymbol{\epsilon})}\left[f(\mathbf{z})\right] \\ &= \mathbb{E}_{p(\boldsymbol{\epsilon})}\left[\nabla_{\boldsymbol{\phi}} f(\mathbf{z})\right] \\ &\simeq \nabla_{\boldsymbol{\phi}} f(\mathbf{z})\end{aligned}$$

A Theoretical Example



$$\int q_{\boldsymbol{\theta}}(\mathbf{z}) \log p(\mathbf{z})\, d\mathbf{z} = \int \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}, \boldsymbol{\sigma}^2) \log \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})\, d\mathbf{z}$$

$$= -\frac{J}{2} \log(2\pi) - \frac{1}{2} \sum_{j=1}^{J} (\mu_j^2 + \sigma_j^2)$$

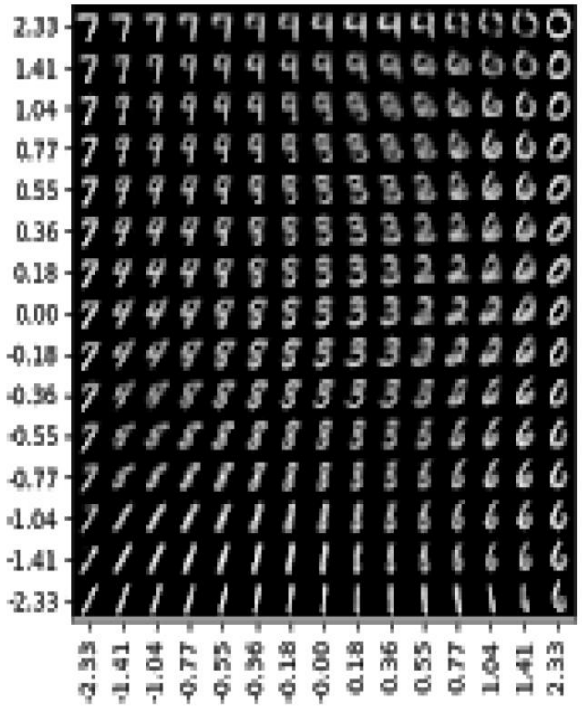
$$\log p(\mathbf{x}|\mathbf{z}) = \log \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\sigma}^2 \mathbf{I})$$

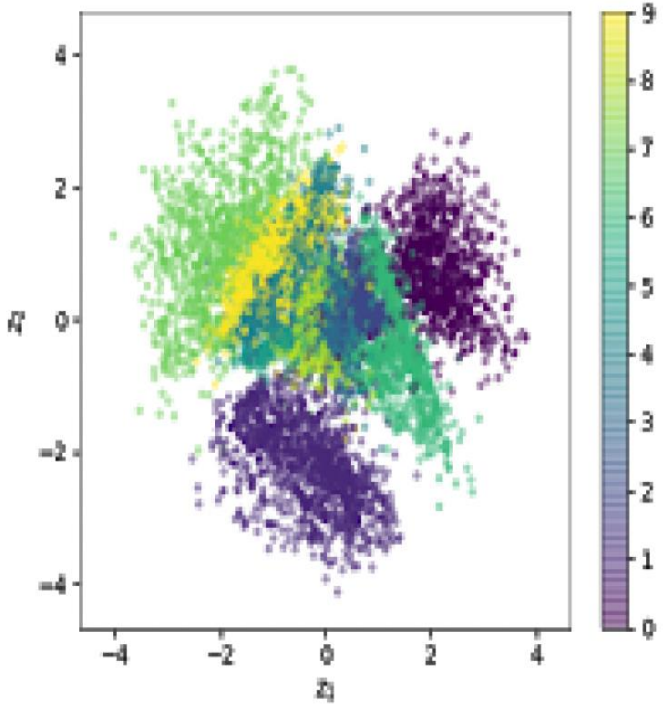
$$\text{where } \boldsymbol{\mu} = \mathbf{W}_4 \mathbf{h} + \mathbf{b}_4$$

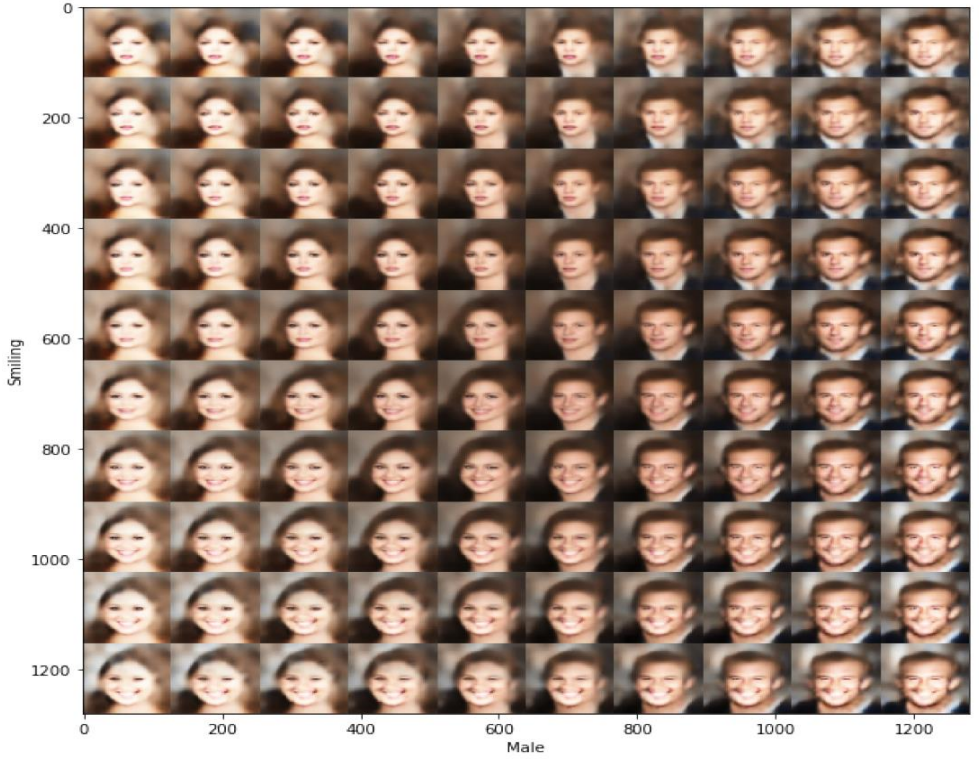
$$\log \boldsymbol{\sigma}^2 = \mathbf{W}_5 \mathbf{h} + \mathbf{b}_5$$

$$\mathbf{h} = \tanh(\mathbf{W}_3 \mathbf{z} + \mathbf{b}_3)$$

Real Examples







Code: Vanilla VAE

```
# ----- Define the Encoder & Decoder -----
# The encoder outputs two vectors: mu (μ) — the mean of the latent distribution & logvar (logσ²) — the log-variance of
the latent distribution. Reparameterization Trick is used: z = μ + exp(½ logσ²) ⊙ϵ, ϵ ~ N(0,1)
# The decoder maps the latent vector z back to the original data space. We will use a final Sigmoid to produce pixel
intensities in [0,1].
class VAE(nn.Module):
    def __init__(self, input_dim, hidden_dim, latent_dim):
        super(VAE, self).__init__()
        # Encoder
        self.enc_fc1 = nn.Linear(input_dim, hidden_dim)
        self.enc_fc2_mu = nn.Linear(hidden_dim, latent_dim)
        self.enc_fc2_logvar = nn.Linear(hidden_dim, latent_dim)
        # Decoder
        self.dec_fc1 = nn.Linear(latent_dim, hidden_dim)
        self.dec_fc2 = nn.Linear(hidden_dim, input_dim)
        self.relu = nn.ReLU()
        self.sigmoid = nn.Sigmoid()
    def encoder(self, x):
        """Encode the input into latent parameters (mu, logvar)."""
        h = self.relu(self.enc_fc1(x))
        mu = self.enc_fc2_mu(h)
        logvar = self.enc_fc2_logvar(h)
        return mu, logvar
    def reparameterize(self, mu, logvar):
        """Reparameterization trick to sample z."""
        std = torch.exp(0.5 * logvar)
        eps = torch.randn_like(std)  # same shape as std
        z = mu + eps * std
        return z
```

Code: Vanilla VAE

```
    def decoder(self, z):
        """Decode the latent vector into reconstructed input."""
        h = self.relu(self.dec_fc1(z))
        x_recon = self.sigmoid(self.dec_fc2(h))
        return x_recon

    def forward(self, x):
        """Forward pass: encoder -> reparam -> decoder."""
        mu, logvar = self.encoder(x)
        z = self.reparameterize(mu, logvar)
        x_recon = self.decoder(z)
        return x_recon, mu, logvar

# ----- Define the loss function -----
# The total loss is the sum of Reconstruction Loss: Typically we use Binary Cross Entropy between the reconstructed image and the original image (scaled to [0,1]) and KL Divergence Loss: Encourages the approximate posterior qϕ(z|x) to be close to the prior p(z)=N(0,I).
def loss_function(x_recon, x, mu, logvar):
    # Reconstruction loss (assuming x, x_recon in [0, 1])
    bce = nn.functional.binary_cross_entropy(
        x_recon, x, reduction='sum'
    )  # sum over all pixels
    # KL Divergence
    # KL(N(mu, sigma^2) || N(0,1))
    # = 0.5 * sum(exp(logvar) + mu^2 - 1 - logvar)
    kl = 0.5 * torch.sum(torch.exp(logvar) + mu**2 - 1.0 - logvar)
    return bce + kl
```

Strengths & Limitations

📌 **Key Idea:**

- Adds a probabilistic spin to traditional autoencoders, enabling data generation.
- Defines an intractable density, requiring variational inference to derive and optimize a lower bound (ELBO).

📌 **Pros:**

✓ Principled generative approach based on probabilistic modeling.
✓ Interpretable latent space enables meaningful structure in representations.
✓ Inference of q(z|x) allows feature extraction for other tasks.

📌 **Cons:**

❌ Optimizes a lower bound on likelihood, which may not be an ideal evaluation metric.
❌ Lower sample quality compared to PixelRNN/PixelCNN.
❌ Blurry reconstructions compared to GANs, which generate sharper images.

📌 **Active Research Areas:**

🔷 Flexible Approximate Posteriors: Moving beyond diagonal Gaussian assumptions to richer models like Gaussian Mixture Models (GMMs) or Categorical Distributions.
🔷 Disentangled Representations: Learning independent latent factors for better interpretability.
🔷 Improving Training Objectives: Hybrid models incorporating adversarial learning (VAE-GANs).
🚀 **Future Directions**: Enhancing sample quality while retaining VAE's structured latent space!

Content

4 GANs and their Architectures

“This (GANS), and the variations that are now being proposed is the most interesting idea in the last 10 years in ML, in my opinion”

–Yann LeCun

What is GAN?

Problem:

- ❖ We want to sample from a complex, high-dimensional training distribution.
- ❖ There is no direct way to explicitly learn or model the data distribution.

Solution:

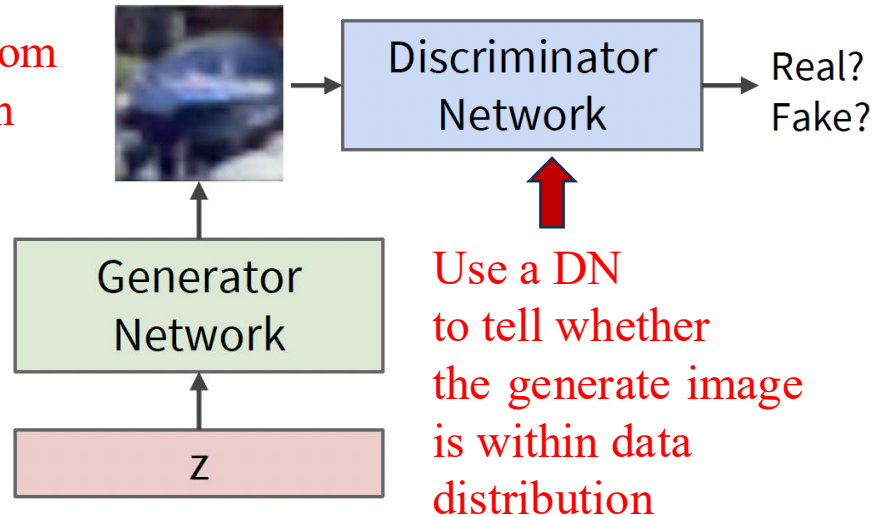
- ➢ Instead of learning the distribution explicitly, GANs learn a transformation.
- ➢ Start by sampling from a simple distribution (e.g., Gaussian noise).
- ➢ Train a neural network to transform the simple distribution into the training data distribution.

Key Idea:

- ✓ GANs learn to generate new samples indirectly through adversarial training.
- ✓ The model never explicitly estimates the probability density function of the data.

Objective: generated images should look "real"

Output: Sample from training distribution



Use a DN to tell whether the generate image is within data distribution ("real") or not

Input: Random noise

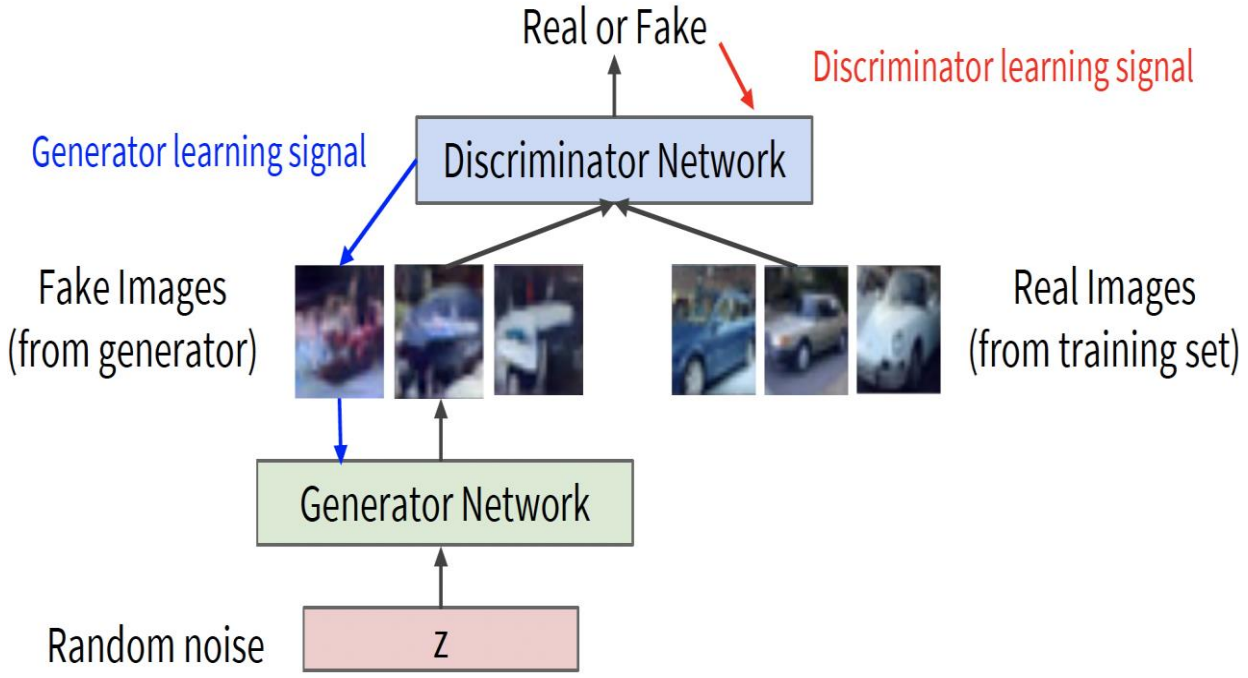
Generative Vs Discriminative Models

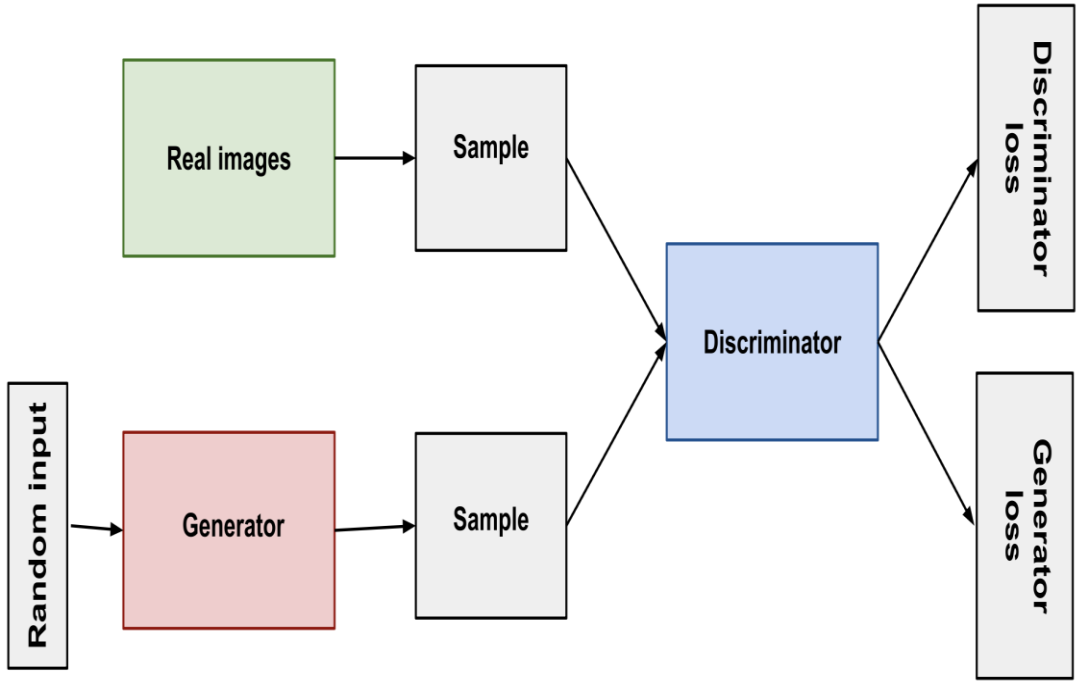
Generative Models:

- Can generate new data samples resembling real data.
- Example: GANs generate realistic images that resemble real ones.

Discriminative Models:

- Focus on classification by distinguishing between different categories.
- Example: A decision tree can classify dogs and cats but cannot generate them.





Overview of GAN Training

Discriminator Network: Tries to distinguish between real and fake images.
Generator Network: Tries to fool the discriminator by generating real-looking images.
Training Process: Both networks are trained jointly in a minimax game.

Minimax objective function:

$$\min_{\theta_g} \max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

Generator objective

Discriminator objective

Discriminator output for real data x

Discriminator output for generated fake data G(z)

- **Discriminator** (θ_d) wants to maximize the objective such that $D(x) \approx 1$ (real) and $D(G(z)) \approx 0$ (fake).
- **Generator** (θ_g) wants to minimize the objective such that $D(G(z)) \approx 1$ (fooling the discriminator).

GAN Training

Alternate between:

1. Gradient ascent on discriminator

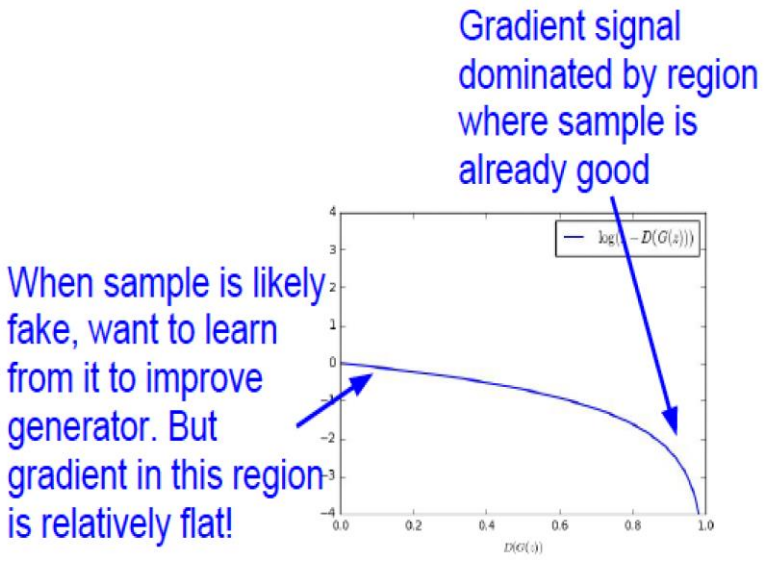
$$\max_{\theta_d} \left[\mathbb{E}_{x \sim p_{data}} \log D_{\theta_d}(x) + \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z))) \right]$$

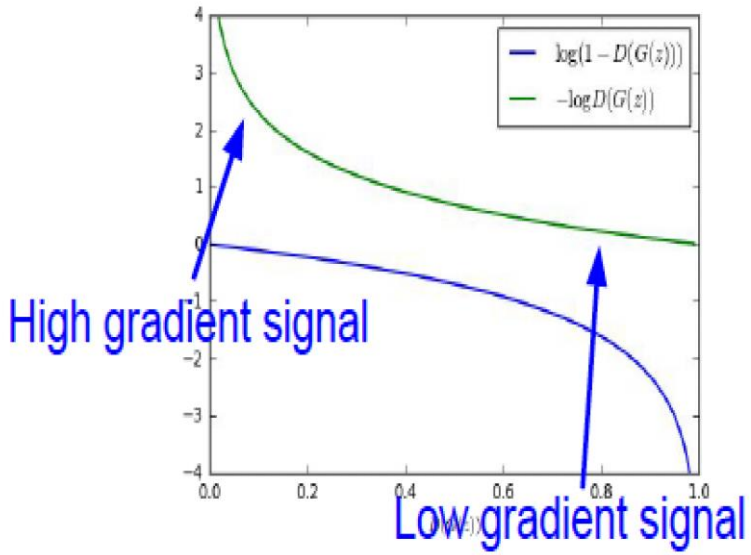
2. Gradient descent on generator

$$\min_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(1 - D_{\theta_d}(G_{\theta_g}(z)))$$

Instead: Consider a different objective

$$\max_{\theta_g} \mathbb{E}_{z \sim p(z)} \log(D_{\theta_d}(G_{\theta_g}(z)))$$





GAN Algorithms

for number of training iterations **do**

for k steps **do**

- Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Sample minibatch of m examples $\{\boldsymbol{x}^{(1)}, \ldots, \boldsymbol{x}^{(m)}\}$ from data generating distribution $p_{\text{data}}(\boldsymbol{x})$.
- Update the discriminator by ascending its stochastic gradient:

$$\nabla_{\theta_d} \frac{1}{m} \sum_{i=1}^{m} \left[\log D_{\theta_d}(x^{(i)}) + \log(1 - D_{\theta_d}(G_{\theta_g}(z^{(i)}))) \right]$$

end for

- Sample minibatch of m noise samples $\{\boldsymbol{z}^{(1)}, \ldots, \boldsymbol{z}^{(m)}\}$ from noise prior $p_g(\boldsymbol{z})$.
- Update the generator by ascending its stochastic gradient (improved objective):

$$\nabla_{\theta_g} \frac{1}{m} \sum_{i=1}^{m} \log(D_{\theta_d}(G_{\theta_g}(z^{(i)})))$$

end for

Alternating Training for GANs

GAN Training Process (Alternating Phases):

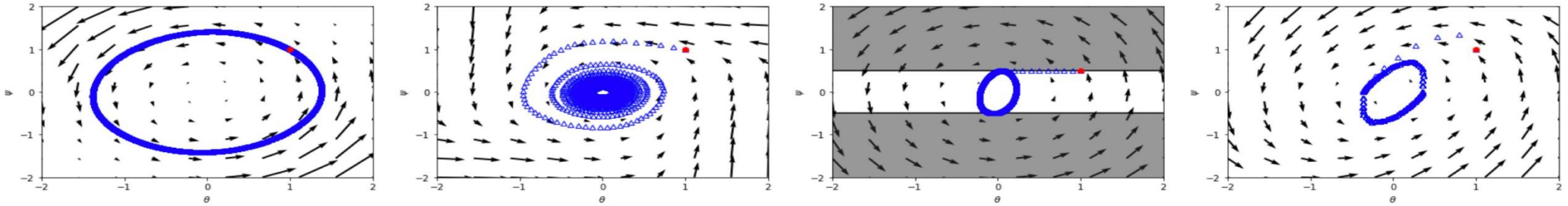
1. Discriminator Training: Trains for one or more epochs while the generator remains unchanged. It learns to differentiate real from generated data, adapting to the generator's flaws.
2. Generator Training: Trains for one or more epochs while the discriminator remains unchanged. This prevents the generator from chasing a moving target.

Training Dynamics:

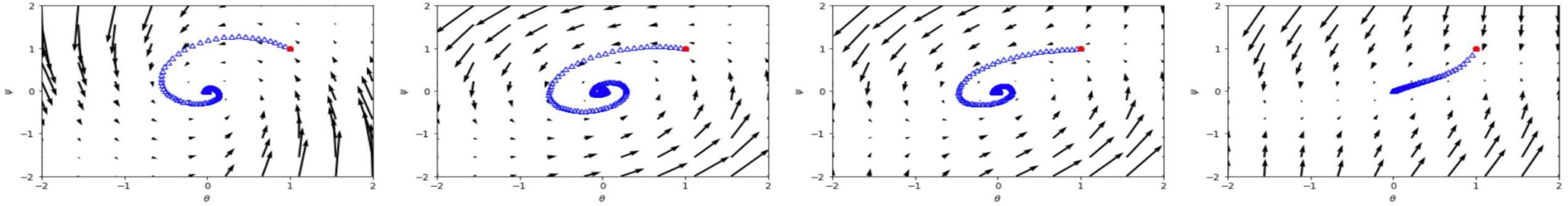
- As the generator improves, the discriminator struggles to distinguish real from fake data.
- A perfect generator results in a discriminator with 50% accuracy (random guessing).
- Overtraining can degrade performance, leading to unstable convergence where the generator receives meaningless feedback.

Challenges in GAN Training

- **Hyperparameter Sensitivity:** GANs are sensitive to learning rates, batch sizes, and architectural choices.
- **Mode Collapse:** The generator produces limited diversity.
- **Training Instability:** The minimax optimization is difficult to balance.
- **Vanishing/Exploding Gradients:** The discriminator can become too strong or weak, leading to poor gradients.
- **Non-Convergence:** The model oscillates instead of converging.



(a) Standard GAN (b) Non-saturating GAN (c) WGAN ($n_d = 5$) (d) WGAN-GP ($n_d = 5$)



(e) Consensus optimization (f) Instance noise (g) Gradient penalty (h) Gradient penalty (CR)

Hyperparameter Sensitivity

1. Adjust Learning Rates Carefully

- ❖ **Learning Rate (α)**: Too high → instability, Too low → slow convergence.
- ❖ **Two-Timescale Update Rule (TTUR)**: Use a smaller learning rate for the generator than the discriminator to balance training.

2. Tune Adam Hyperparameters

- ➢ Standard settings **(β1=0.9, β2=0.999)** can lead to oscillations.
- ➢ For **GANs**, reducing β1 to **0.5** improves stability.

3. Normalize Inputs and Use Spectral Normalization

- ✓ Normalize training images between **[−1,1]** instead of [0,1] (for Tanh activation).
- ✓ Use **Spectral Normalization** on the discriminator to control weight magnitudes.

4. Improve Loss Functions

- ○ **Wasserstein Loss (WGAN)**: Uses Earth-Mover distance for better gradient behavior.
- ○ **Gradient Penalty (WGAN-GP)**: Adds stability and prevents exploding gradients:

5. Use Progressive Training

- ❑ **Start with low-resolution images**, gradually increasing resolution (used in **Progressive Growing GANs**).
- ❑ Helps GAN **learn simple features first** before complex details.

Hyperparameter Sensitivity

6. Apply Regularization Techniques

- **Batch Normalization**: Helps control variance, but can cause mode collapse in GANs.
- **Instance Normalization**: Often more stable than batch normalization.
- **Dropout in Discriminator**: Helps prevent overfitting.

7. Monitor Convergence and Use Early Stopping

- **Track GAN metrics** (FID, Inception Score) instead of just loss values.
- **Avoid overtraining**: If the discriminator gets too strong, **freeze it temporarily**.

8. Use Larger Batch Sizes

- GANs often benefit from **larger batch sizes** (e.g., 128–512) to stabilize updates.
- **Gradient accumulation** can be used if GPU memory is limited.

9. Data Augmentation

- **Apply transformations (rotation, flipping, color jitter)** to make training more robust.
- Prevents the discriminator from memorizing training data.

10. Experiment with Alternative Architectures

- **Self-Attention GANs (SAGAN)**: Improves global structure modeling.
- **BigGAN**: Uses **larger batch sizes** and **orthogonal regularization** for stability.

Mode Collapse

Mode Collapse in GANs refers to a common failure mode where the **generator** fails to capture the full diversity of the data distribution and produces **limited variations** of samples. Instead of generating a wide range of outputs, it collapses to generating a few or even a single type of sample repeatedly.

Why Does Mode Collapse Occur?

- **Imbalanced Generator-Discriminator Learning**
- **Training Instability**
- **Lack of Diversity-Promoting Mechanisms**

1.

Effects of Mode Collapse

- **Reduced Sample Diversity** → Poor representation of the real dataset.
- **Low-Quality Generation** → Outputs look repetitive and lack variety.
- **Unreliable Model** → The generator fails to generalize.

Illustration of example monotonous output. (source)

Techniques to Mitigate Mode Collapse

1.Minibatch Discrimination

Encourages diversity by comparing samples in each batch.

2.Feature Matching

Instead of just fooling the discriminator, the generator learns to match feature statistics of real data.

3.Wasserstein GAN (WGAN)

Uses the Earth Mover (Wasserstein) distance to **stabilize training** and avoid collapsing to few modes.

4.Unrolled GANs

Allows the generator to anticipate discriminator updates, preventing it from getting stuck in mode collapse.

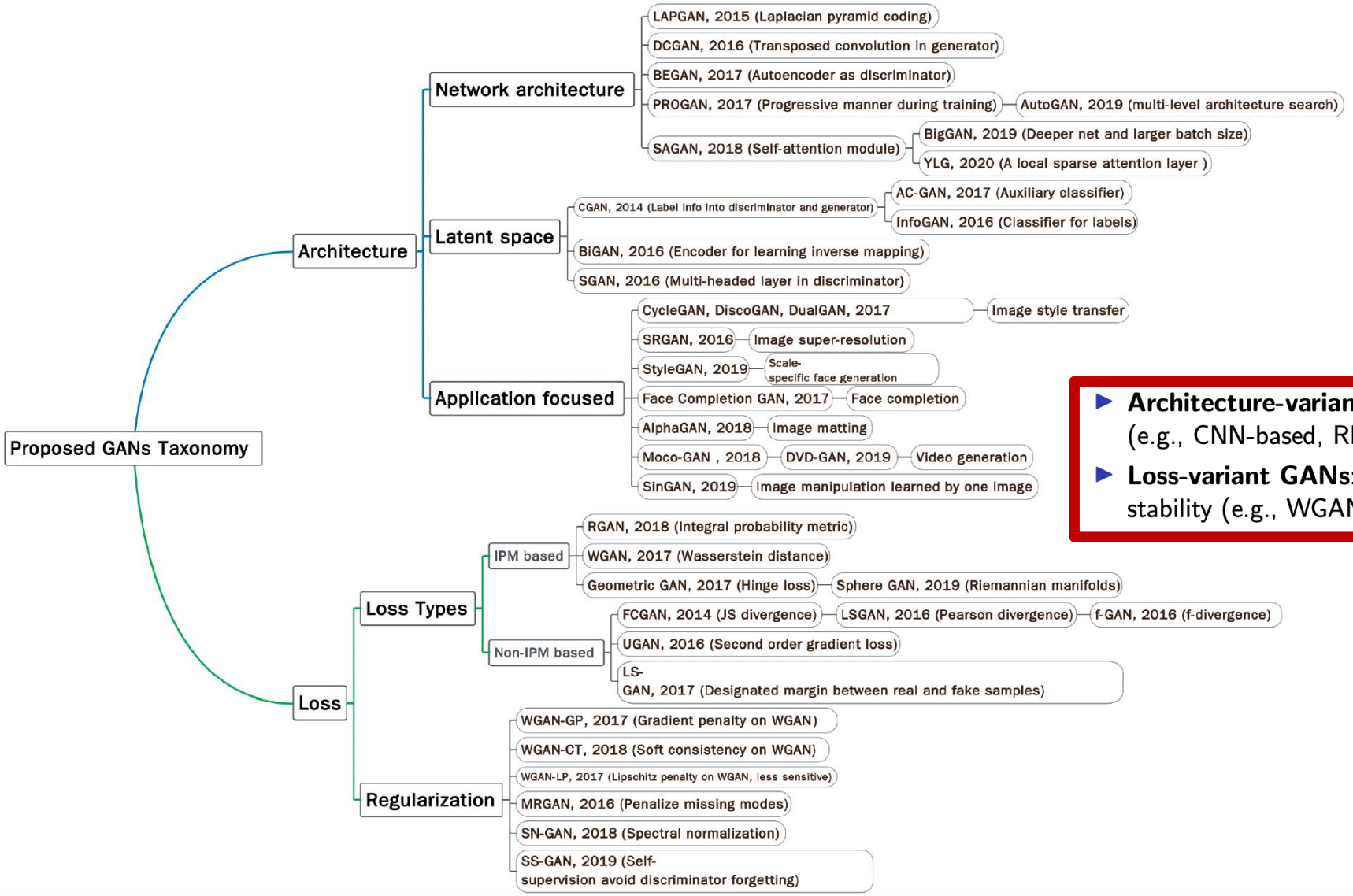
5.Mutual Information Regularization

Forcing the generator to learn **meaningful latent representations** that generate diverse outputs.

.

More details of example GAN suffering mode collapse: https://neptune.ai/blog/gan-failure-modes

The taxonomy of the recent GANs

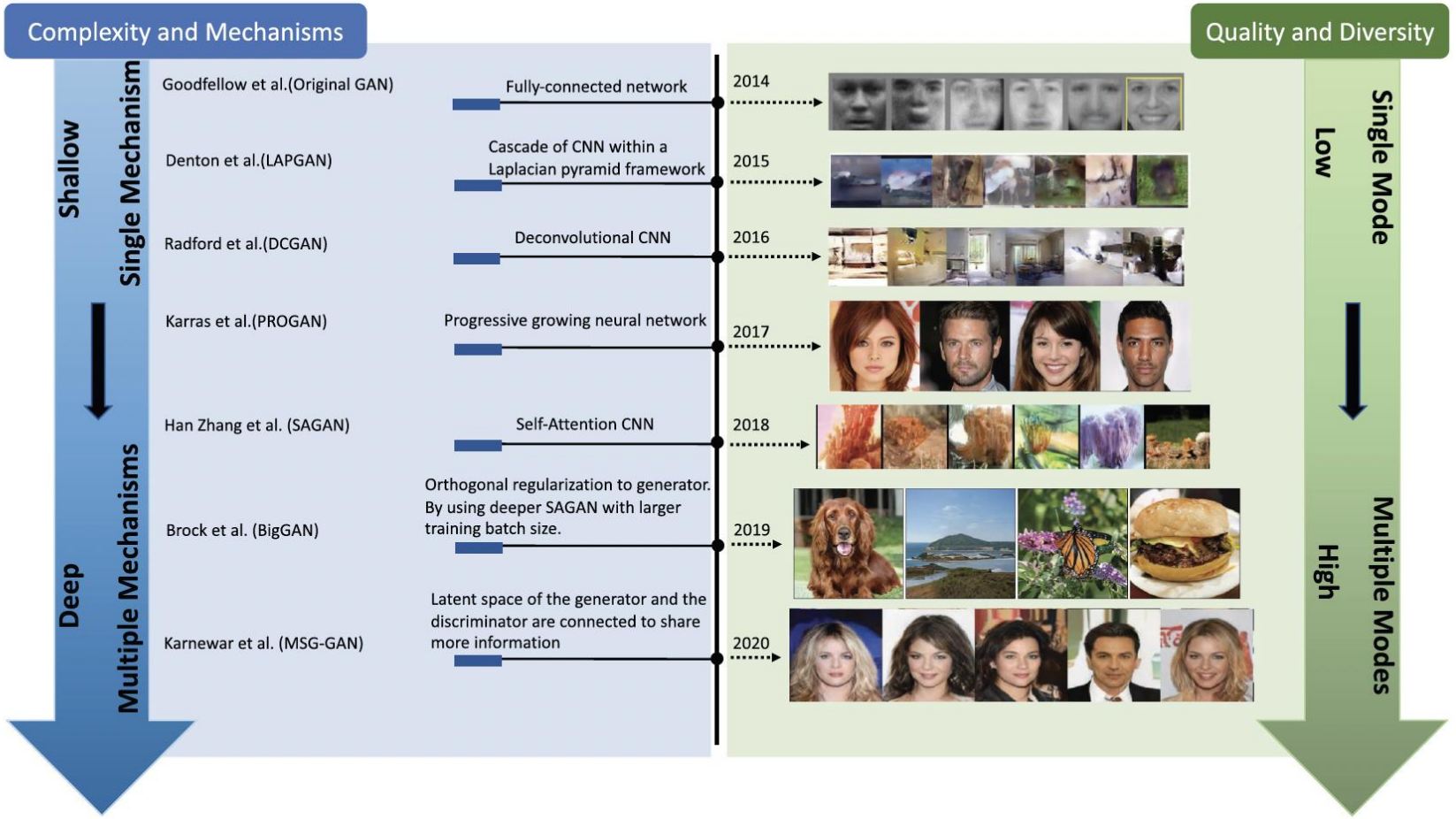


- **Architecture-variant GANs**: Modify network structures (e.g., CNN-based, RNN-based models).
- **Loss-variant GANs**: Modify loss functions to improve stability (e.g., WGAN, LSGAN, f-GAN).

Different GANs

Model	Stability	Mode Collapse	Convergence	Sample Quality	Special Features
GAN	Low	High	Unstable	Medium	Baseline
LSGAN	Medium	Medium	More stable	Medium	Least squares loss
WGAN	High	Low	More stable	High	Wasserstein distance
WGAN-GP	Very High	Very Low	Very stable	Very High	Gradient Penalty
cGAN	Medium	Medium	Stable	High	Class conditioning
StyleGAN	High	Low	Stable	Very High	Style control

Timeline of GAN architectures



Complexity in blue stream refers to size of the architecture and computational cost such as batch size. Mechanisms refer to the number of types of operations (e.g., convolution, deconvolution, self-attention) used in the architecture (e.g., FCGAN uses fully connected layers for both discriminator and generator. In this case, the value for mechanisms is 1).

Loss-variant GANs

- Jointly training two networks is challenging, can be unstable. Choosing objectives with better loss landscapes helps training, and is an active area of research.
- $X \sim P_X$ vs. $G(Z) \sim P_G$ with $Z \sim N(0, I)$.
- Training GAN is equivalent to minimizing Jensen-Shannon divergence between generator and data distributions.
- $D(P_X, P_G) = \sup_{f \in \mathcal{F}} \left\{ \mathbb{E}_{X \sim P_X} \phi_1(f(X)) - \mathbb{E}_{Y \sim P_G} \phi_2(f(Y)) \right\}$

GAN	Discriminator Loss	Generator Loss
MM GAN	$\mathcal{L}_{\mathrm{D}}^{\mathrm{GAN}} = -\mathbb{E}_{x \sim p_d}[\log(D(x))] - \mathbb{E}_{\hat{x} \sim p_g}[\log(1 - D(\hat{x}))]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{GAN}} = \mathbb{E}_{\hat{x} \sim p_g}[\log(1 - D(\hat{x}))]$
NS GAN	$\mathcal{L}_{\mathrm{D}}^{\mathrm{NSGAN}} = -\mathbb{E}_{x \sim p_d}[\log(D(x))] - \mathbb{E}_{\hat{x} \sim p_g}[\log(1 - D(\hat{x}))]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{NSGAN}} = -\mathbb{E}_{\hat{x} \sim p_g}[\log(D(\hat{x}))]$
WGAN	$\mathcal{L}_{\mathrm{D}}^{\mathrm{WGAN}} = -\mathbb{E}_{x \sim p_d}[D(x)] + \mathbb{E}_{\hat{x} \sim p_g}[D(\hat{x})]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{WGAN}} = -\mathbb{E}_{\hat{x} \sim p_g}[D(\hat{x})]$
WGAN GP	$\mathcal{L}_{\mathrm{D}}^{\mathrm{WGANGP}} = \mathcal{L}_{\mathrm{D}}^{\mathrm{WGAN}} + \lambda \mathbb{E}_{\hat{x} \sim p_g}[(\|\nabla D(\alpha x + (1 - \alpha \hat{x})\|_2 - 1)^2]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{WGANGP}} = -\mathbb{E}_{\hat{x} \sim p_g}[D(\hat{x})]$
LS GAN	$\mathcal{L}_{\mathrm{D}}^{\mathrm{LSGAN}} = -\mathbb{E}_{x \sim p_d}[(D(x) - 1)^2] + \mathbb{E}_{\hat{x} \sim p_g}[D(\hat{x})^2]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{LSGAN}} = -\mathbb{E}_{\hat{x} \sim p_g}[(D(\hat{x} - 1))^2]$
DRAGAN	$\mathcal{L}_{\mathrm{D}}^{\mathrm{DRAGAN}} = \mathcal{L}_{\mathrm{D}}^{\mathrm{GAN}} + \lambda \mathbb{E}_{\hat{x} \sim p_d + \mathcal{N}(0,c)}[(\|\nabla D(\hat{x})\|_2 - 1)^2]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{DRAGAN}} = \mathbb{E}_{\hat{x} \sim p_g}[\log(1 - D(\hat{x}))]$
BEGAN	$\mathcal{L}_{\mathrm{D}}^{\mathrm{BEGAN}} = \mathbb{E}_{x \sim p_d}[\|x - \mathrm{AE}(x)\|_1] - k_t \mathbb{E}_{\hat{x} \sim p_g}[\|\hat{x} - \mathrm{AE}(\hat{x})\|_1]$	$\mathcal{L}_{\mathrm{G}}^{\mathrm{BEGAN}} = \mathbb{E}_{\hat{x} \sim p_g}[\|\hat{x} - \mathrm{AE}(\hat{x})\|_1]$

GAN-related Loss Functions

GANs aim to approximate the real data distribution $p_{data}(x)$ using a generator network $G(\eta)$, where $\eta \sim p_z(\eta)$ is drawn from a simple prior distribution.

- The training process is driven by a discriminator D(x), which distinguishes real from generated samples.
- The loss function should measure **the divergence** between $p_{data}(x)$ and $p_g(x)$ where $p_g(x)$ is the distribution of generated samples

Minimax Loss: In the original GANs, the generator tries to minimize the following function while the discriminator tries to maximize it:

$$\min_G \max_D V(D,G) = E_x[\log(D(x))] + E_z[\log(1 - D(G(z)))]$$

The formula derives from the cross-entropy between the real and generated distributions.

$$\max_D V(D,G) = \max_D \{\mathbb{E}_{x\sim P_{\text{data}}}[\log D(x)] + \mathbb{E}_{x\sim P_g}[\log(1 - D(x))]\}$$

For a given x, the optimal discriminator is given by

$$D^*(x) = \frac{P_{\text{data}}(x)}{P_{\text{data}}(x) + P_g(x)}$$

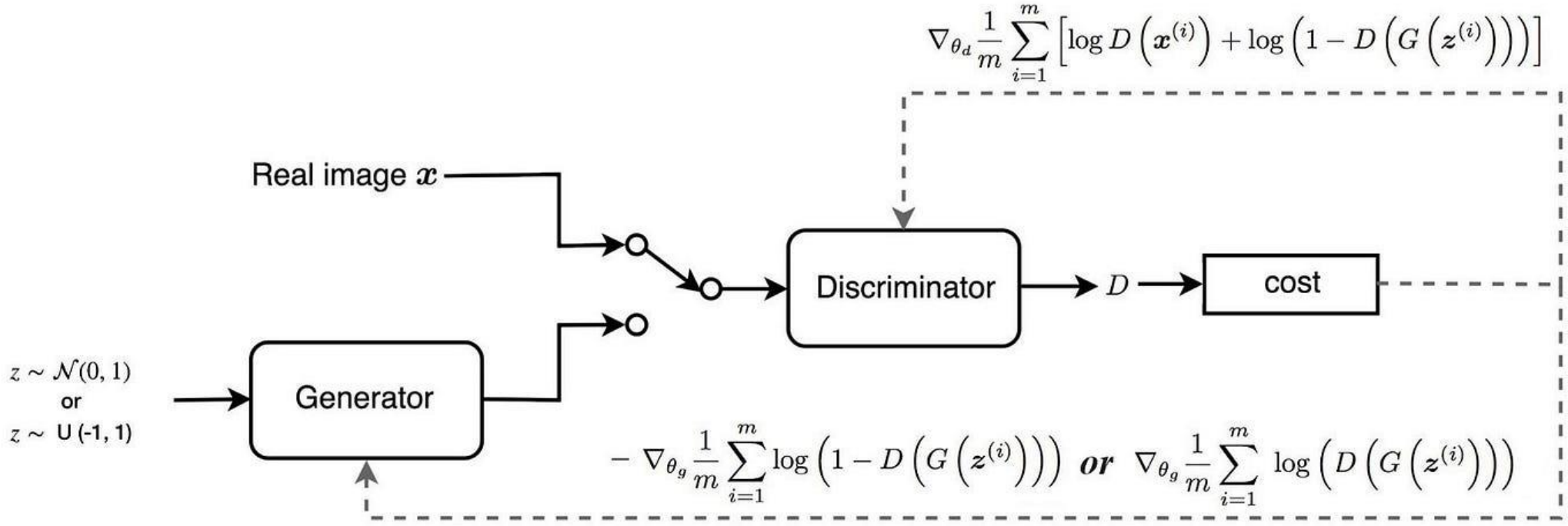
Thus, minimizing the GAN objective is equivalent to minimizing the Jensen-Shannon divergence as follows:

$$\min_G V(G,D^*) = \min_G JS(P_{\text{data}} \| P_g) + \log 4 = \min_G KL(P_{\text{data}} \| M) + KL(P_g \| M)$$

$$M(x) = \tfrac{1}{2}(P_{\text{data}}(x) + P_g(x))$$

Minimax Loss

The Standard GAN loss function can further be categorized into two parts: Discriminator loss and Generator loss. The diagram below summarizes how we train the discriminator and the generator using the corresponding gradient.



Non-Saturating GAN Loss

f-divergence

The f-divergence between probability densities p and q

$$\mathbb{D}_f(p\|q) = \mathbb{E}_{X\sim q}\left[f\left(\frac{p(X)}{q(X)}\right)\right] = \int_{\mathcal{X}} f\left(\frac{p(x)}{q(x)}\right) q(x)dx$$

where $f : (0, +\infty) \mapsto \mathcal{R}^1$ is a convex function with $f(1) = 0$.

- If $p = q$, then $\mathbb{D}_f(p\|q) = 0$.
- Jensen's inequality:

$$\mathbb{D}_f(p\|q) \geq f\left[\mathbb{E}_{X\sim q}\left(\frac{p(X)}{q(X)}\right)\right] = f(1) = 0.$$

- If $f(x)$ is strictly convex in a neighborhood of 1, then

$$\mathbb{D}_f(p\|q) = 0 \iff p = q.$$

- The KL divergence is a special case by taking $f(x) = x\log x - x + 1$.

- KL: $f(x) = x\log(x) - x + 1$.
- χ^2: $f(x) = \frac{1}{2}(x-1)^2$.
- Hellinger: $f(x) = 2(\sqrt{x}-1)$.
- L_1: $f(x) = |x-1|$.
- Jensen-Shannon (JS): $f(x) = x\log x - (x+1)\log\frac{x+1}{2}$.

Least Squares GAN (LSGAN)

- **Objective:** Improve stability and quality of GAN training by replacing standard binary cross-entropy loss with least squares loss.
- **Proposed by:** Mao et al. (2017) in the paper *"Least Squares Generative Adversarial Networks"*
- **Key Motivation:** Standard GANs suffer from **vanishing gradients** when the discriminator becomes too confident. Least squares loss provides **stronger gradients** and better sample quality.

In LSGAN, the **discriminator** is trained with the following **least squares loss**:

$$L_D = \frac{1}{2}\mathbb{E}_{x\sim p_{data}}[(D(x)-1)^2] + \frac{1}{2}\mathbb{E}_{z\sim p_z}[D(G(z))^2]$$

In LSGAN, the **generator** is trained to minimize:

$$L_G = \frac{1}{2}\mathbb{E}_{z\sim p_z}[(D(G(z))-1)^2]$$

Minimizing LSGAN loss is equivalent to minimizing the Pearson χ^2 divergence.

- ❖ LSGAN minimizes the Pearson χ^2 divergence, making it more stable than standard GANs.
- ❖ Compared to Jensen-Shannon divergence (used in standard GANs), χ^2 divergence is more sensitive to small differences in distributions.
- ❖ LSGAN penalizes fake samples more aggressively, which helps avoid mode collapse.

Wasserstein GAN (WGAN)

Let Ω be a subset of $\mathbb{R}^d$. Let $\mathcal{B}_p(\Omega)$ be the set of Borel probability measures on Ω with finite pth moment. The p-Wasserstein metric is defined as

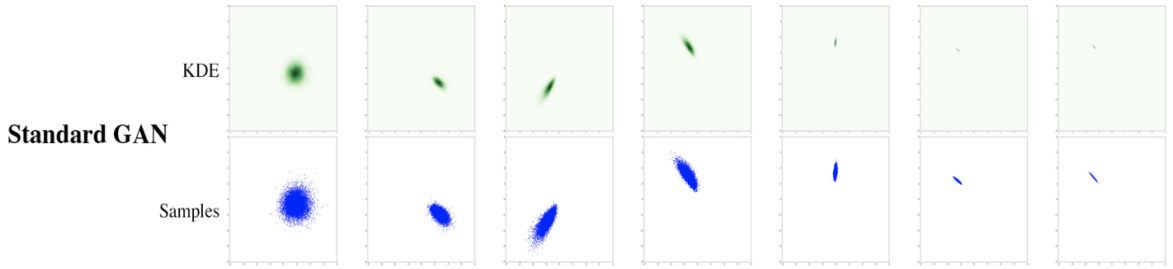
$$W_p(\mu,\nu) = \left(\inf_{\gamma\in\Gamma(\mu,\nu)}\int |x-y|^p d\gamma(x,y)\right)^{1/p}, \quad \mu \text{ and } \nu \in \mathcal{B}_p(\Omega).$$

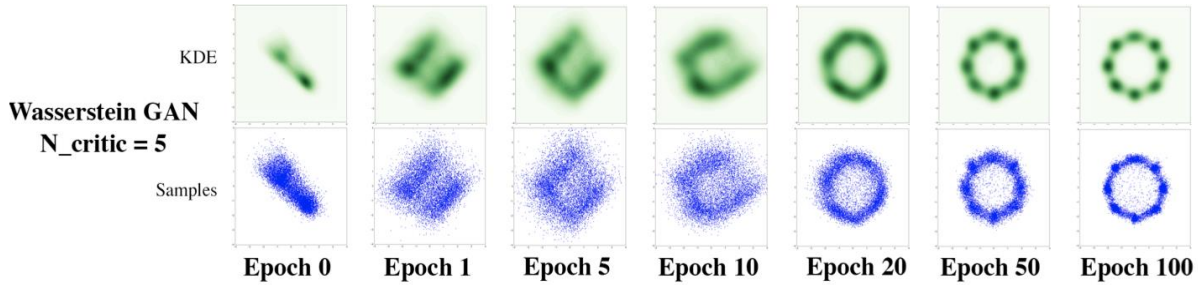
For the special case of p = 1, the p-Wasserstein metric is also known as the Monge-Rubinstein metric, or the earth mover distance.

The 1-Wasserstein metric can be expressed as (Villani, 2008),

$$W_1(\mu,\nu) = \sup_{f\in\mathcal{F}_1}\left\{\int f(x)d\mu(x) - \int f(x)d\nu(x)\right\},$$

This expression of 1-Wasserstein metric is computationally convenient, which is used in the construction of Wasserstein generative adversarial networks (WGAN) (Arjovsky et al., 2017).





Training WGAN

Critic (Discriminator) Loss:

$$L_D = \mathbb{E}_{x \sim P_{\text{data}}}[D(x)] - \mathbb{E}_{z \sim P_z}[D(G(z))]$$

$$\nabla_{\theta_D} L_D = \nabla_{\theta_D} \left(\mathbb{E}_{x \sim P_{\text{data}}}[D(x)] - \mathbb{E}_{z \sim P_z}[D(G(z))] \right)$$

$$\theta_D \leftarrow \theta_D + \eta_D \nabla_{\theta_D} L_D$$

Methods to Enforce Lipschitz Constraint:

Weight Clipping (Original WGAN):

$$-c \leq w \leq c$$

Gradient Penalty (WGAN-GP):

$$L_{\text{GP}} = \lambda \mathbb{E}_{\hat{x} \sim P_{\hat{x}}}[(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2]$$

Generator Loss:

$$L_G = -\mathbb{E}_{z \sim P_z}[D(G(z))]$$

$$\nabla_{\theta_G} L_G = -\nabla_{\theta_G} \mathbb{E}_{z \sim P_z}[D(G(z))]$$

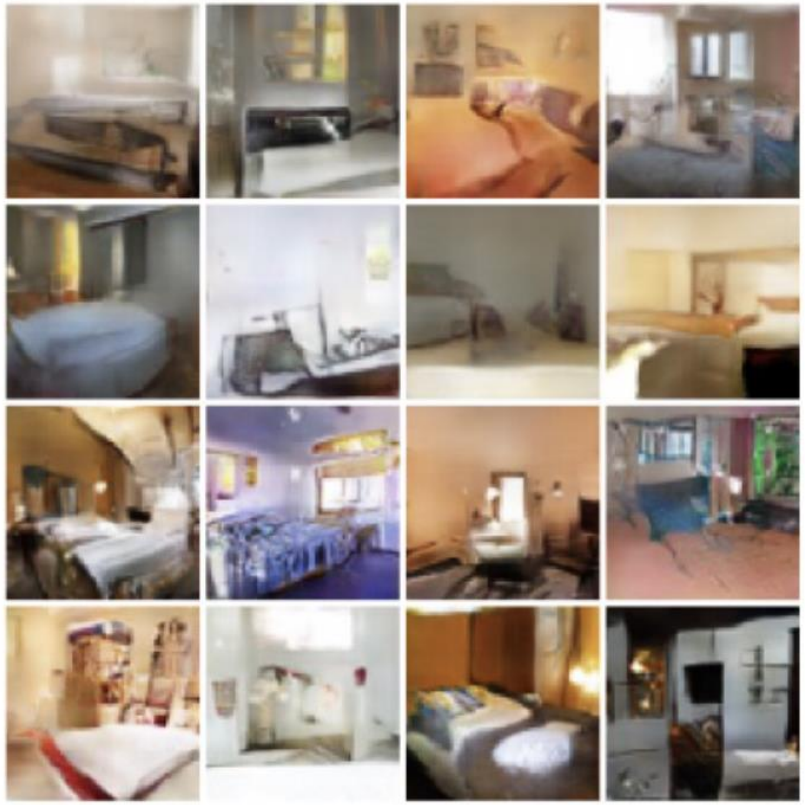
$$\theta_G \leftarrow \theta_G + \eta_G \nabla_{\theta_G} L_G$$

Training Process:

1. Update the critic D multiple times per generator update.
2. Compute Wasserstein distance using the critic's output.
3. Update generator G to minimize the critic's output.

WGAN vs WGAN-NP

Wasserstein GAN (WGAN)



Arjovsky, Chintala, and Bouttou, "Wasserstein GAN", 2017

WGAN with Gradient Penalty (WGAN-GP)

Gulrajani et al, "Improved Training of Wasserstein GANs", NeurIPS 2017

Progressive GAN (PGAN)

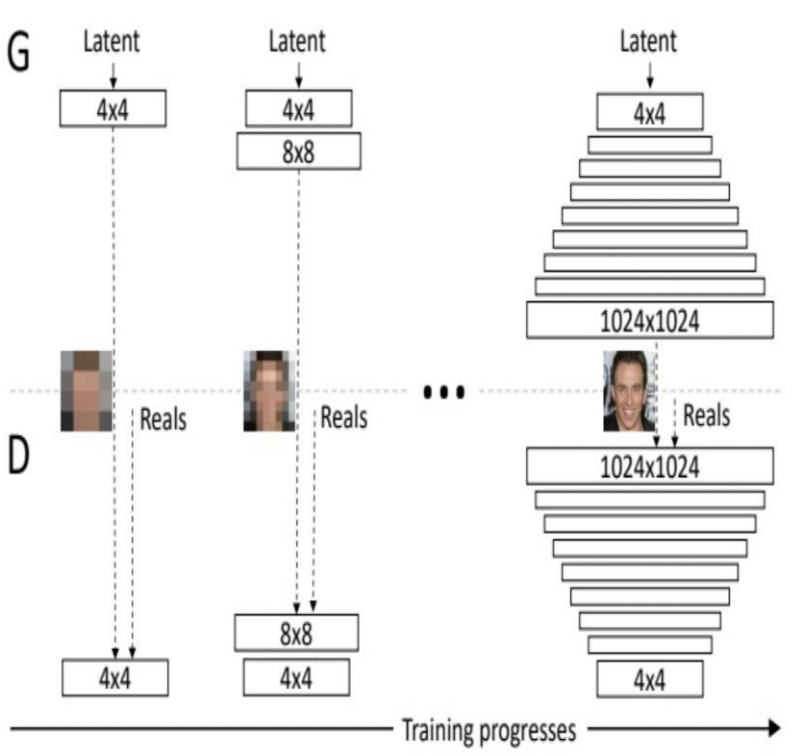
- Progressive GAN (PGAN) is a technique for progressive growth of layers in both the generator and discriminator.
- Introduced by Karras et al. (2017) for high-resolution image synthesis.
- Allows training GANs stably at resolutions up to 1024×1024.
- Works by starting small and growing larger over training iterations.

Progressive Growing Mechanism

- Training begins with low-resolution images (e.g., 4×4).
- New layers are added progressively to both generator and discriminator.
- Old layers remain trainable, allowing smooth transition.
- Uses smooth transition (fade-in layers) when adding new resolutions.

Architecture of Progressive GAN

- Generator and Discriminator start with small networks (4×4).
- New convolutional layers are added progressively to increase resolution.
- Uses skip connections to stabilize training.
- Mini-batch standard deviation is used to improve diversity.



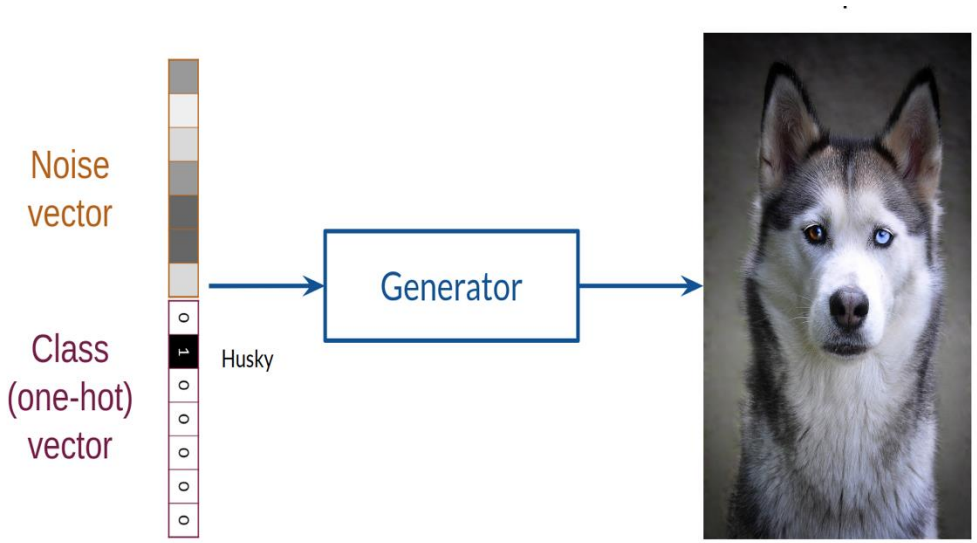
Code: Vanilla GAN

```
import torch
import torch.nn as nn
import torchvision.models as models

# ----- Define Hyperparameters ---
lr = 0.0002       # Learning rate
z_dim = 64        # Dimensionality of the noise vector
image_dim = 28*28 # 784 for MNIST (28 x 28)
hidden_dim = 128  # Hidden layer dimensionality for both Generator and Discriminator
batch_size = 128
epochs = 50       # number of epoches

# ----- Define the Generator -----
# A fully connected (MLP) generator that takes a random noise vector z and outputs a 28x28 image (784-dimensional
vector). We apply a Tanh activation to the final layer to constrain the pixel values between -1 and 1.
class Generator(nn.Module):
    def __init__(self, z_dim, hidden_dim, out_dim):
        super(Generator, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(noise_dim, hidden_dim),
            nn.ReLU(True),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(True),
            nn.Linear(hidden_dim, out_dim),
            nn.Tanh()
        )

    def forward(self, x):
        return self.net(x)
```

Code: Vanilla GAN

```
import torch
import torch.nn as nn
import torchvision.models as models

# ----- Define the Discriminator -----
# A fully connected (MLP) discriminator that takes a 784-dimensional vector (flattened 28x28 image) and outputs a
single probability (real vs. fake). We apply a Sigmoid at the end to interpret the output as a probability.
class Discriminator(nn.Module):
    def __init__(self, in_dim, hidden_dim):
        super(Discriminator, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim, hidden_dim),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Linear(hidden_dim, hidden_dim),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Linear(hidden_dim, 1),
            nn.Sigmoid()
        )
    def forward(self, x):
        return self.net(x)

# -----  Instantiate Model and Optimizers  -----
# Initialize generator and discriminator
gen = Generator(z_dim, hidden_dim, image_dim).to(device)
disc = Discriminator(image_dim, hidden_dim).to(device)
criterion = nn.BCELoss() # Binary Cross Entropy loss
# Optimizers (use Adam for both)
optimizer_gen = optim.Adam(gen.parameters(), lr=lr, betas=(0.5, 0.999))
optimizer_disc = optim.Adam(disc.parameters(), lr=lr, betas=(0.5, 0.999))
```

Controllable Generation and Conditional GAN



Most of the practical applications require the ability to sample a conditional distribution, like:

- Next frame prediction.
- "in-painting",
- segmentation,
- style transfer.

This would in particular address some of the shortcomings of unconditional GANs.

Illustration of example to generate a breed of dog (source)

- CGAN is a supervised extension of GANs where the generator and discriminator receive additional information.
- Allows generating samples based on specific conditions.

CGAN

The CGAN proposed by Mirza and Osindero (2014) consists of parameterizing both G and D by a conditioning quantity Y.

$$\min_G \max_D \mathbb{E}_{x,y\sim p_{data}(x,y)}[\log D(x,y)] + \mathbb{E}_{z\sim p_z(z),y\sim p_y(y)}[\log(1 - D(G(z,y),y))]$$

This adds semantic meaning to latent space manifold and provides more control in the types of output generated by the generator.

Training Algorithm:

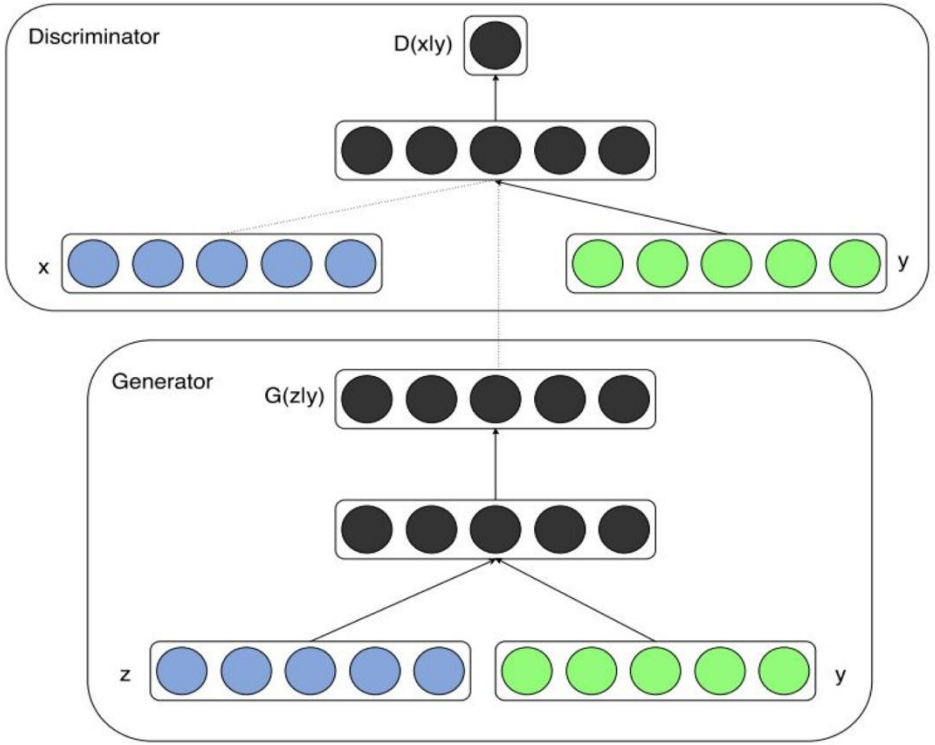
1. Initialize generator G, discriminator D, and dataset.
2. For each iteration:
   - Sample real data $(x, y) \sim p_{data}(x, y)$.
   - Generate fake samples $G(z, y)$ with $z \sim p_z(z)$.
   - Update D by minimizing:

     $$L_D = -\mathbb{E}_{(x,y)\sim p_{data}}[\log D(x,y)] - \mathbb{E}_{z\sim p_z, y\sim p_y}[\log(1 - D(G(z,y),y))]$$

   - Update G by minimizing:

     $$L_G = -\mathbb{E}_{z\sim p_z, y\sim p_y}[\log D(G(z,y),y)]$$

3. Repeat until convergence.

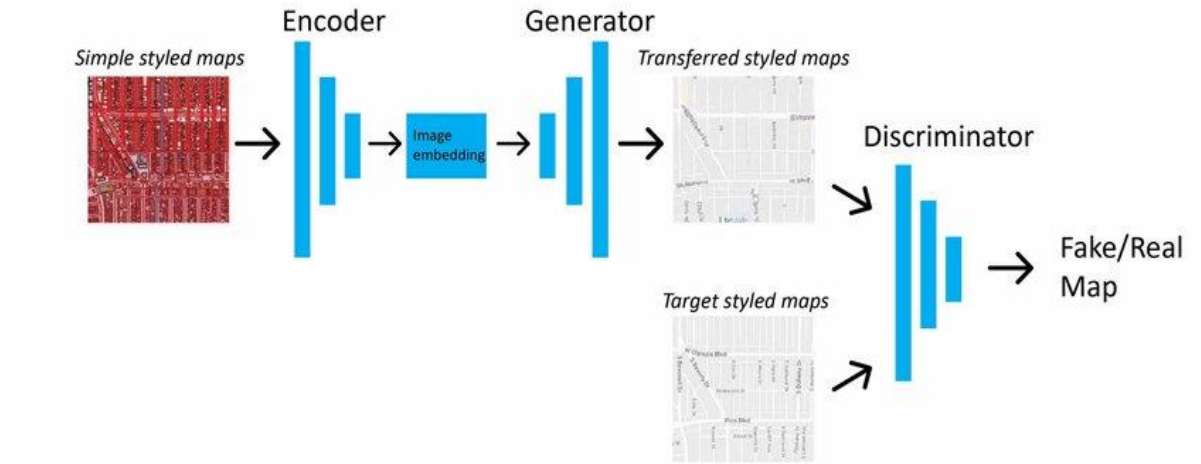


Pix2Pix

- Pix2Pix is a supervised image-to-image translation model based on cGANs.
- Introduced in Isola et al. (2017) for tasks like sketch-to-photo, satellite-to-map, and more.

Pix2Pix Objective: $G^* = \arg\min_G \max_D \mathcal{L}_{cGAN}(G, D) + \lambda \mathcal{L}_{L1}(G)$ $\mathcal{L}_{L1}(G) = \mathbb{E}_{x,y,z} \|y - G(x, z)\|_1$

$$\mathcal{L}_{cGAN}(G, D) = \mathbb{E}_{x,y}[\log D(x, y)] + \mathbb{E}_{x,z}[\log(1 - D(x, G(x, z)))]$$



Training Algorithm

1. Initialize generator G and discriminator D.
2. For each training step:
   - Sample real image pairs $(x, y) \sim p_{data}(x, y)$.
   - Generate fake image $G(x, z)$.
   - Update D by minimizing:

$$L_D = -\mathbb{E}[\log D(x, y)] - \mathbb{E}[\log(1 - D(x, G(x, z)))]$$

   - Update G by minimizing:

$$L_G = -\mathbb{E}[\log D(x, G(x, z))] + \lambda \|y - G(x, z)\|_1$$

3. Repeat until convergence.

Pix2Pix Generator: U-Net

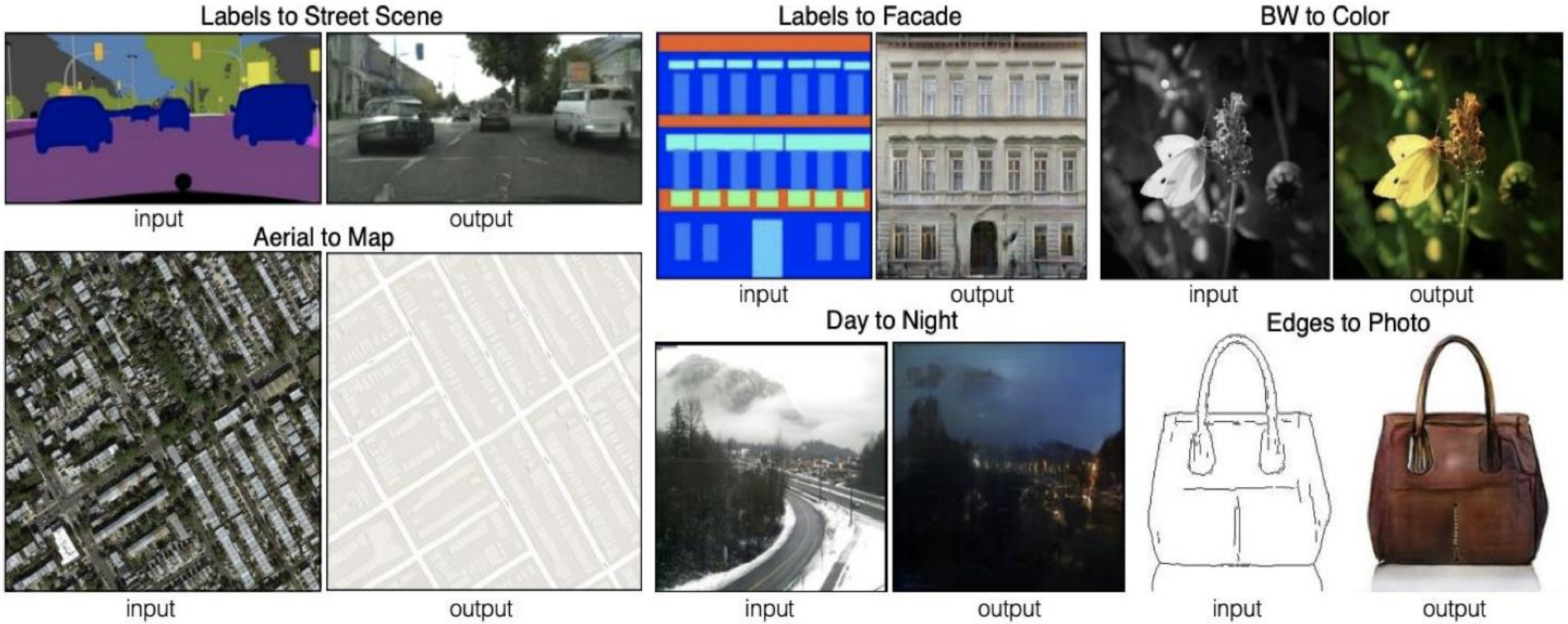
- The generator is based on a U-Net architecture, using an encoder-decoder structure with skip connections.
- The encoder extracts deep features while the decoder reconstructs the image.
- Skip connections help preserve fine-grained details.

Pix2Pix Discriminator: PatchGAN

- Uses a convolutional PatchGAN discriminator instead of a full-image classifier.
- PatchGAN classifies small image patches instead of the entire image.
- Helps focus on local texture realism and prevents blurriness.

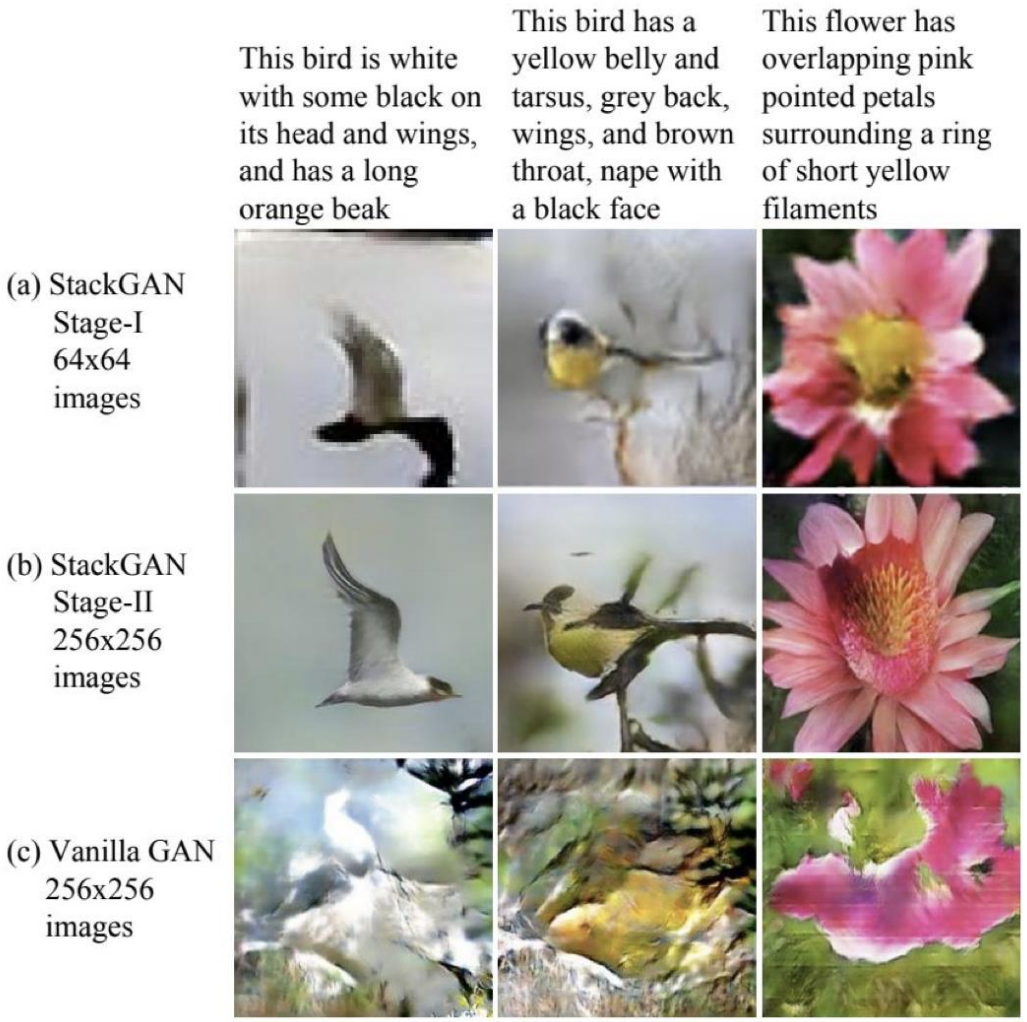
Application of CGANs

Image-to-image translation



https://arxiv.org/pdf/1611.07004.pdf

Application of CGANs



Text to image synthesis

Comparison of StackGAN (stacked conditional GAN) and a one-stage GAN for generating 256×256 images. (a) Given text descriptions, Stage-I of StackGAN sketches rough shapes and basic colors of objects, yielding low-resolution images. (b) Stage-II of StackGAN takes Stage-I results and text descriptions as inputs, and generates high-resolution images with photo-realistic details. (c) Results by a vanilla 256×256 GAN which simply adds more upsampling layers to state-of-the-art GAN-INT-CLS [26]. It is unable to generate any plausible images of 256×256 resolution.

http://arxiv.org/pdf/1612.03242

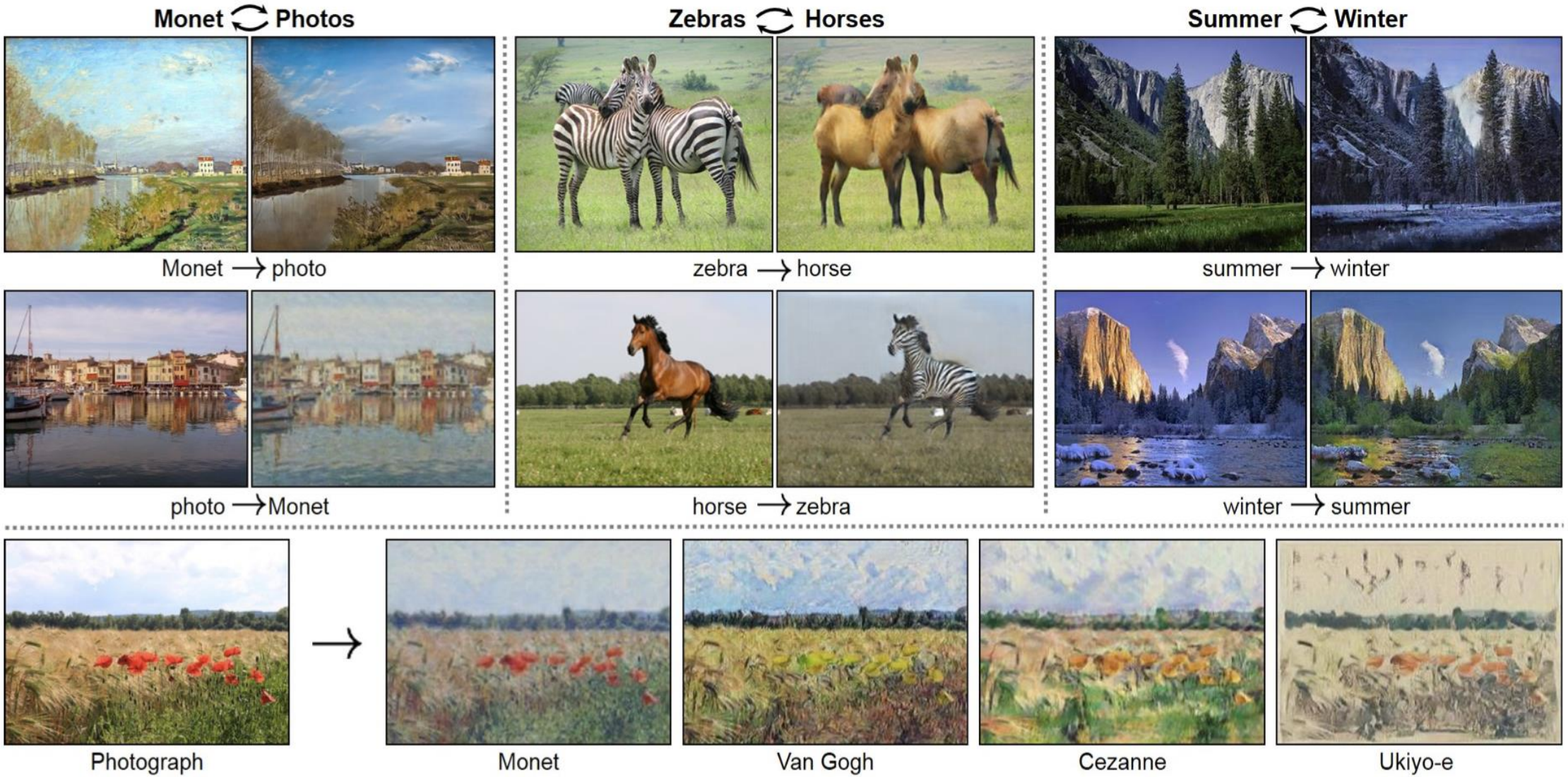
Code: Conditional GAN

```
import torch
import torch.nn as nn

# ----- Define Hyperparameters ---
lr = 0.0002       # Learning rate
z_dim = 64        # Dimensionality of the noise vector
image_dim = 28*28 # 784 for MNIST (28 x 28)
hidden_dim = 128  # Hidden layer dimensionality for both Generator and Discriminator
batch_size = 128
epochs = 50       # number of epoches
# ----- Define the Generator -----
# We concatenate [z, label_onehot] into a single vector of size z_dim + label_dim before passing through an MLP. The
output is a flattened 28x28 image (size 784), which we squish to [-1,1] using Tanh.
class Generator(nn.Module):
    def __init__(self, z_dim, label_dim, hidden_dim, out_dim):
        super(Generator, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(noise_dim, hidden_dim),
            nn.ReLU(True),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(True),
            nn.Linear(hidden_dim, out_dim),
            nn.Tanh()
        )
    def forward(self, x):
        # labels: (batch_size, label_dim)
        # z: (batch_size, z_dim)
        x = torch.cat([z, labels], dim=1)  # Concatenate noise + label
        return self.net(x)
```

Code: Conditional GAN

```
import torch
import torch.nn as nn

# ----- Define the Discriminator -----
# We concatenate [image, label_onehot] into a single vector of size image_size + label_dim before passing through an MLP. The final output is a single probability (real or fake), obtained via Sigmoid.
class Discriminator(nn.Module):
    def __init__(self, in_dim, label_dim, hidden_dim):
        super(Discriminator, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(in_dim + label_dim, hidden_dim),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Linear(hidden_dim, hidden_dim),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Linear(hidden_dim, 1),
            nn.Sigmoid()
        )
    def forward(self, x, labels):
        # x: (batch_size, image_size)
        # labels: (batch_size, label_dim)
        x = torch.cat([x, labels], dim=1)  # Concatenate image + label
        return self.net(x)

# -----  Instantiate Model and Optimizers  -----
gen = Generator(z_dim, label_dim, hidden_dim, image_size).to(device)
disc = Discriminator(image_size, label_dim, hidden_dim).to(device)
criterion = nn.BCELoss()
optimizer_gen = optim.Adam(gen.parameters(), lr=lr, betas=(0.5, 0.999))
optimizer_disc = optim.Adam(disc.parameters(), lr=lr, betas=(0.5, 0.999))
```

Cycle GAN to Transfer Image Domains



Cycle GAN

- CycleGAN is an unsupervised image-to-image translation method.
- Unlike Pix2Pix, it does not require paired data.
- Uses two generators and two discriminators for learning mappings between two domains.
- Useful for photo enhancement, style transfer, and domain adaptation.

❖ Convert an image from one representation to another.

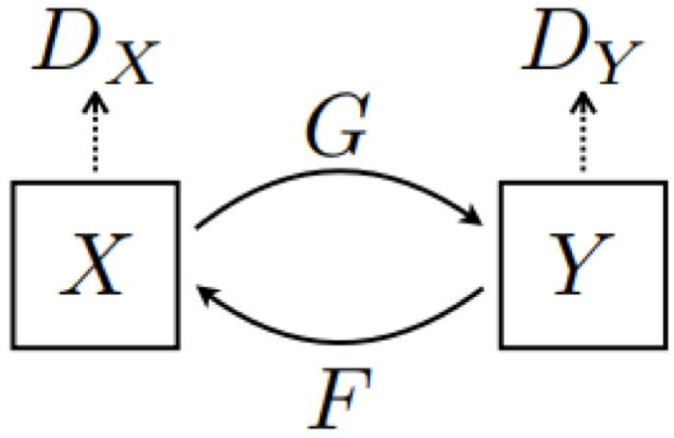
❖ Capture characteristics of one image domain and figure out how these characteristics could be translated into the other domain.

❖ Two mapping G : X → Y and F : Y → X. Two discriminators: D_X and D_Y.

Encourage G to generate images similar to images in domain Y and D_Y to distinguish G(x) from y.

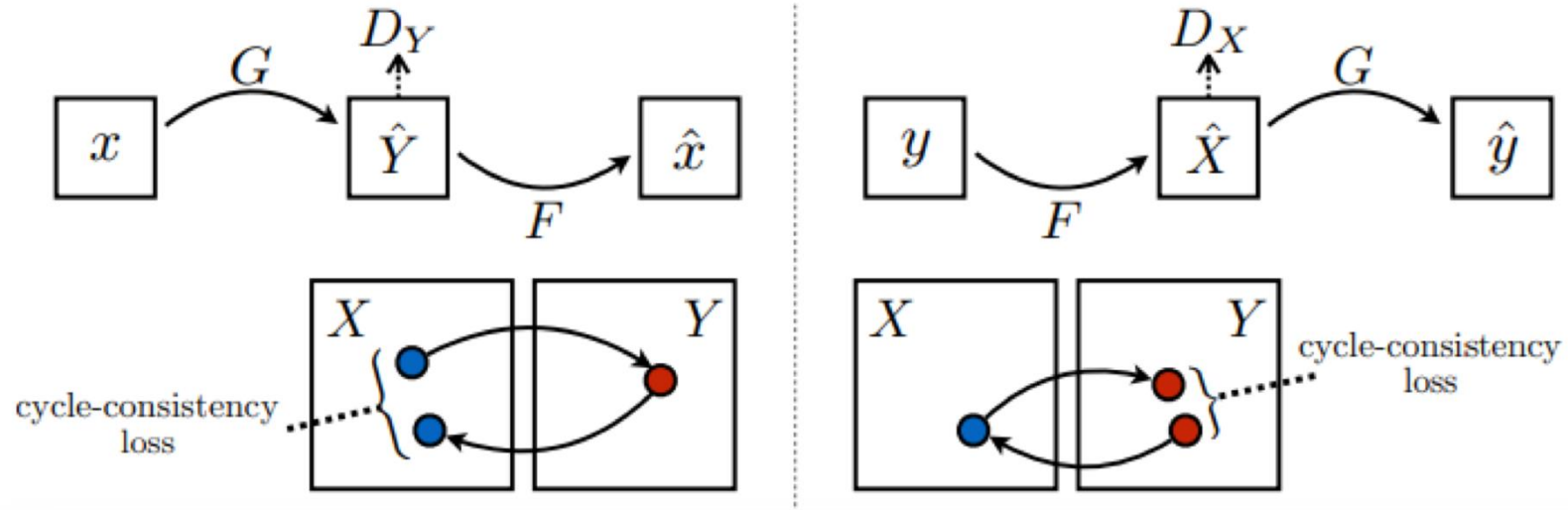
$$L_{GAN}(G, D_Y, X, Y) = \mathbb{E}_{y \sim p_{data}(y)}[log D_Y(y)] + \mathbb{E}_{x \sim p_{data}(x)}[log(1 - D_Y(G(x)))]$$

Similarly, for F, we also have $L_{GAN}(F, D_X, X, Y)$.

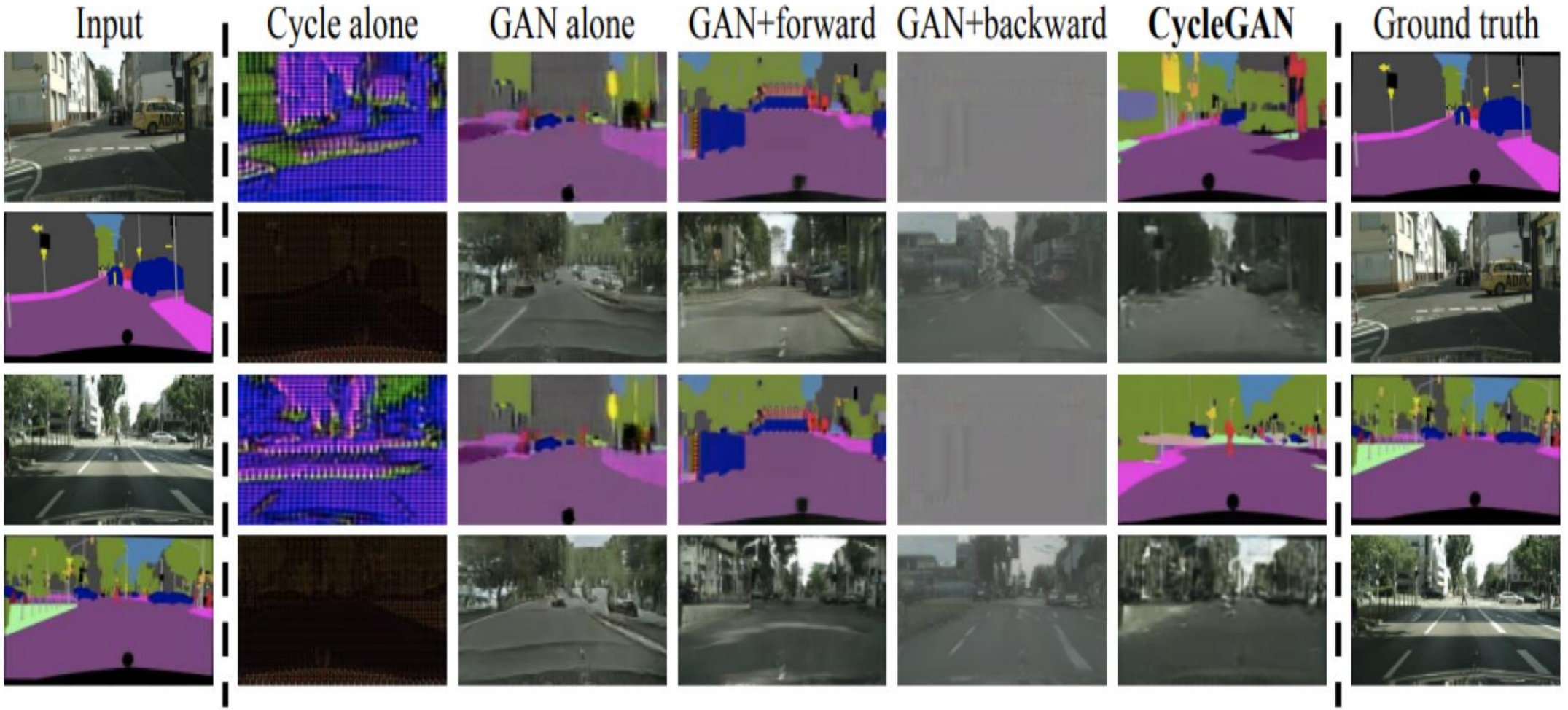


Consistency Loss

- Need extra regularization to make sure mapping function is cycle-consistent, (capable of mapping the input images to any subsets of images in the target domain), i.e., x → G(x) → F(G(x)) ≈ x.
- Cycle consistency loss: $L_{cyc}(G,F) = \mathbb{E}_{x\sim p_{data}(x)}\left[\left|\left|F\left(G(x)\right)-x\right|\right|_1\right] + \mathbb{E}_{y\sim p_{data}(y)}\left[\left|\left|G\left(F(y)\right)-y\right|\right|_1\right]$
- The full objective: $\mathrm{L}(G,F,D_X,D_Y) = L_{GAN}(G,D_Y,X,Y) + L_{GAN}(F,D_X,X,Y)$.

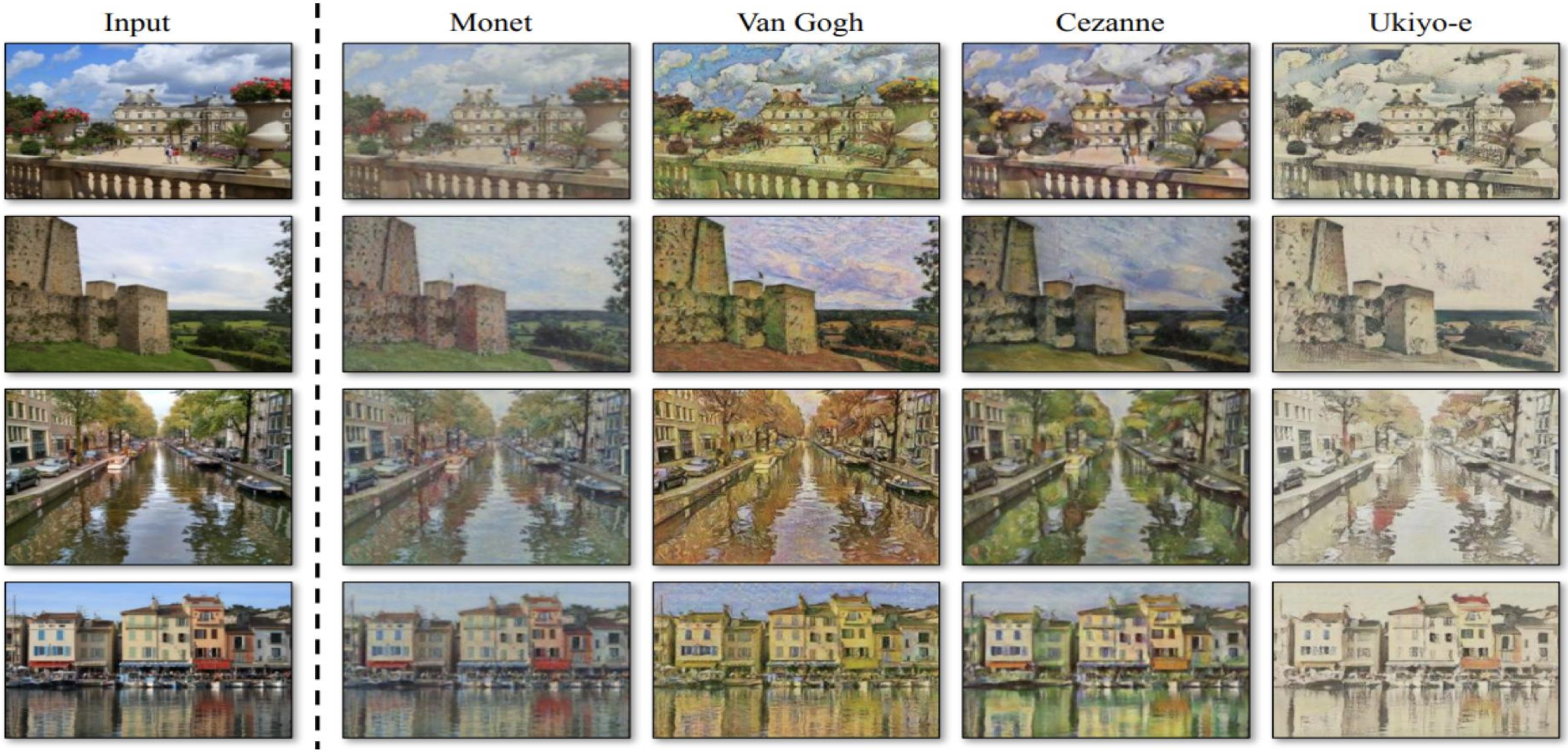


Cycle GAN Example Result



Jun-Yan Zhu*, Taesung Park*, Phillip Isola, and Alexei A. Efros. ”Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks”, in IEEE International Conference on Computer Vision (ICCV), 2017

Cycle GAN Example Result



Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, Alexei A. Efros Image-to-Image Translation with Conditional Adversarial Networks, CVPR, 2017

Tips to improve GAN performance

- Change the cost function for a better optimization goal.
- Add additional penalties to the cost function to enforce constraints.
- Avoid overconfidence and overfitting.
- Better ways of optimizing the model.
- Add labels (Conditional GAN).
- More details and other implementation tips: https://towardsdatascience.com/gan-ways-to-improve-gan-performance-acf37f9f59b

Code: Cycle GAN

```
# The key idea is to learn two translation mappings
between two domains X and Y without requiring paired
examples.
# ----- Define Residual Blocks -----
class ResidualBlock(nn.Module):
    """Residual Block with instance normalization."""
    def __init__(self, channels):
        super(ResidualBlock, self).__init__()
        self.block = nn.Sequential(
            nn.ReflectionPad2d(1),
            nn.Conv2d(channels, channels, kernel_size=3,
stride=1),
            nn.InstanceNorm2d(channels),
            nn.ReLU(True),

            nn.ReflectionPad2d(1),
            nn.Conv2d(channels, channels, kernel_size=3,
stride=1),
            nn.InstanceNorm2d(channels)
        )
    def forward(self, x):
        return x + self.block(x)

# - Define ResNet-based Generators (G: X → Y, F: Y → X)--
class GeneratorResNet(nn.Module):
    """
    Generator that transforms input images (domain X to
domain Y or vice versa).
    Uses several downsampling layers, residual blocks, and
upsampling layers.
    """
```

```
def __init__(self, in_channels, out_channels, n_res_blocks=6,
ngf=64):
        super(GeneratorResNet, self).__init__()
        # Initial convolution block
        model = [nn.ReflectionPad2d(3),
            nn.Conv2d(in_channels, ngf, kernel_size=7, stride=1),
            nn.InstanceNorm2d(ngf),
            nn.ReLU(True)]
        # Downsampling
        curr_dim = ngf
        for _ in range(2):
            model += [nn.Conv2d(curr_dim, curr_dim*2, kernel_size=
stride=2, padding=1),
                nn.InstanceNorm2d(curr_dim*2),
                nn.ReLU(True)]
            curr_dim *= 2
        # Residual blocks
        for _ in range(n_res_blocks):
            model += [ResidualBlock(curr_dim)]
        # Upsampling
        for _ in range(2):
            model += [nn.ConvTranspose2d(curr_dim, curr_dim//2,
kernel_size=3, stride=2, padding=1, output_padding=1),
nn.InstanceNorm2d(curr_dim//2),nn.ReLU(True)]
            curr_dim //= 2
        # Output layer
        model += [nn.ReflectionPad2d(3), nn.Conv2d(curr_dim,
out_channels, kernel_size=7, stride=1), nn.Tanh()]
        self.model = nn.Sequential(*model)

    def forward(self, x):
        return self.model(x)
```

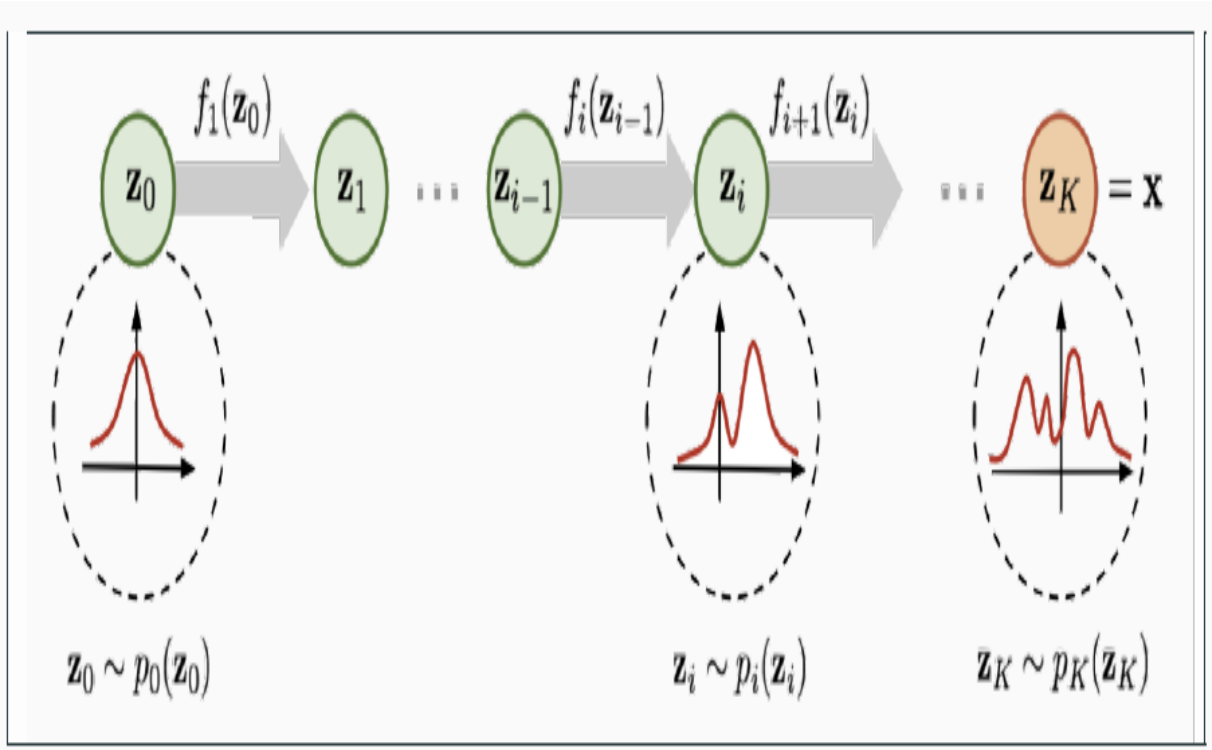
Code: Cycle GAN

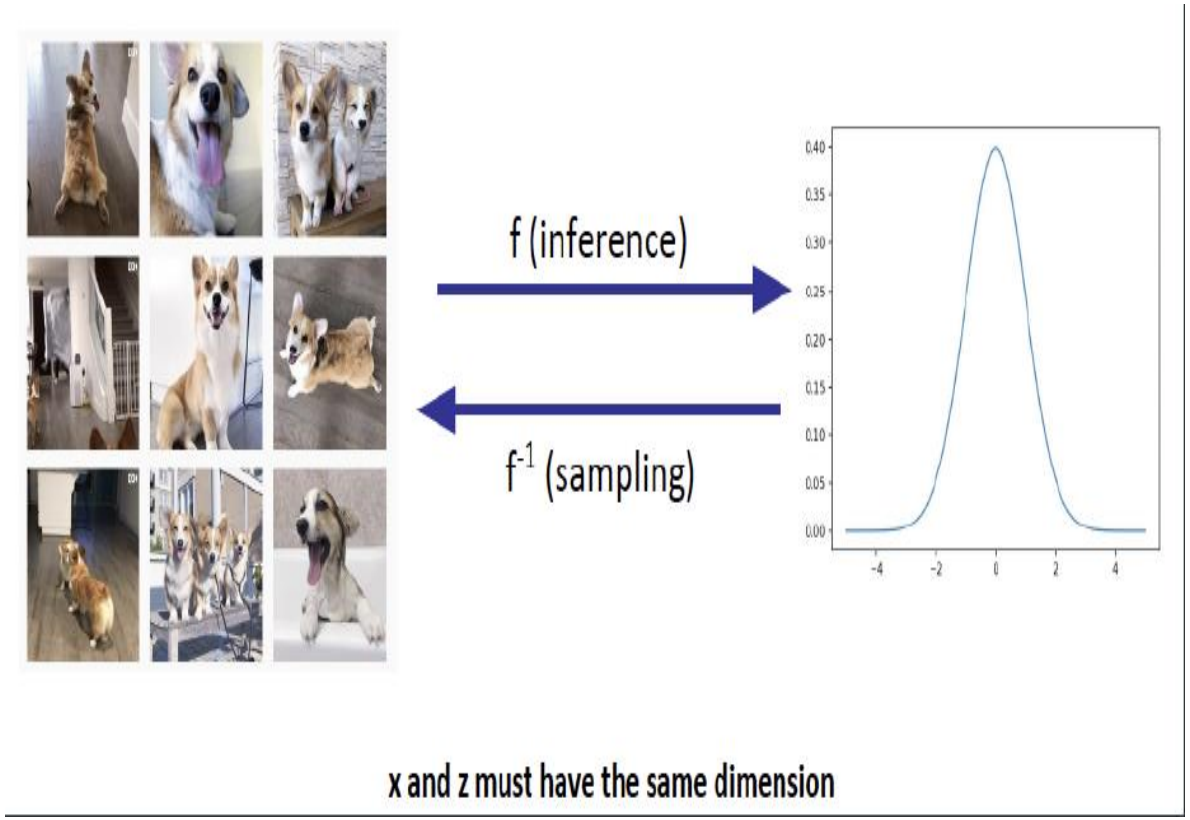
```
# ----- Define the Discriminator -----
class Discriminator(nn.Module):
    """
    PatchGAN discriminator: tries to classify each NxN patch in the image
    as real or fake. Output is a feature map of "realness" scores.
    """
    def __init__(self, in_channels=3, ndf=64):
        super(Discriminator, self).__init__()
        # A small patch-based ConvNet
        model = [
            nn.Conv2d(in_channels, ndf, kernel_size=4, stride=2, padding=1),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(ndf, ndf*2, kernel_size=4, stride=2, padding=1),
            nn.InstanceNorm2d(ndf*2),
            nn.LeakyReLU(0.2, inplace=True),

            nn.Conv2d(ndf*2, ndf*4, kernel_size=4, stride=2, padding=1),
            nn.InstanceNorm2d(ndf*4),
            nn.LeakyReLU(0.2, inplace=True),

            # Last convolution
            nn.Conv2d(ndf*4, 1, kernel_size=4, stride=1, padding=1)
            # No Sigmoid here, we use BCEWithLogitsLoss
        ]
        self.model = nn.Sequential(*model)

    def forward(self, x):
        return self.model(x)
```

Content

5 Normalized Flow

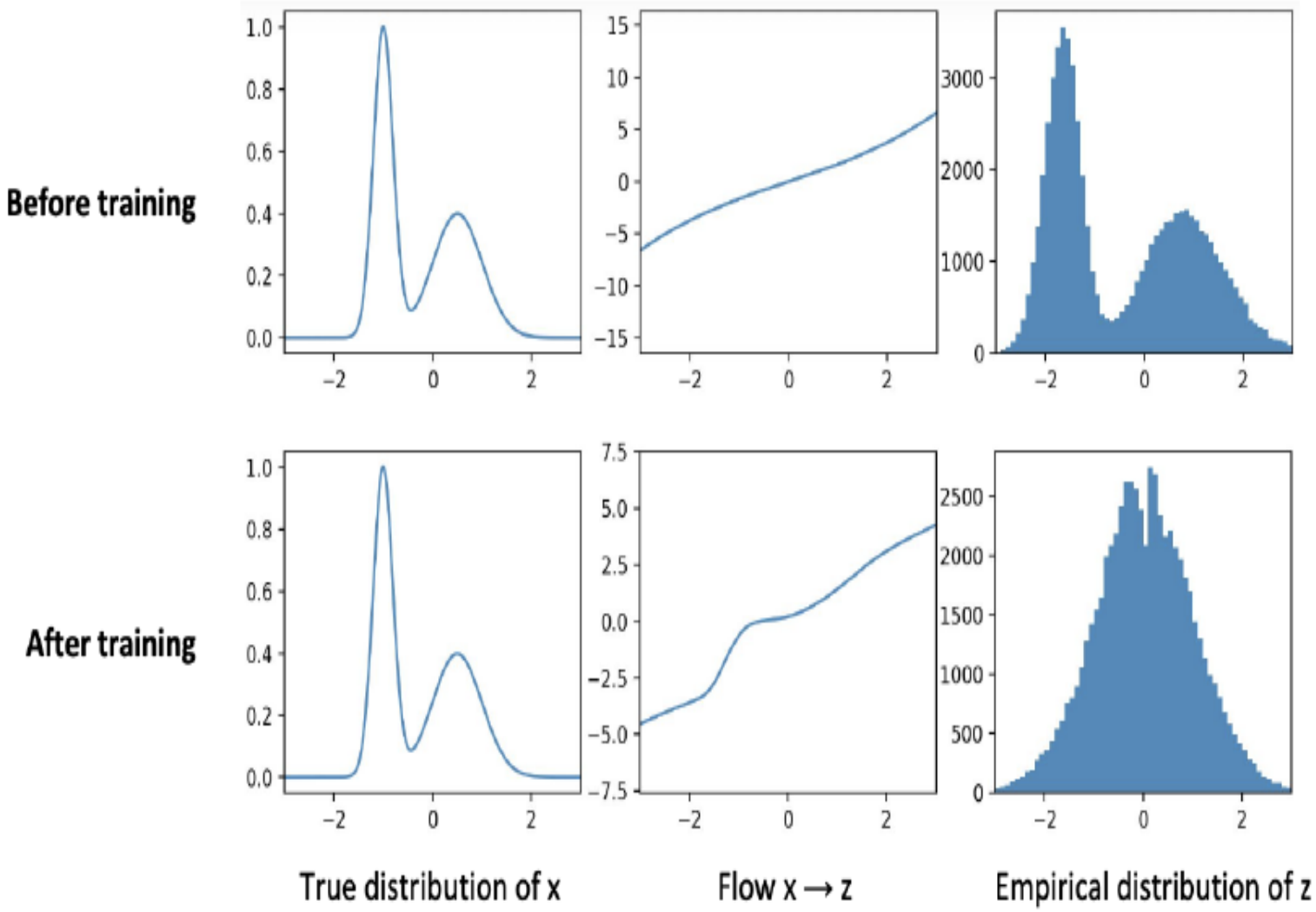
Normalizing Flow





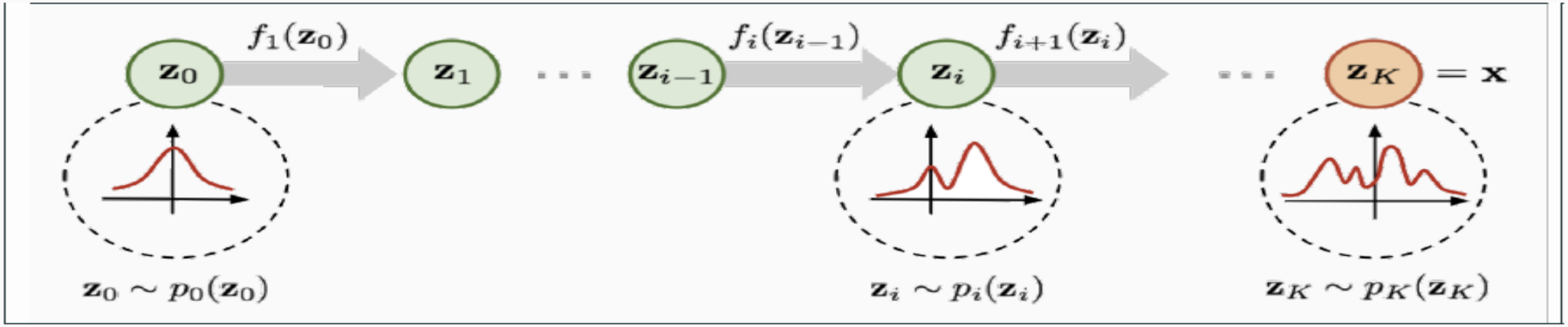
- **Normalizing Flow (NF)**: a flow of invertible transformations that takes $z_0 \sim \mathcal{N}(0, I)$ and outputs x following a complex distribution

A Simple Illustration: X->N(0, 1)



It shows that training a normalizing flow learns a nonlinear, invertible map that turns a complex x-distribution into a near-standard normal z-distribution—making both **exact likelihood evaluation** and **generation** possible.

Normalizing Flow



- **A flow-based generative model** takes samples of x and learns $f_1^{-1}, f_2^{-1}, \cdots f_k^{-1}$ such that

$$z_0 = f_1^{-1} \circ f_2^{-1} \circ \cdots \circ f_K^{-1}(x)$$

- Once the functions are learned, it uses

$$x = f_k \circ f_{k-1} \circ \cdots \circ f_1(z_0)$$

to approximate the distribution of x

A normalizing flow learns a sequence of invertible maps so that data can be transformed into simple Gaussian noise (for training and likelihood), and Gaussian noise can be transformed back into realistic samples (for generation).

Unlocking Normalizing Flows: The Future of Generative Modeling

The Flow Mechanism

Simple Base Distribution
(e.g., Gaussian)

The Power of Diffeomorphisms
Transformations must be differentiable and invertible to ensure the density is well-defined.

Complex Target Distribution
(Real Data)

From Simple Base to Complex Target
A simple distribution is warped through a sequence of invertible bijections to model comples data.

Change of Varisbles Law
Uses Jacobian determinants to quantify volume changes, ensuring the total probability mass remains one.

Jacobian Determinant

Strategic Advantages & Applications

Exact Latent Variable Inference
Unlike VAEs, flows provide a unique, perfectly reversible encoding for every data point.

Stable Likelihood Training
Negative log likelihood can be directly computed and minimized as a loss function for stability.

Adversarial Loss (GAN)

On-Manifold Semantic Perturbations
Modifying latent codes allows for generating realiolis. label preserving data augmentations that improve classifier generalization.

Generative Model Architectures Comparison

	Exact	Likelihood	Latent Inference
Invertibility	Normalizing Flows	GANs	VAEs
	Exact & Reversible	Non-Invertible	Approximate
	Explicitly Modeled	Implicitly Modeled	Lower Bound (ELBO)
	Exact Inference	Hard to Compute	Approximate

NotebookLM

Expressive Power of Flow-Based Models

Normalizing flows can approximate a broad class of smooth densities arbitrarily well via invertible transformations of a simple base distribution. In practice, their expressive power is governed by architectural constraints required for tractable Jacobian determinants (e.g., triangular structure), which can increase the depth needed to capture strong cross-dimensional interactions, sharp multimodality, or near-disconnected supports.

$$p_X(\mathbf{x}) = \prod_{i=1}^{D} p_X(x_i \mid x_{<i})$$

Rosenblatt Transformation

$$z_i = F_i(x_i, x_{<i}) = \int_{-\infty}^{x_i} p_X(x_i' \mid x_{<i})\, dx_i' = \Pr(X_i' \le x_i \mid x_{<i})$$

$$x_i = \left(F_i(\cdot, x_{<i})\right)^{-1}(z_i), \qquad i = 1, \ldots, D$$

$$\det J_F(\mathbf{x}) = \prod_{i=1}^{D} \frac{\partial F_i}{\partial x_i} = \prod_{i=1}^{D} p_X(x_i \mid x_{<i}) = p_X(\mathbf{x})$$

$$p_Z(\mathbf{z}) = p_X(\mathbf{x}) \left|\det J_F(\mathbf{x})\right|^{-1} = p_X(\mathbf{x}) \left|p_X(\mathbf{x})\right|^{-1} = 1$$

Conclusion

The transformed variable
z = F(x)
is uniformly distributed
on the open unit cube (0,1)^D.

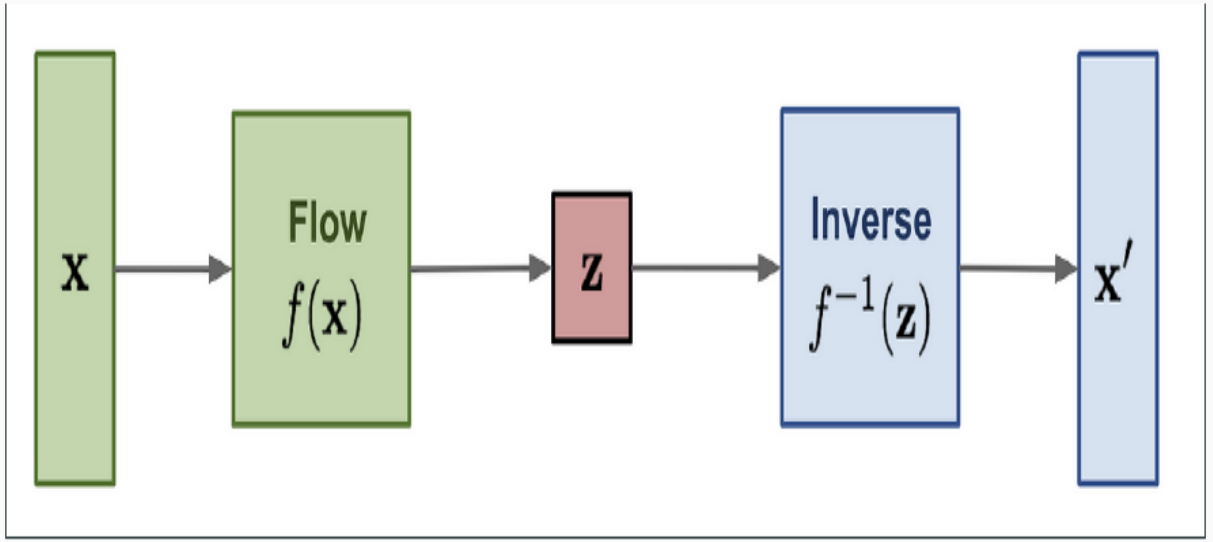
Normalizing Flow – Deriving p(x)

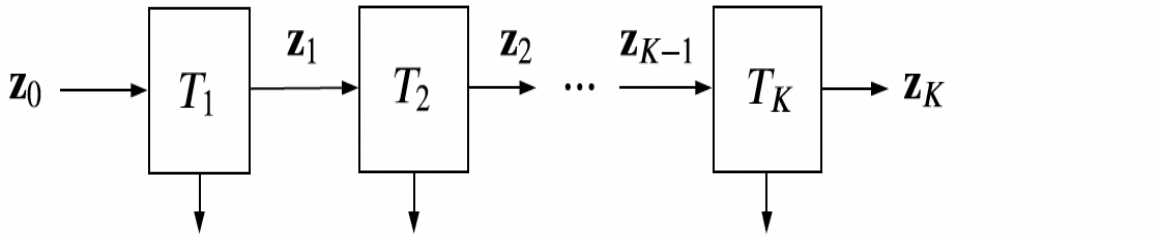
- **Change of Variables Theorem:**

  Let $z \sim \pi(z)$ and $x = f(z)$, where f is invertible. Then

$$p(x) = \pi(z)\left|\det \frac{dz}{dx}\right| = \pi(f^{-1}(x))\left|\det \frac{df^{-1}(x)}{dz}\right|,$$

  where we call the matrix $\frac{df^{-1}(x)}{dx}$ the Jacobian of f^{-1}





- Applying the change of variables theorem recursively and taking the log, we obtain the log-likelihood:

$$\log p(x) = \log \pi_0(z_0) - \sum_{i=1}^{K} \log\left|\det \frac{df_i}{dz_{i-1}}\right|$$

Normalizing Flow

- With NF architecture, the objective function arises naturally. For samples $x^{(1)}, x^{(2)}, \cdots, x^{(N)}$,

$$\mathcal{L}(x^{(1)}, x^{(2)}, \cdots, x^{(N)}) = -\frac{1}{N}\sum_{i=1}^{N} \log p(x^{(i)})$$

- The key is in finding a suitable class of the transformations f_i.
- Challenges: Ideally, the transformations are
  — capable of growing arbitrarily complex
  —- yet their inverse and Jacobian are easy to compute

$$z_0 \sim p_0, \quad z_k = f_k(z_{k-1}), \quad x = z_K.$$

$$-\log p_\theta(x) = -\log p_0(z_0) + \sum_{k=1}^{K} \log \left|\det J_{f_k}(z_{k-1})\right|.$$

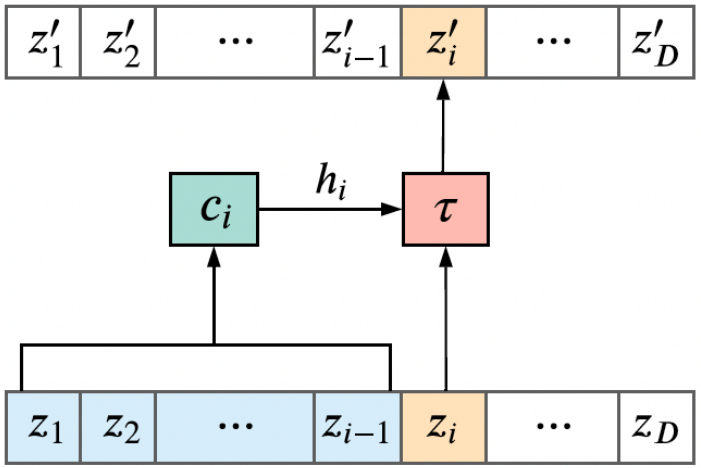
You want f_k to be:

- **Expressive** (can warp distributions a lot), but also
- **Computationally tractable**, meaning:

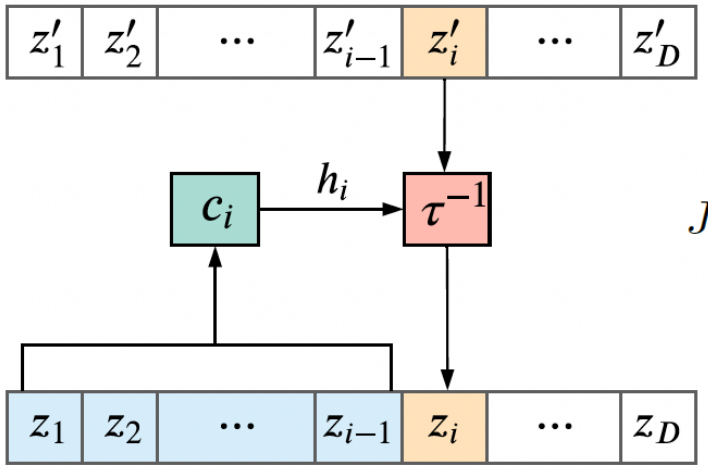
  inverse f_k^{-1} is fast, and $log\,|\det J_{f_k}|$ is fast.

This is why flows use special architectures that yield easy Jacobians.

Autoregressive Flow



(a) Forward



(b) Inverse

$$J_{f_\phi}(\mathbf{z}) = \begin{bmatrix} \frac{\partial \tau}{\partial z_1}(z_1; \boldsymbol{h}_1) & & \mathbf{0} \\ & \ddots & \\ \mathbf{L}(\mathbf{z}) & & \frac{\partial \tau}{\partial z_D}(z_D; \boldsymbol{h}_D) \end{bmatrix}.$$

$$z'_i = \tau(z_i; \boldsymbol{h}_i) \quad \text{where} \quad \boldsymbol{h}_i = c_i(\mathbf{z}_{<i})$$

$$\log \left|\det J_{f_\phi}(\mathbf{z})\right| = \log \left| \prod_{i=1}^{D} \frac{\partial \tau}{\partial z_i}(z_i; \boldsymbol{h}_i) \right| = \sum_{i=1}^{D} \log \left| \frac{\partial \tau}{\partial z_i}(z_i; \boldsymbol{h}_i) \right|$$

$$z_i = \tau^{-1}(z'_i; \boldsymbol{h}_i) \quad \text{where} \quad \boldsymbol{h}_i = c_i(\mathbf{z}_{<i}).$$

τ is termed the *transformer* and c_i the i-th *conditioner*.

The transformer is a strictly monotonic function of z_i (and therefore invertible)

Autoregressive Flow: Transformers

Affine transformers

$$\tau(\mathrm{z}_i; \boldsymbol{h}_i) = \alpha_i \mathrm{z}_i + \beta_i \quad \text{where} \quad \boldsymbol{h}_i = \{\alpha_i, \beta_i\}$$

$$\log \left| \det J_{f_{\boldsymbol{\phi}}}(\mathbf{z}) \right| = \sum_{i=1}^{D} \log |\alpha_i| = \sum_{i=1}^{D} \tilde{\alpha}_i.$$

Combination-based transformers

- Conic combination: $\tau(\mathrm{z}) = \sum_{k=1}^{K} w_k \tau_k(\mathrm{z})$, where $w_k > 0$ for all k.
- Composition: $\tau(\mathrm{z}) = \tau_K \circ \cdots \circ \tau_1(\mathrm{z})$.

$$\tau(\mathrm{z}_i; \boldsymbol{h}_i) = w_{i0} + \sum_{k=1}^{K} w_{ik} \sigma(\alpha_{ik} \mathrm{z}_i + \beta_{ik}) \quad \text{where} \quad \boldsymbol{h}_i = \{w_{i0}, \ldots, w_{iK}, \alpha_{ik}, \beta_{ik}\}$$

$$\alpha_{ik} > 0 \text{ and } w_{ik} > 0 \text{ for all } k \geq 1$$

Integration-based transformers

$$\tau(\mathrm{z}_i; \boldsymbol{h}_i) = \int_0^{\mathrm{z}_i} g(\mathrm{z}; \boldsymbol{\alpha}_i)\, d\mathrm{z} + \beta_i \quad \text{where} \quad \boldsymbol{h}_i = \{\boldsymbol{\alpha}_i, \beta_i\}$$

$g(\cdot; \boldsymbol{\alpha}_i)$ can be any positive-valued neural network parameterized by $\boldsymbol{\alpha}_i$

$$\tau(\mathrm{z}_i; \boldsymbol{h}_i) = \int_0^{\mathrm{z}_i} \sum_{k=1}^{K} \left(\sum_{\ell=0}^{L} \alpha_{ik\ell}\, \mathrm{z}^{\ell} \right)^2 d\mathrm{z} + \beta_i$$

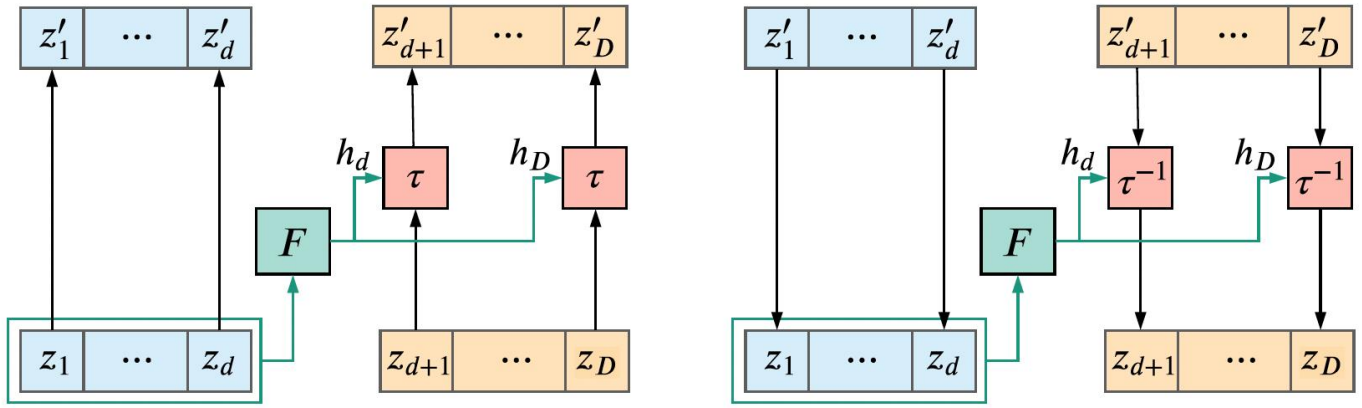
Autoregressive Flow: Conditioner

Recurrent conditioners

$$\boldsymbol{h}_i = c(\boldsymbol{s}_i) \quad \text{where} \quad \begin{aligned} \boldsymbol{s}_1 &= \text{initial state} \\ \boldsymbol{s}_i &= \text{RNN}(\mathrm{z}_{i-1}, \boldsymbol{s}_{i-1}) \text{ for } i > 1. \end{aligned}$$

Coupling layers

- Parameters $(\boldsymbol{h}_1, \ldots, \boldsymbol{h}_d)$ are constants, i.e. not a function of $\mathbf{z}$.
- Parameters $(\boldsymbol{h}_{d+1}, \ldots, \boldsymbol{h}_D)$ are functions of $\mathbf{z}_{\leq d}$ only, i.e. they don't depend on $\mathbf{z}_{>d}$.



(a) Forward (b) Inverse

RealNVP

- Short for "Real-valued non-volume preserving" model
- Each transformation $f_i : \mathbb{R}^D \to \mathbb{R}^D$ for some $i \in \{1, 2, \cdots K\}$ used in this model is termed an **affine coupling layer**

Say $f(x) = y$ is an affine coupling layer. Then for some $d < D$ and some functions $s : \mathbb{R}^d \to \mathbb{R}^{D-d}$ and $t : \mathbb{R}^d \to \mathbb{R}^{D-d}$,

$$\begin{aligned} y_{1:d} &= x_{1:d} \\ y_{d+1:D} &= x_{d+1:D} \odot \exp(s(x_{1:d})) + t(x_{1:d}). \end{aligned}$$

The inverse is

$$\begin{aligned} x_{1:d} &= y_{1:d} \\ x_{d+1:D} &= (y_{d+1:D} - t(y_{1:d})) \odot \exp(-s(y_{1:d})). \end{aligned}$$

RealNVP

- The Jacobian of f is

$$J = \begin{bmatrix} \mathbb{I}_n & 0_{d\times(D-d)} \\ \dfrac{\partial y_{d+1:D}}{\partial x_{1:d}} & diag\left[\exp\left(s(x_{1:d})\right)\right] \end{bmatrix}$$

- Thus

$$det(J) = \prod_{i=1}^{D-d} \exp\left(s(x_{1:d})_i\right) = \exp\left(\sum_{i=1}^{D-d} s(x_{1:d})_i\right)$$

- Remark: s and t does not contribute to the complexity of the inverse or the Jacobian matrix
  - s and t can be arbitrarily complex – deep neural nets

- Each affine flow involves partitioning the elements of the input vector into two groups
  - Termed *masking* by the original paper

- Let b be a *binary mask*, a vector with d ones and $D - d$ zeros. Then we can rewrite y as

$$y = b \odot x + (1 - b) \odot \left(x \odot \exp\left(s(b \odot x)\right) + t(b \odot x)\right)$$

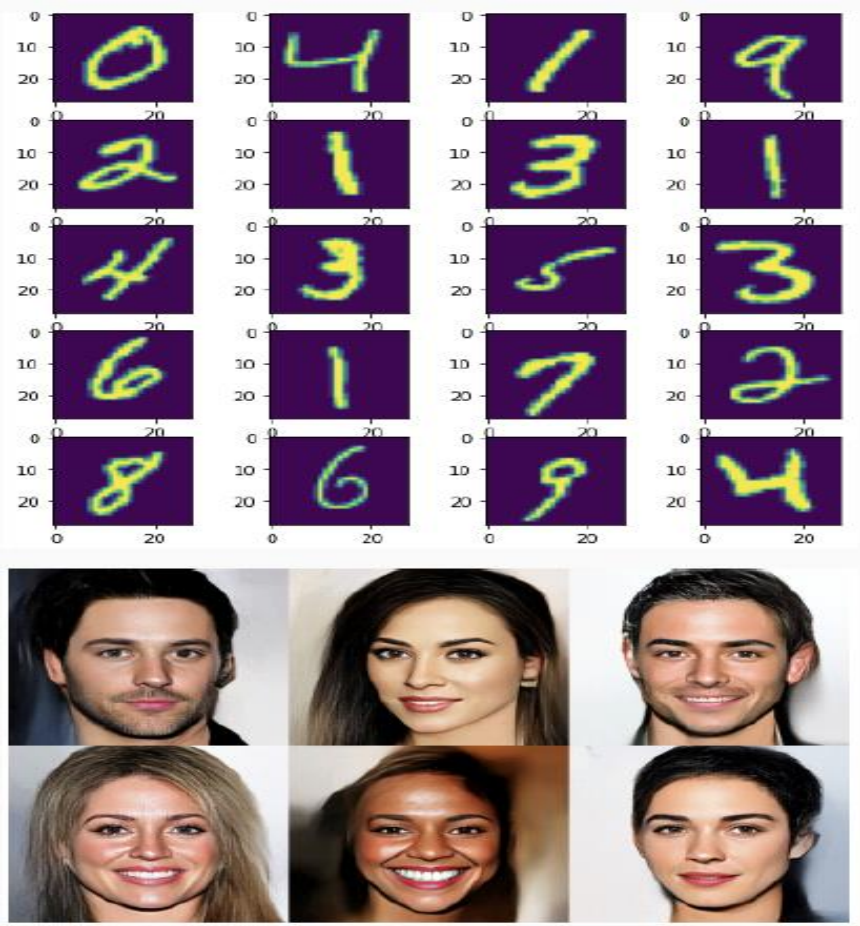
RealNVP – important of good masking

This image illustrates a comparative performance analysis of different **coupling layer masking strategies** in a Normalizing Flow generative model (likely a variant of **RealNVP** or **Glow**), trained on a dataset of human faces (similar to CelebA). The comparison demonstrates how different spatial and channel-wise partitioning affects the model's ability to synthesize coherent global structures and fine details.

Checkerboard x4; channel squeeze; channel x3; channel unsqueeze; checkerboard x3

(Mask top half; mask bottom half; mask left half; mask right half) x2

Generative Images



Flow-Based Models Summary

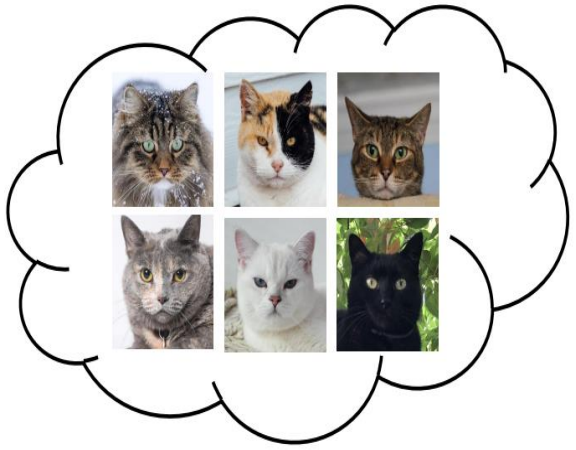
- The ultimate goal: a likelihood-based model with
  - fast sampling
  - fast inference
  - fast training
  - good samples
  - good compression
- Flows seem to let us achieve some of these criteria.
- But how exactly do we design and compose flows for great performance? That is an open question.

Content

6 Diffusion Models

What is diffusion model? –generative model

Learning to generate data



Samples from a Data Distribution

Train

Neural Network

Sample

Top 50 keywords

Keyword	Count
Large Language Models	318
Reinforcement Learning	201
Graph Neural Networks	123
Diffusion Models	112

Sidenote: Top3 conferences in AI are NeurIPS, ICML and ICLR.

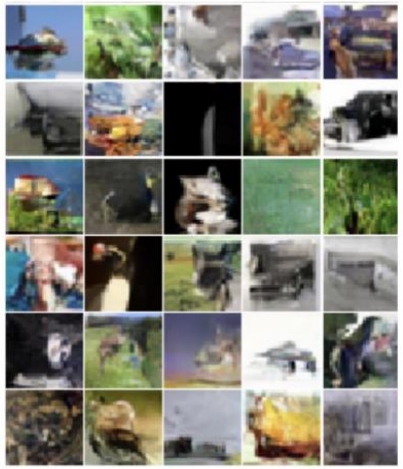
The history of generative models

VAE->GAN->Diffusion

VAEs, 2013

GANs, 2014

PixelCNN, 2016



BigGAN, 2019

Imagen, 2022

VAE → GAN → Diffusion

This summarizes the **evolutionary path** of generative models—from early variational methods, through adversarial training, to diffusion-based architectures, each offering better **fidelity**, **control**, and **semantic understanding**.

Text to Image

DALL·E 2

"a teddy bear on a skateboard in times square"

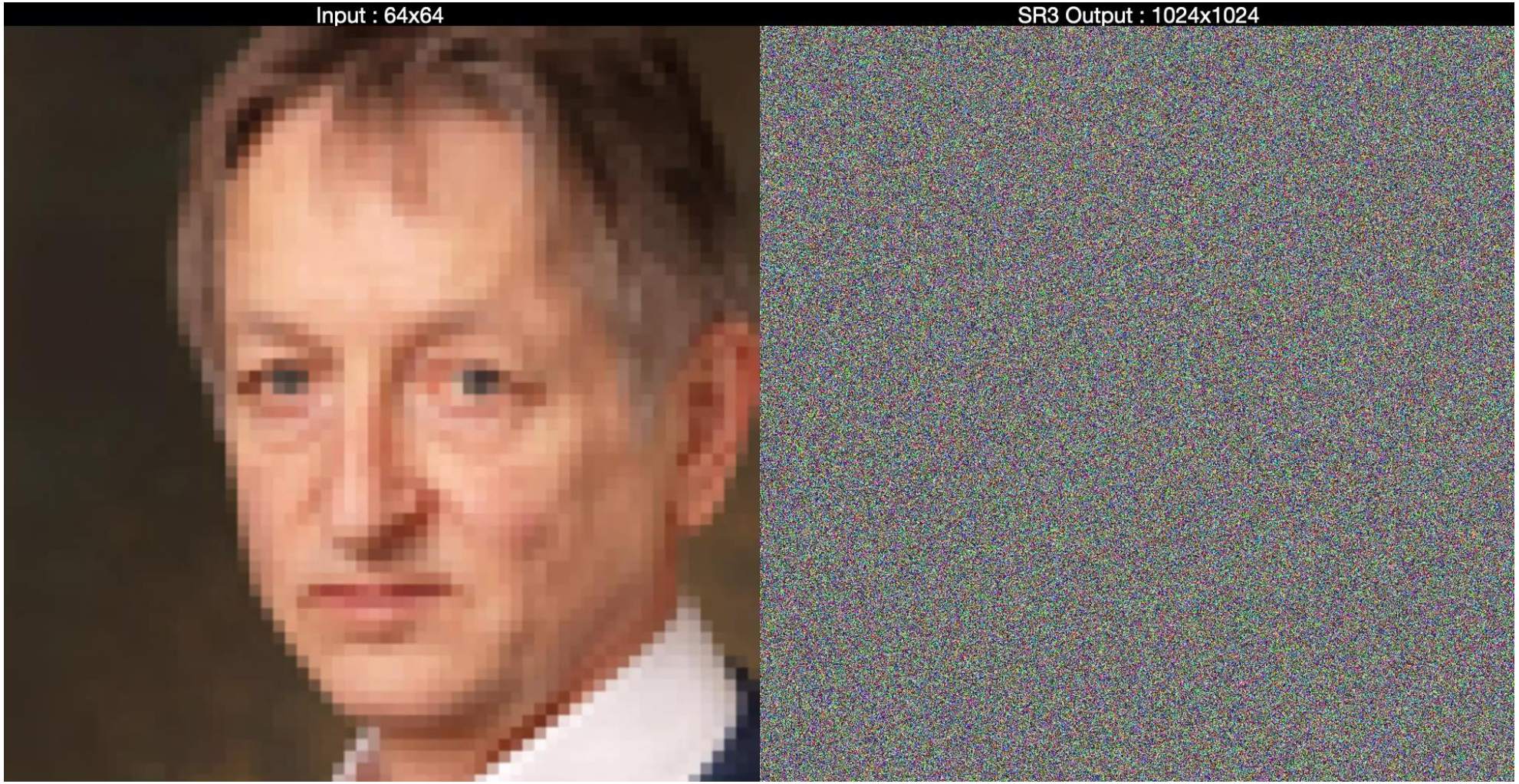
"Hierarchical Text-Conditional Image Generation with CLIP Latents" Ramesh et al., 2022

Imagen

A group of teddy bears in suit in a corporate office celebrating the birthday of their friend. There is a pizza cake on the desk.

"Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding", Saharia et al., 2022

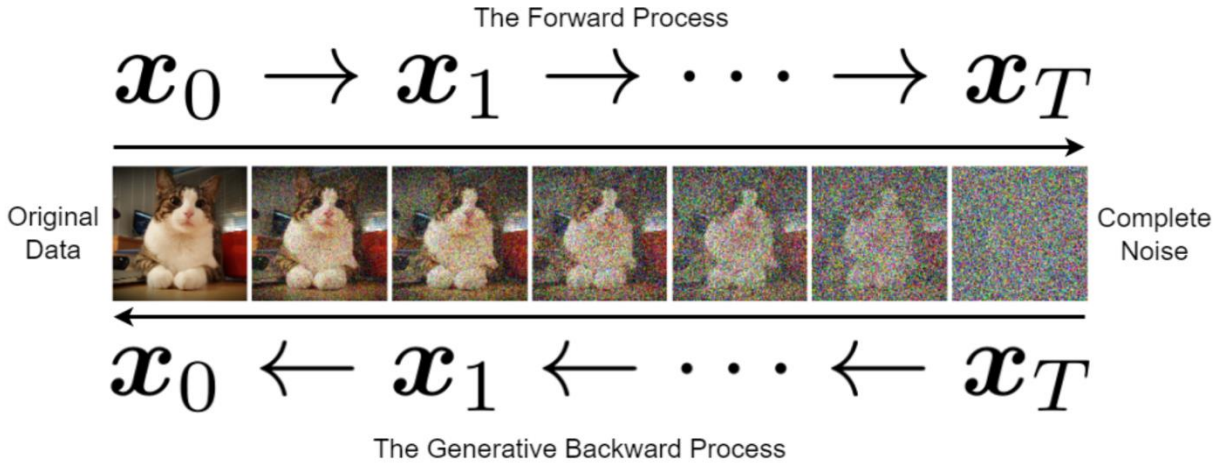
Image Resolution



Discrete time diffusion models

Diffusion models are incremental updates where the assembly of the whole gives us the encoder-decoder structure.

- Forward diffusion process $q(x_t|x_{t-1})$, gradually adds Gaussian noise to an image, until you end up with pure noise.

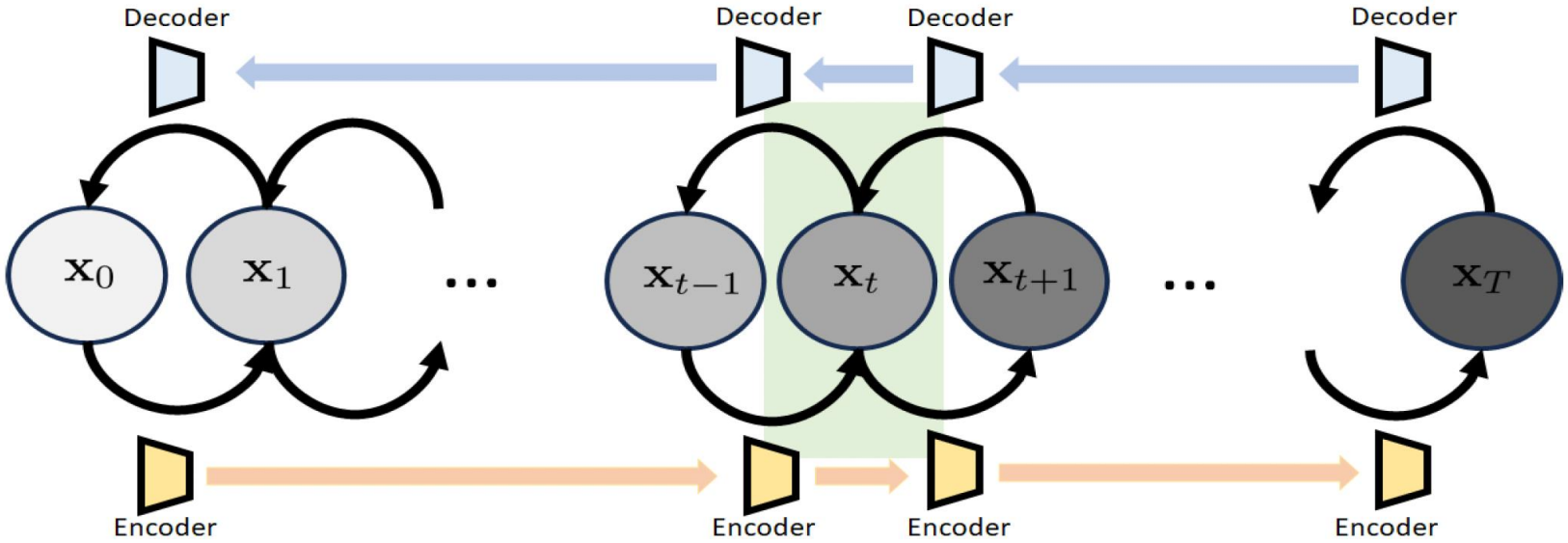


forward from $\mathbf{x}_0$ to $\mathbf{x}_T$: $q_{\boldsymbol{\phi}}(\mathbf{x}_{0:T}) = q(\mathbf{x}_0) \prod_{t=1}^{T} q_{\boldsymbol{\phi}}(\mathbf{x}_t \mid \mathbf{x}_{t-1})$

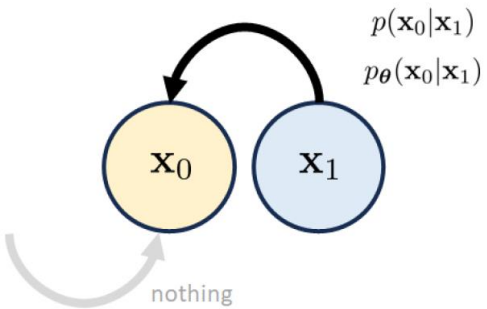
reverse from $\mathbf{x}_T$ to $\mathbf{x}_0$: $p_{\boldsymbol{\theta}}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$

- Reverse denoising diffusion process $p_\theta(x_{t-1}|x_t)$, where a neural network is trained to gradually denoise an image starting from pure noise, until you end up with an actual image.

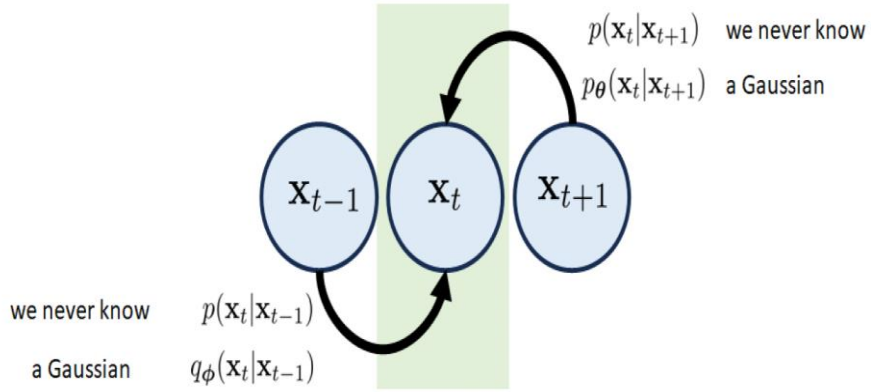
Variational diffusion models



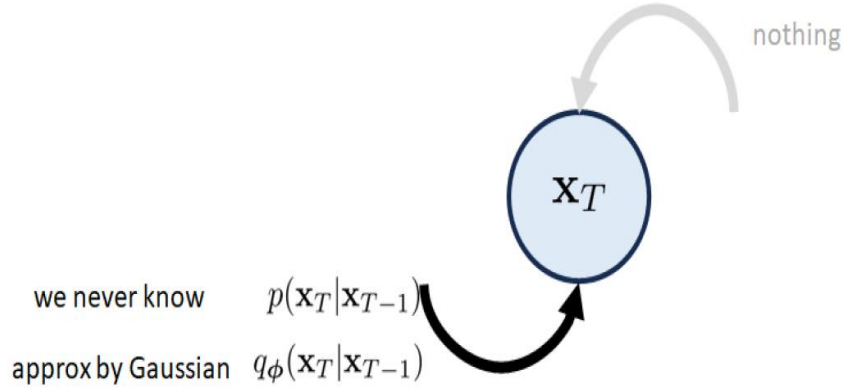
Initial Block



Transitional Block



Final Block



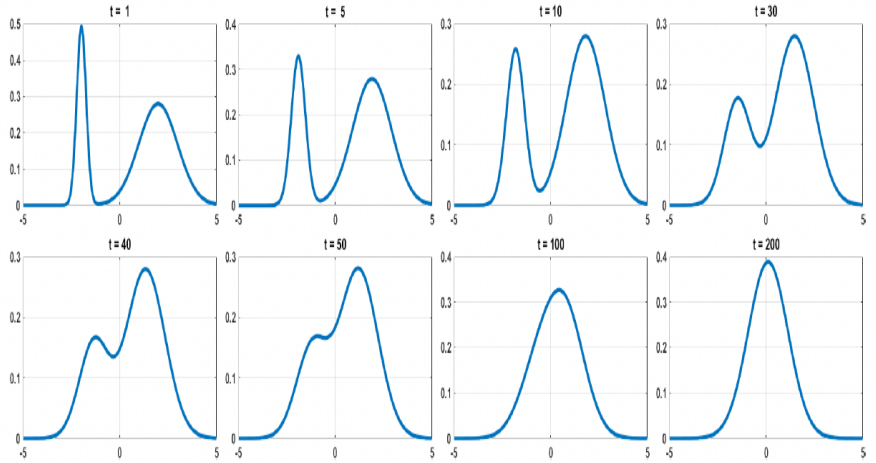
Forward Process

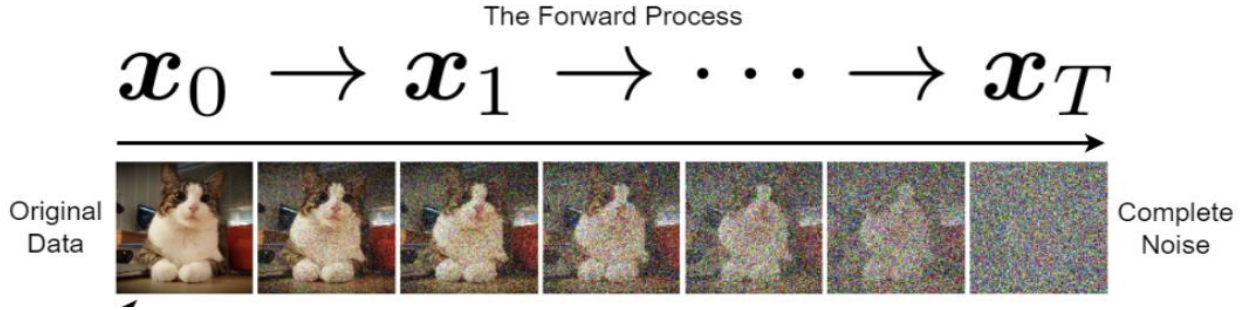
Theorem 2.1. (Why $\sqrt{\alpha}$ and $1-\alpha$?) Suppose that $q_{\boldsymbol{\phi}}(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t \,|\, a\mathbf{x}_{t-1}, b^2\mathbf{I})$ for some constants a and b. If we want to choose a and b such that the distribution of $\mathbf{x}_t$ will become $\mathcal{N}(0, \mathbf{I})$, then it is necessary that

$$a = \sqrt{\alpha} \qquad \text{and} \qquad b = \sqrt{1-\alpha}.$$

Therefore, the transition distribution is

$$q_{\boldsymbol{\phi}}(\mathbf{x}_t|\mathbf{x}_{t-1}) \overset{\text{def}}{=} \mathcal{N}(\mathbf{x}_t \,|\, \sqrt{\alpha}\mathbf{x}_{t-1}, (1-\alpha)\mathbf{I}). \tag{2.4}$$





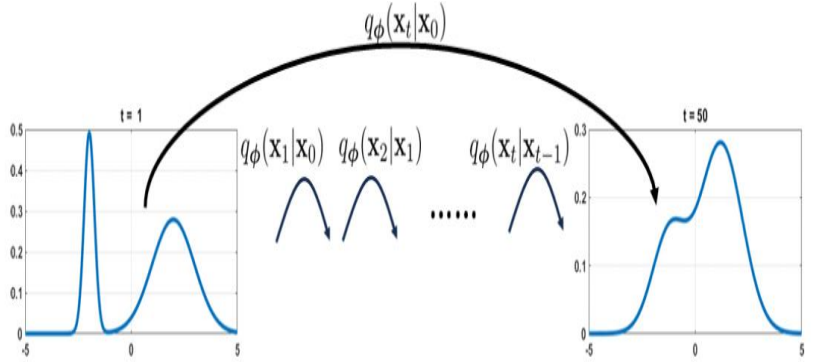
forward from $\mathbf{x}_0$ to $\mathbf{x}_T$:

$$q_{\boldsymbol{\phi}}(\mathbf{x}_{0:T}) = q(\mathbf{x}_0) \prod_{t=1}^{T} q_{\boldsymbol{\phi}}(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

Theorem 2.2. (Conditional Distribution $q_{\boldsymbol{\phi}}(\mathbf{x}_t|\mathbf{x}_0)$). The conditional distribution $q_{\boldsymbol{\phi}}(\mathbf{x}_t|\mathbf{x}_0)$ is given by

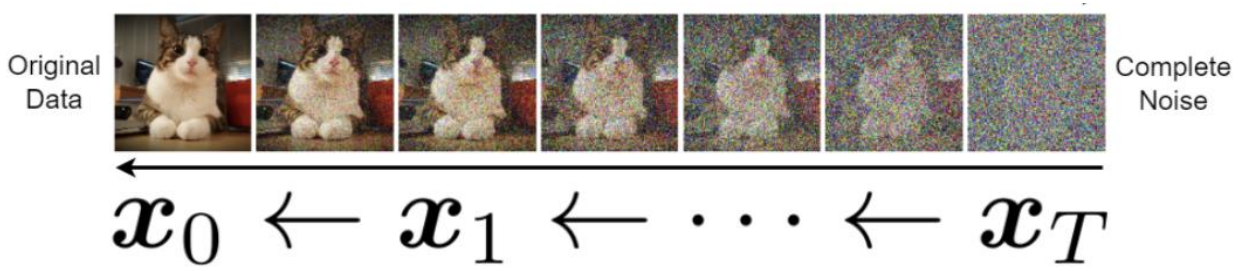
$$q_{\boldsymbol{\phi}}(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t \,|\, \sqrt{\bar{\alpha}_t}\mathbf{x}_0, \; (1-\bar{\alpha}_t)\mathbf{I}), \tag{2.8}$$

where $\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$.



Backward Process

The more interesting part is the backward process, which is realized through a chain of **denoising operations.** Each denoising step should be coupled with the corresponding step in the forward process. We try to obtain $p_\theta(x_{t-1}|x_t)$, so we can generate image x_0 from Gaussian noise x_T. However, the explicit form of $p_\theta(x_{t-1}|x_t)$ is not available, but we can model it with a Gaussian and try to learn the parameters with neural network:



The Generative Backward Process

reverse from $\mathbf{x}_T$ to $\mathbf{x}_0$:

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^{T} p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$

In the original Denoising Diffusion Probabilistic Model (DDPM) paper, the author assume the variance Σ_θ to be fixed and only μ_θ is unknown. In the improved DDPM paper, variance is also modeled.

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}\left(\mathbf{x}_{t-1}| \quad \underbrace{\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t)}_{\text{a network}} \quad , \sigma_q^2(t)\mathbf{I}\right),$$

$$\underbrace{\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t)}_{\text{a network}} \overset{\text{def}}{=} \frac{(1-\overline{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\overline{\alpha}_t}\mathbf{x}_t + \frac{(1-\alpha_t)\sqrt{\overline{\alpha}_{t-1}}}{1-\overline{\alpha}_t} \underbrace{\widehat{\mathbf{x}}_{\boldsymbol{\theta}}(\mathbf{x}_t)}_{\text{another network}} .$$

How to learn the parameters?

Evidence Lower Bound

The training tries to maximize the likelihood of x_0.

For training, we can form variational upper bound (negative ELBO) that is commonly used for training variational autoencoders:

$$\mathbb{E}_{q(\mathbf{x}_0)}\left[-\log p_\theta(\mathbf{x}_0)\right] \leq \mathbb{E}_{q(\mathbf{x}_0)q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\left[-\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}\right] =: L$$

We can obtain the following decomposition:

Theorem 2.3. (ELBO for Variational Diffusion Model). The ELBO for the variational diffusion model is

$$\begin{aligned}\text{ELBO}_{\boldsymbol{\phi},\boldsymbol{\theta}}(\mathbf{x}) = \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{x}_1|\mathbf{x}_0)}\Big[\log \underbrace{p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1)}_{\text{how good the initial block is}}\Big] \\ - \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{x}_{T-1}|\mathbf{x}_0)}\Big[\underbrace{\mathbb{D}_{\text{KL}}\Big(q_{\boldsymbol{\phi}}(\mathbf{x}_T|\mathbf{x}_{T-1})\|p(\mathbf{x}_T)\Big)}_{\text{how good the final block is}}\Big] \\ - \sum_{t=1}^{T-1} \mathbb{E}_{q_{\boldsymbol{\phi}}(\mathbf{x}_{t-1},\mathbf{x}_{t+1}|\mathbf{x}_0)}\Big[\underbrace{\mathbb{D}_{\text{KL}}\Big(q_{\boldsymbol{\phi}}(\mathbf{x}_t|\mathbf{x}_{t-1})\|p_{\boldsymbol{\theta}}(\mathbf{x}_t|\mathbf{x}_{t+1})\Big)}_{\text{how good the transition blocks are}}\Big], \qquad (2.14)\end{aligned}$$

where $\mathbf{x}_0 = \mathbf{x}$, and $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.

$$q_{\boldsymbol{\phi}}(\mathbf{x}_{t-1}, \mathbf{x}_{t+1}|\mathbf{x}_0)$$

- **Challenge**:
  - It's not explicitly known.
  - Even though it's assumed to be **Gaussian**, we still require access to **future samples** ($\mathbf{x}_{t+1}$ (to draw the current sample ($\mathbf{x}_t$,(which is **counterintuitive**.

Evidence Lower Bound

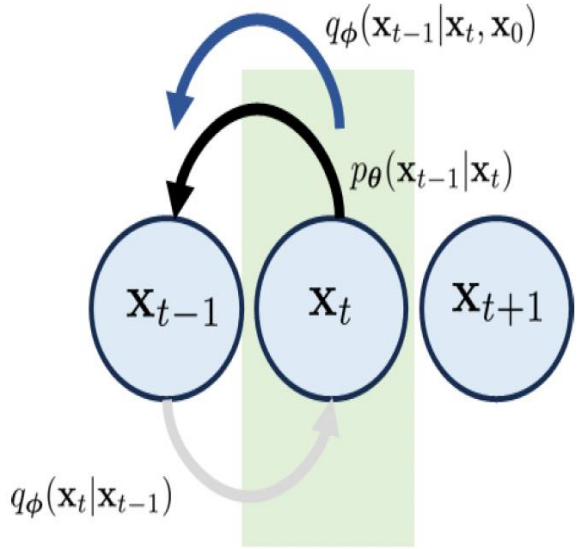
Bayesian Trick.

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t)q(\mathbf{x}_t)}{q(\mathbf{x}_{t-1})} \quad \xrightarrow{\text{condition on } \mathbf{x}_0} \quad q(\mathbf{x}_t|\mathbf{x}_{t-1}, \mathbf{x}_0) = \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)}.$$

Theorem 2.4. (ELBO for Variational Diffusion Model). Let $\mathbf{x} = \mathbf{x}_0$, and $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$. The ELBO for a variational diffusion model in Theorem 2.3 can be equivalently written as

$$\text{ELBO}_{\phi,\theta}(\mathbf{x}) = \mathbb{E}_{q_\phi(\mathbf{x}_1|\mathbf{x}_0)}[\log \underbrace{p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{\text{same as before}}] - \underbrace{\mathbb{D}_{\text{KL}}\left(q_\phi(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T)\right)}_{\text{new prior matching}}$$

$$- \sum_{t=2}^{T} \mathbb{E}_{q_\phi(\mathbf{x}_t|\mathbf{x}_0)} \underbrace{\left[\mathbb{D}_{\text{KL}}\left(q_\phi(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)\|p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)\right)\right]}_{\text{new consistency}}. \qquad (2.21)$$



Distribution of Backward Process

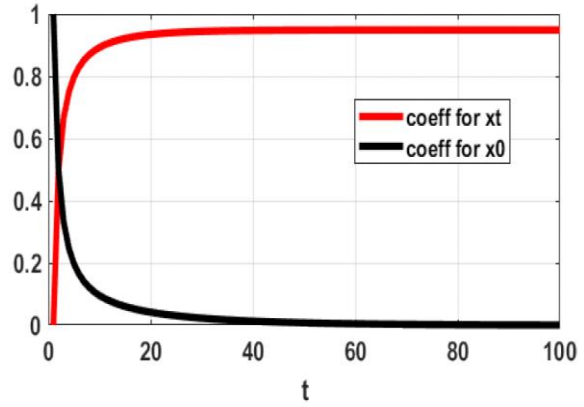
Theorem 2.5. The distribution $q_{\boldsymbol{\phi}}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ takes the form of

$$q_{\boldsymbol{\phi}}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1} \,|\, \boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0), \boldsymbol{\Sigma}_q(t)),$$

where

$$\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0) = \frac{(1-\overline{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\overline{\alpha}_t}\mathbf{x}_t + \frac{(1-\alpha_t)\sqrt{\overline{\alpha}_{t-1}}}{1-\overline{\alpha}_t}\mathbf{x}_0$$
$$\boldsymbol{\Sigma}_q(t) = \frac{(1-\alpha_t)(1-\sqrt{\overline{\alpha}_{t-1}})}{1-\overline{\alpha}_t}\mathbf{I} \stackrel{\text{def}}{=} \sigma_q^2(t)\mathbf{I},$$

where $\overline{\alpha}_t = \prod_{i=1}^{t} \alpha_i$.



: The trajectory of the coefficients for $\mathbf{x}_t$ and for $\mathbf{x}_0$, as t grows.

Theorem 2.6. The **ELBO for a variational diffusion model** in Eqn (2.21) can be simplified to

$$\text{ELBO}_{\boldsymbol{\theta}}(\mathbf{x}) = \mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)}[\log p_{\boldsymbol{\theta}}(\mathbf{x}_0|\mathbf{x}_1)] - \underbrace{\mathbb{D}_{\text{KL}}\Big(q(\mathbf{x}_T|\mathbf{x}_0)\|p(\mathbf{x}_T)\Big)}_{\text{nothing to train}}$$
$$-\sum_{t=2}^{T}\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\Big[\frac{1}{2\sigma_q^2(t)}\|\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0) - \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t)\|^2\Big], \qquad (2.38)$$

where $\mathbf{x} = \mathbf{x}_0$, and $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.

$$q_{\boldsymbol{\phi}}(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\Big(\mathbf{x}_{t-1} \,|\, \underbrace{\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0)}_{\text{known}}, \underbrace{\sigma_q^2(t)\mathbf{I}}_{\text{known}}\Big),$$

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}\Big(\mathbf{x}_{t-1}| \underbrace{\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t)}_{\text{neural network}}, \underbrace{\sigma_q^2(t)\mathbf{I}}_{\text{known}}\Big).$$

Model training

L_{t-1}: Tractable term. From Bayes rule:

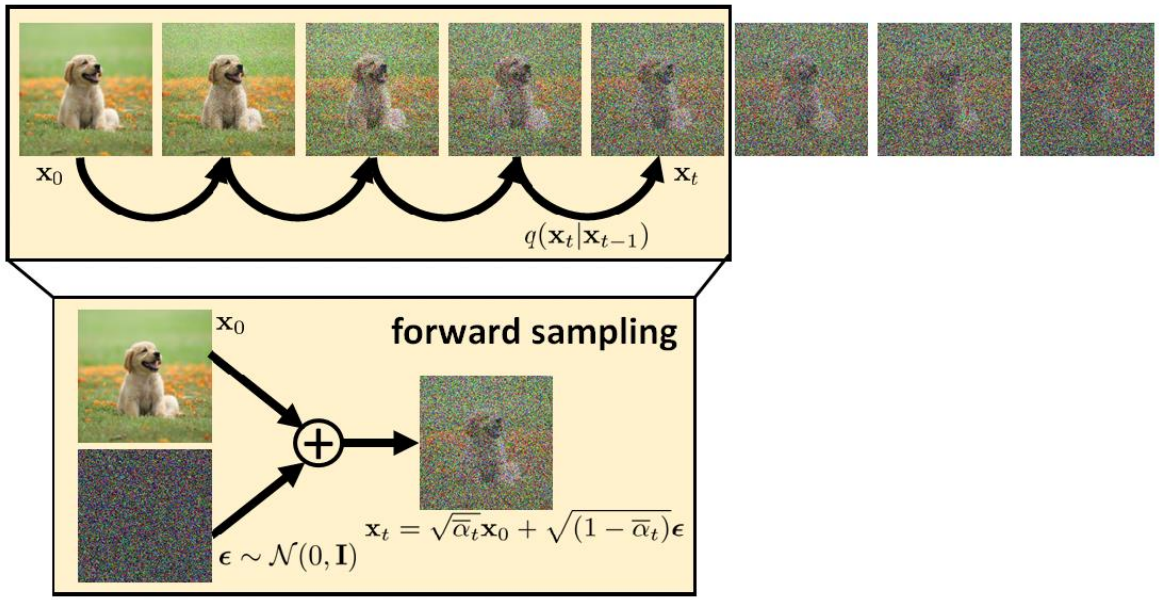
$$q_{\boldsymbol{\phi}}(\mathbf{x}_{t-1}|\mathbf{x}_t,\mathbf{x}_0)=\mathcal{N}\Big(\mathbf{x}_{t-1}\,|\,\underbrace{\boldsymbol{\mu}_q(\mathbf{x}_t,\mathbf{x}_0)}_{\text{known}},\underbrace{\sigma_q^2(t)\mathbf{I}}_{\text{known}}\Big),\qquad \boldsymbol{\mu}_q(\mathbf{x}_t,\mathbf{x}_0)=\frac{(1-\overline{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\overline{\alpha}_t}\mathbf{x}_t+\frac{(1-\alpha_t)\sqrt{\overline{\alpha}_{t-1}}}{1-\overline{\alpha}_t}\mathbf{x}_0.$$

$$p_{\boldsymbol{\theta}}(\mathbf{x}_{t-1}|\mathbf{x}_t)=\mathcal{N}\Big(\mathbf{x}_{t-1}|\quad\underbrace{\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t)}_{\text{neural network}}\quad,\underbrace{\sigma_q^2(t)\mathbf{I}}_{\text{known}}\Big).\qquad \underbrace{\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t)}_{\text{a network}}\overset{\text{def}}{=}\frac{(1-\overline{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\overline{\alpha}_t}\mathbf{x}_t+\frac{(1-\alpha_t)\sqrt{\overline{\alpha}_{t-1}}}{1-\overline{\alpha}_t}\underbrace{\widehat{\mathbf{x}}_{\boldsymbol{\theta}}(\mathbf{x}_t)}_{\text{another network}}.$$

$$\begin{aligned}
&\operatorname*{argmax}_{\boldsymbol{\theta}}\sum_{\mathbf{x}_0\in\mathcal{X}}\text{ELBO}(\mathbf{x}_0)\\
&=\operatorname*{argmin}_{\boldsymbol{\theta}}\sum_{\mathbf{x}_0\in\mathcal{X}}\sum_{t=1}^{T}\mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{1}{2\sigma_q^2(t)}\frac{(1-\alpha_t)^2\overline{\alpha}_{t-1}}{(1-\overline{\alpha}_t)^2}\left\|\widehat{\mathbf{x}}_{\boldsymbol{\theta}}\left(\sqrt{\overline{\alpha}_t}\mathbf{x}_0+\sqrt{(1-\overline{\alpha}_t)}\boldsymbol{\epsilon}_t\right)-\mathbf{x}_0\right\|^2\right]\\
&=\operatorname*{argmin}_{\boldsymbol{\theta}}\sum_{\mathbf{x}_0\in\mathcal{X}}\sum_{t=1}^{T}\frac{1}{M}\sum_{m=1}^{M}\frac{1}{2\sigma_q^2(t)}\frac{(1-\alpha_t)^2\overline{\alpha}_{t-1}}{(1-\overline{\alpha}_t)^2}\left\|\widehat{\mathbf{x}}_{\boldsymbol{\theta}}\left(\sqrt{\overline{\alpha}_t}\mathbf{x}_0+\sqrt{(1-\overline{\alpha}_t)}\boldsymbol{\epsilon}_t^{(m)}\right)-\mathbf{x}_0\right\|^2,
\end{aligned}$$

Training Diffusion in DDPM

Forward Diffusion in DDPM

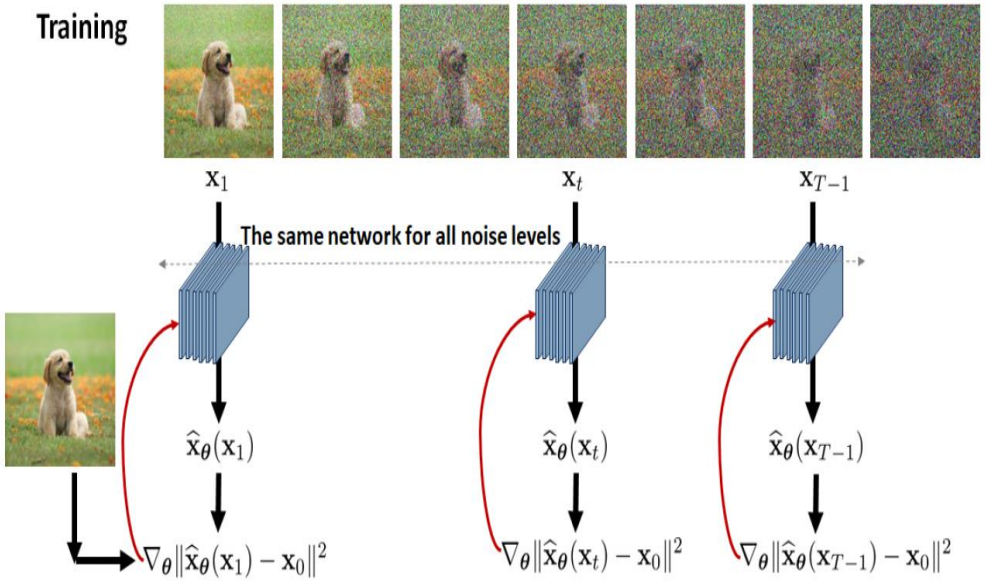


Training Algorithm for DDPM. For every image $\mathbf{x}_0$ in your training dataset:

- Repeat the following steps until convergence.
- Pick a random time stamp $t \sim \text{Uniform}[1, T]$.
- Draw a sample $\mathbf{x}_t^{(m)} \sim \mathcal{N}(\mathbf{x}_t \,|\, \sqrt{\bar{\alpha}_t}\mathbf{x}_0,\ (1-\bar{\alpha}_t)\mathbf{I})$, i.e.,

$$\mathbf{x}_t^{(m)} = \bar{\alpha}_t \mathbf{x}_0 + \sqrt{(1-\bar{\alpha}_t)}\boldsymbol{\epsilon}_t^{(m)}, \qquad \boldsymbol{\epsilon}_t^{(m)} \sim \mathcal{N}(0, \mathbf{I}).$$

Training Diffusion in DDPM

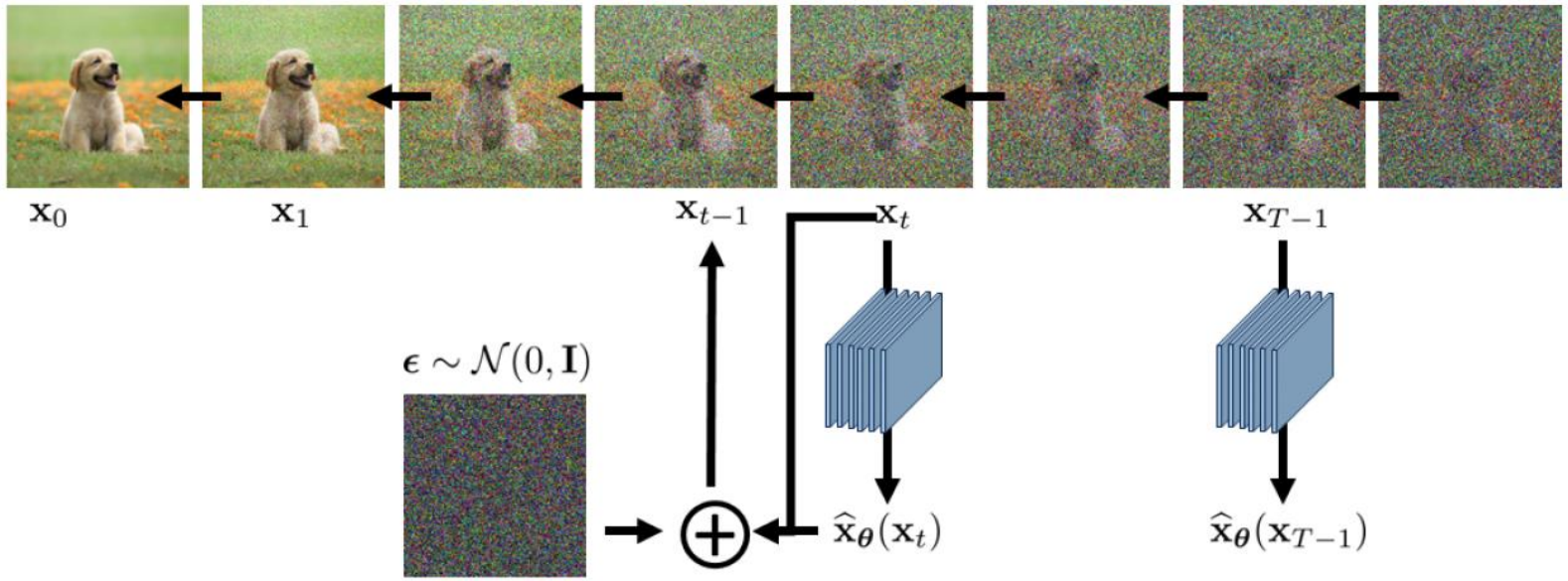


- Take gradient descent step on

$$\nabla_\theta \left\{ \frac{1}{M} \sum_{m=1}^{M} \left\| \hat{\mathbf{x}}_\theta(\mathbf{x}_t^{(m)}) - \mathbf{x}_0 \right\|^2 \right\}.$$

You can do this in batches, just like how you train any other neural networks. Note that, here, you are training **one** denoising network $\hat{\mathbf{x}}_\theta$ for **all** noisy conditions.

Inference in DDPM



Inference of DDPM.

- You give us a white noise vector $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
- Repeat the following for $t = T, T-1, \ldots, 1$.
- We calculate $\widehat{\mathbf{x}}_{\boldsymbol{\theta}}(\mathbf{x}_t)$ using our trained denoiser.
- Update according to

$$\mathbf{x}_{t-1} = \frac{(1-\overline{\alpha}_{t-1})\sqrt{\alpha_t}}{1-\overline{\alpha}_t}\mathbf{x}_t + \frac{(1-\alpha_t)\sqrt{\overline{\alpha}_{t-1}}}{1-\overline{\alpha}_t}\widehat{\mathbf{x}}_{\boldsymbol{\theta}}(\mathbf{x}_t) + \sigma_q(t)\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

Prediction Noise

Training DDPM using $\widehat{\epsilon}_{\boldsymbol{\theta}}(\mathbf{x}_t)$. For every image $\mathbf{x}_0$ in your training dataset:

- Repeat the following steps until convergence.
- Pick a random time stamp $t \sim \text{Uniform}[1, T]$.
- Draw a sample $\boldsymbol{\epsilon}_0 \sim \mathcal{N}(0, \mathbf{I})$.
- Draw a sample $\mathbf{x}_t \sim \mathcal{N}(\mathbf{x}_t \,|\, \sqrt{\overline{\alpha}_t}\mathbf{x}_0, \;\; (1-\overline{\alpha}_t)\mathbf{I})$, i.e.,

$$\mathbf{x}_t = \sqrt{\overline{\alpha}_t}\mathbf{x}_0 + \sqrt{(1-\overline{\alpha}_t)}\boldsymbol{\epsilon}_0.$$

- Take gradient descent step on

$$\nabla_{\boldsymbol{\theta}} \left\| \widehat{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\mathbf{x}_t) - \boldsymbol{\epsilon}_0 \right\|^2.$$

Inference of DDPM using $\hat{\epsilon}_{\theta}(\mathrm{x}_t)$.

- You give us a white noise vector $\mathrm{x}_T \sim \mathcal{N}(0, \mathrm{I})$.
- Repeat the following for $t = T, T-1, \ldots, 1$.
- We calculate $\hat{\mathrm{x}}_{\theta}(\mathrm{x}_t)$ using our trained denoiser.
- Update according to

$$\mathrm{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathrm{x}_t - \frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}}\hat{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\mathrm{x}_t)\right) + \sigma_q(t)\mathrm{z}, \qquad \mathrm{z} \sim \mathcal{N}(0, \mathrm{I}).$$

$$\begin{aligned}
\boldsymbol{\mu}_q(\mathbf{x}_t, \mathbf{x}_0) &= \frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})\mathbf{x}_t + \sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)\mathbf{x}_0}{1-\overline{\alpha}_t} \\
&= \frac{\sqrt{\alpha_t}(1-\overline{\alpha}_{t-1})\mathbf{x}_t + \sqrt{\overline{\alpha}_{t-1}}(1-\alpha_t)\cdot\frac{\mathbf{x}_t - \sqrt{1-\overline{\alpha}_t}\boldsymbol{\epsilon}_0}{\sqrt{\overline{\alpha}_t}}}{1-\overline{\alpha}_t} \\
&= \frac{1}{\sqrt{\alpha_t}}\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}\sqrt{\alpha_t}}\boldsymbol{\epsilon}_0.
\end{aligned}$$

$$\boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t) = \frac{1}{\sqrt{\alpha_t}}\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}\sqrt{\alpha_t}}\widehat{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\mathbf{x}_t).$$

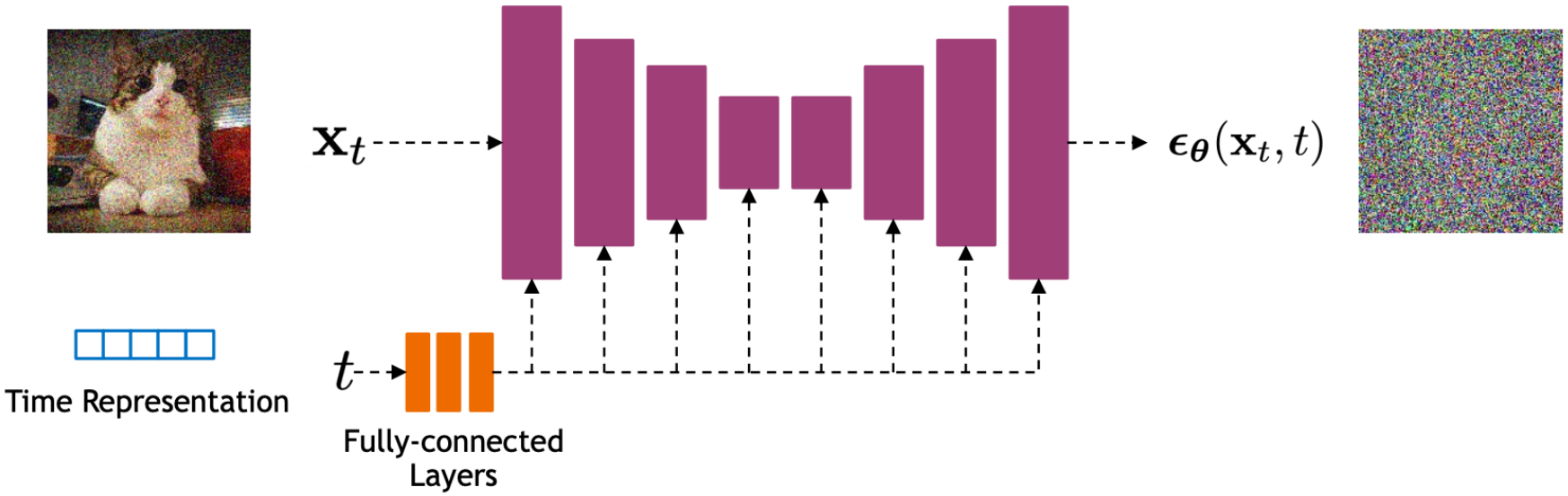
$$\begin{aligned}
\text{ELBO}_{\boldsymbol{\theta}}(\mathbf{x}_0, \boldsymbol{\epsilon}_0) &= -\sum_{t=1}^{T} \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{1}{2\sigma_q^2(t)}\frac{(1-\alpha_t)^2}{(1-\overline{\alpha}_t)\alpha_t}\left\|\widehat{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\mathbf{x}_t) - \boldsymbol{\epsilon}_0\right\|^2\right] \\
&= -\sum_{t=1}^{T} \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)}\left[\frac{1}{2\sigma_q^2(t)}\frac{(1-\alpha_t)^2}{(1-\overline{\alpha}_t)\alpha_t}\left\|\widehat{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}\left(\sqrt{\overline{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\overline{\alpha}_t}\boldsymbol{\epsilon}_0\right) - \boldsymbol{\epsilon}_0\right\|^2\right]
\end{aligned}$$

$$\begin{aligned}
\mathbf{x}_{t-1} &= \boldsymbol{\mu}_{\boldsymbol{\theta}}(\mathbf{x}_t) + \sigma_q(t)\mathbf{z}, \qquad \text{where} \qquad \mathbf{z} \sim \mathcal{N}(0, \mathbf{I}) \\
&= \left(\frac{1}{\sqrt{\alpha_t}}\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}\sqrt{\alpha_t}}\widehat{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\mathbf{x}_t)\right) + \sigma_q(t)\mathbf{z} \\
&= \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\overline{\alpha}_t}}\widehat{\boldsymbol{\epsilon}}_{\boldsymbol{\theta}}(\mathbf{x}_t)\right) + \sigma_q(t)\mathbf{z}.
\end{aligned}$$

Model training

So the loss minimization degenerate to noise prediction task, which is minimizing the noise prediction MSE:

In practice, they use Unet with time encoding to construct ϵ.



Denoising Diffusion Implicit Model (DDIM)

It is very slow to generate a sample from DDPM by following the Markov chain of the reverse diffusion process, as T can be up to one or a few thousand steps.

DDPM Transition Distribution is defined as

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) \overset{\text{def}}{=} \mathcal{N}\Big(\mathbf{x}_t \,|\, \sqrt{\alpha_t}\mathbf{x}_{t-1}, (1-\alpha_t)\mathbf{I}\Big)$$

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}\Big(\mathbf{x}_t \,|\, \sqrt{\bar{\alpha}_t}\mathbf{x}_0, \quad (1-\bar{\alpha}_t)\mathbf{I}\Big)$$

Markov Property:

- The forward diffusion process is memoryless.
- Each step only depends on the immediate previous step.
- This can lead to slow convergence.

DDIM Transition Distribution is defined as

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}\Big(\sqrt{\alpha_t}\mathbf{x}_0, \quad (1-\alpha_t)\mathbf{I}\Big),$$

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}\left(\sqrt{\alpha_{t-1}}\mathbf{x}_0 + \sqrt{1-\alpha_{t-1}}\left(\frac{\mathbf{x}_t - \sqrt{\alpha_t}\mathbf{x}_0}{\sqrt{1-\alpha_t}}\right) \quad , \quad \sigma_t^2\mathbf{I}\right)$$

DDIM Rational:

$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) \overset{\text{def}}{=} \mathcal{N}\Big(\mathbf{x}_t \,\Big|\, \sqrt{\frac{\alpha_t}{\alpha_{t-1}}}\mathbf{x}_{t-1}, \quad (1-\frac{\alpha_t}{\alpha_{t-1}})\mathbf{I}\Big)$$

$$q(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}\Big(\sqrt{\alpha_t}\mathbf{x}_0, \quad (1-\alpha_t)\mathbf{I}\Big)$$

$$\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}, \qquad \text{where} \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

$$\mathbf{x}_{t-1} = \sqrt{\alpha_{t-1}}\mathbf{x}_0 + \sqrt{1-\alpha_{t-1}}\boldsymbol{\epsilon}, \qquad \text{where} \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

$$\mathbf{x}_{t-1} = \sqrt{\alpha_{t-1}}\mathbf{x}_0 + \sqrt{1-\alpha_{t-1}}\boldsymbol{\epsilon}$$

$$= \sqrt{\alpha_{t-1}}\mathbf{x}_0 + \sqrt{1-\alpha_{t-1}}\left(\frac{\mathbf{x}_t - \sqrt{\alpha_t}\mathbf{x}_0}{\sqrt{1-\alpha_t}}\right).$$

Speed up Diffusion Model Sampling

One simple way is to run a strided sampling schedule by taking the sampling update every T/S steps to reduce the process from T to S steps. The new sampling schedule for generation is $\{\tau_1, \ldots, \tau_S\}$ where $\tau_1 \leq \tau_2 \leq \cdots \leq \tau_S \in [1, T]$.

$$q_{\sigma,\tau}(\mathbf{x}_{\tau_{t-1}}|\mathbf{x}_{\tau_t}, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{\tau_{t-1}}; \sqrt{\bar{\alpha}_{\tau_{t-1}}}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_{\tau_{t-1}} - \sigma^2_{\tau_t}}\frac{\mathbf{x}_{\tau_i} - \sqrt{\bar{\alpha}_{\tau_t}}\mathbf{x}_0}{\sqrt{1 - \bar{\alpha}_{\tau_t}}}, \sigma^2_{\tau_t}\mathbf{I})$$

Can be founded in improved DDPM paper

The other approach is adjusting the sample stochasticity, which is adopted in the denoising diffusion implicit model(DDIM) paper. Specifically,

$$\sigma^*_t = \eta\sigma_t.$$

You will try DDIM yourself in HW

In the DDIM paper, they set $\eta = 0$ making the sampling process deterministic. They observed that DDIM ($\eta = 0$) can produce the best quality samples when S is small, while DDPM ($\eta = 1$) performs much worse on small S. DDPM does perform better when we can afford to run the full reverse Markov diffusion steps ($S = T$).

Conditional diffusion models

The models we introduced before are all unconditional models. We have no control on the generating process and the contents we want. For example, the image trained using ImageNet can not generate cats specifically.

Unconditional models are often used to explore the capability of the generation model, to build a product usable for users, we have to make it able to follow instructions, which is the conditional diffusion model.

Conditional diffusion models can be generally classifies into two types, **classifier guidance model and classifier free model.** The classifier guidance model is budget friendly as it only requires a pretrained classifier and can be plug into the pretrained unconditional diffusion models by large companies. On the contrary, classifier free methods need to retrain the whole diffusion model but it can provided better performance and is adopted by large companies.

Conditional diffusion models

(a) Image synthesis with language guidance

A smiling woman with straight, blonde hair wearing sunglasses.

(b) Image synthesis with image guidance

Image Guidance

Generated images

(c) Image synthesis with both language and image guidance

Language Guidance + Image Guidance

A photo of a woman with curly hair. +

Generated images

Classifier guidance

Suppose we have a classifier $p_\phi(y|x_t)$, which we can train by generating training data with the forward process(adding noise to x_0)

We can use that classifier to direct the diffusion model in generating an image corresponding to label y.

Classifier guidance

With bayes rule

$$p_{\theta,\phi}(x_{t-1}|x_t, y) = \frac{p_\theta(x_{t-1}|x_t)p_\phi(y|x_{t-1}, x_t)}{p_\phi(y|x_t)}$$

Adding noise won't affect classifying, $p_\phi(y|x_{t-1}) \approx p_\phi(y|x_{t-1}, x_t)$

$$p_{\theta,\phi}(x_{t-1}|x_t, y) = p_\theta(x_{t-1}|x_t)\frac{p_\phi(y|x_{t-1})}{p_\phi(y|x_t)} = p_\theta(x_{t-1}|x_t)e^{\log p_\phi(y|x_{t-1}) - \log p_\phi(y|x_t)}$$

Apply Taylor expansion

$$\begin{aligned}\log p_\phi(y|x_{t-1}) - \log p_\phi(y|x_t) &\approx (x_{t-1} - x_t)\nabla_{x_t} \log p_\phi(y|x_t) \\ &\approx (x_{t-1} - \mu_\theta(x_t, t))\nabla_{x_t} \log p_\phi(y|x_t)\end{aligned}$$

We know that

$$p_\theta(x_{t-1}|x_t) \sim \mathcal{N}(x_t; \mu_\theta(x_t, t), \sigma_\theta^2(x_t, t)\mathbf{I}) \propto \exp\left(-\frac{(x_t - \mu_\theta(x_t,t))^2}{2\sigma_\theta^2(x_t,t)}\right)$$

So we have

$$\begin{aligned}p_{\theta,\phi}(x_{t-1}|x_t, y) &\propto \exp\left(-\frac{(x_t - \mu_\theta(x_t, t))^2}{2\sigma_\theta^2(x_t, t)} + (x_{t-1} - \mu_\theta(x_t, t))\nabla_{x_t} \log p_\phi(y|x_t)\right) \\ &\propto \exp\left(-\frac{(x_t - \mu_\theta(x_t, t) - \sigma_\theta^2(x_t, t)\nabla_{x_t} \log p_\phi(y|x_t))^2}{2\sigma_\theta^2(x_t, t)}\right)\end{aligned}$$

Classifier guidance

Rewriting the equation on previous page:

$$p_{\theta,\phi}(x_{t-1}|x_t, y) = \mathcal{N}(x_t; \mu_\theta(x_t, t) + \sigma_\theta^2(x_t, t)\nabla_{x_t} \log p_\phi(y|x_t), \sigma_\theta^2(x_t, t)\mathbf{I})$$
$$\sim \frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1-\overline{\alpha_t}}}\epsilon_\theta(x_t, t)) + \sigma_\theta^2(x_t, t)\nabla_{x_t} \log p_\phi(y|x_t) + \sigma_\theta(x_t, t)z$$
$$(z \sim \mathcal{N}(0, \mathbf{I}))$$

As for the unconditioned diffusion model:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}}\left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}}\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\right) + \sigma_t\mathbf{z}$$

Comparing these two, we can see that only the term $\sigma_\theta^2(x_t, t)\nabla_{x_t} \log p_\phi(y|x_t)$ is added. In practice, we use the classifier to classify the image x_t generated by the diffusion model and calculate the probability of the target class y. Then we take gradient with respect to x_t and use that to guide the backward sampling process.

Classifier free

As for classifier free model, just need to apply Bayes Theorem to substitute the classifier guidance part.

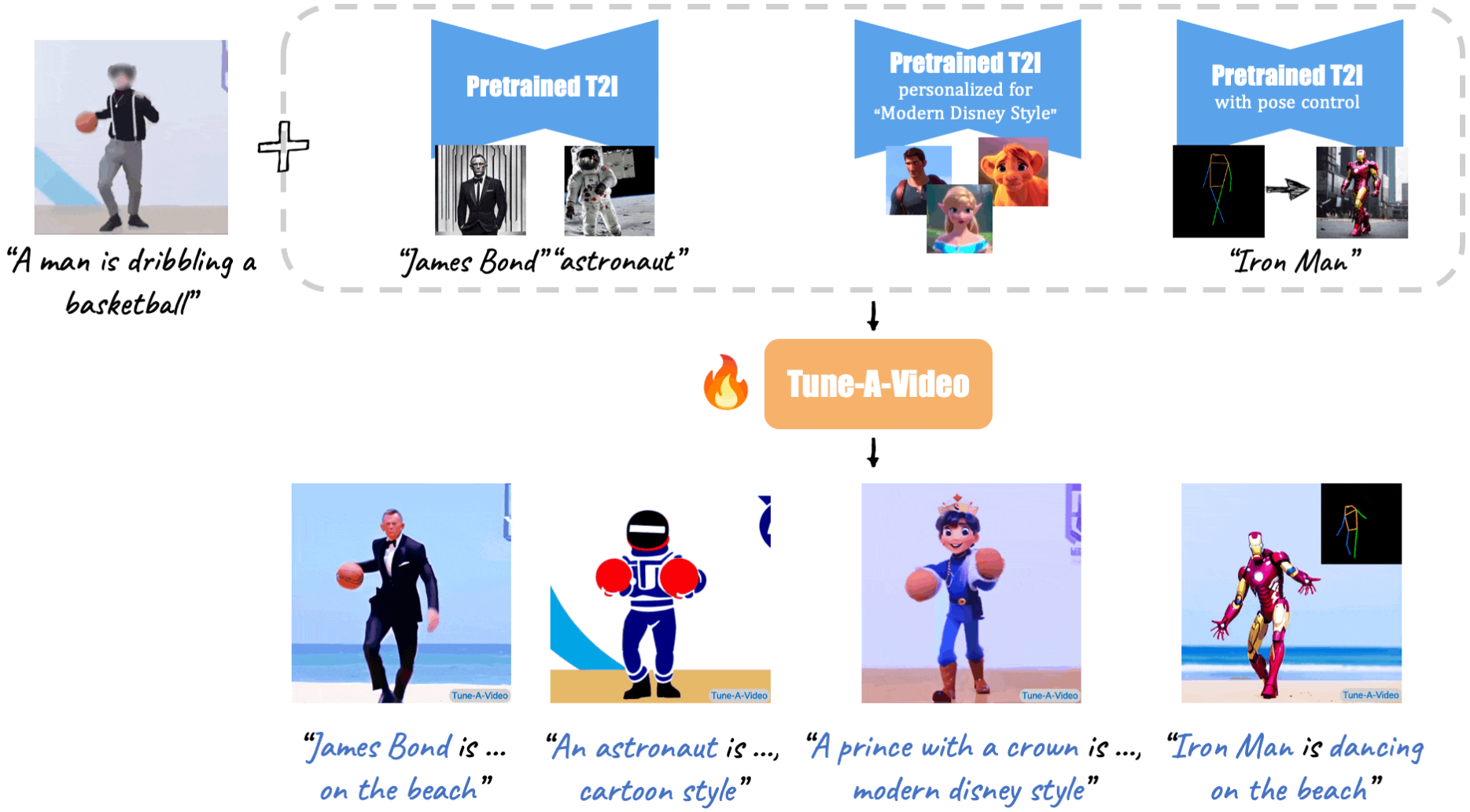
$$\begin{aligned}\nabla_{\mathbf{x}_t} \log p\left(y \mid \mathbf{x}_t\right) &= \nabla_{\mathbf{x}_t} \log p\left(\mathbf{x}_t \mid y\right) - \nabla_{\mathbf{x}_t} \log p\left(\mathbf{x}_t\right) \\ &= -\frac{1}{\sqrt{1-\bar{\alpha}_t}}\left(\boldsymbol{\epsilon}_\theta\left(\mathbf{x}_t, t, y\right) - \boldsymbol{\epsilon}_\theta\left(\mathbf{x}_t, t\right)\right)\end{aligned}$$

Then we can merge the

$$\begin{aligned}\bar{\boldsymbol{\epsilon}}_\theta\left(\mathbf{x}_t, t, y\right) &= \boldsymbol{\epsilon}_\theta\left(\mathbf{x}_t, t, y\right) - \sqrt{1-\bar{\alpha}_t} w \nabla_{\mathbf{x}_t} \log p\left(y \mid \mathbf{x}_t\right) \\ &= \boldsymbol{\epsilon}_\theta\left(\mathbf{x}_t, t, y\right) + w\left(\boldsymbol{\epsilon}_\theta\left(\mathbf{x}_t, t, y\right) - \boldsymbol{\epsilon}_\theta\left(\mathbf{x}_t, t\right)\right) \\ &= (w+1)\boldsymbol{\epsilon}_\theta\left(\mathbf{x}_t, t, y\right) - w\boldsymbol{\epsilon}_\theta\left(\mathbf{x}_t, t\right)\end{aligned}$$

So we only need to implement one epsilon network and set y to zero with no label.

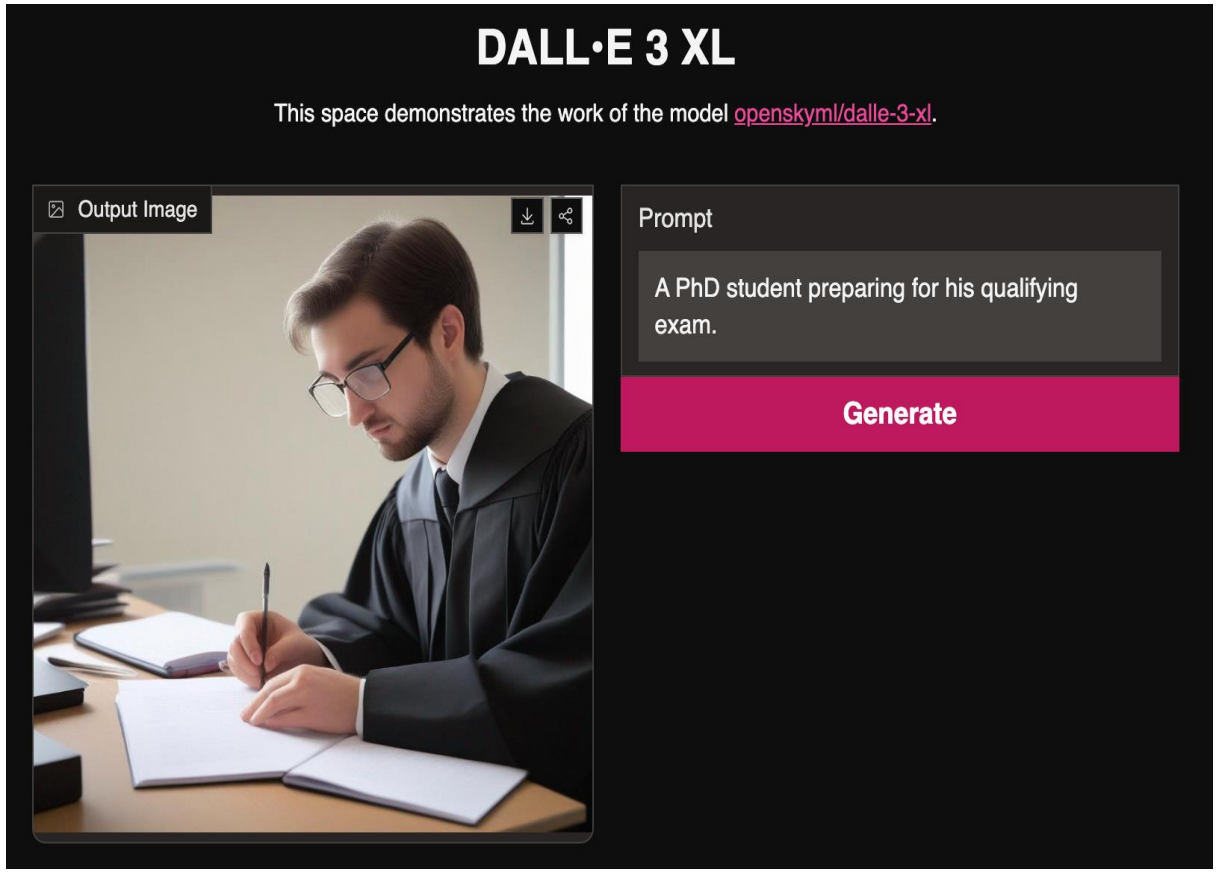
Video Generation



Try Yourself

DALL·E 3

Many diffusion models spaces in huggingface, try them yourself!



Content

7 Score Matching

Score-Matching Langevin Dynamics (SMLD)

Why this chapter matters: from optimization-style updates to genuine distributional sampling

- Sampling from a distribution
- (Stein's) score function
- Score matching techniques
- Concluding remark

Core question

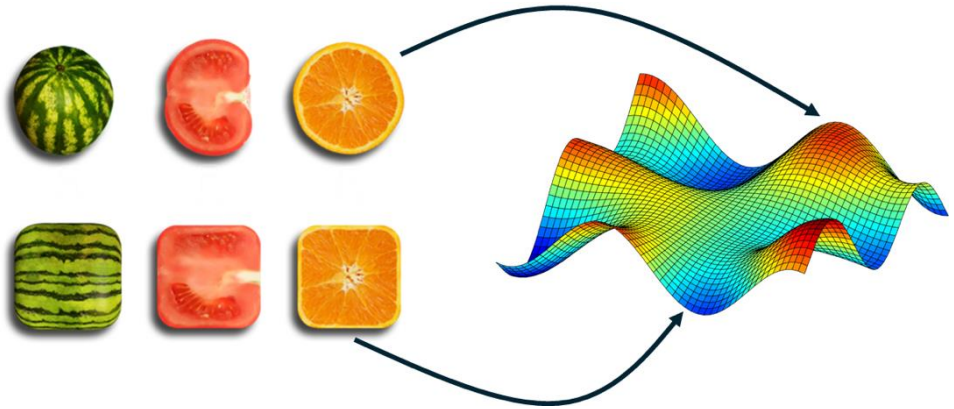
If the target density p(x) is unknown, what quantity do we need in order to sample from it?

Answer

$$\nabla_x \log p(x)$$

Once the score is known, **Langevin dynamics** can turn iterative updates into a sampler.

Motivated example



- We are given a distribution p(x) and we want to draw samples from p(x).
- We expect the underlying distribution p(x) will have a higher value for those "normal-looking" fruit images than those "weird-looking" images.
- Searching for a higher probability can be translated into an optimization:

$$x^\star = \arg\max_x \log p(x)$$

- Different from MLE: in MLE the data x is fixed but the model parameters are changing. Here, the model parameters are fixed but the data point is changing.
- For optimization, the gradient descent algorithm defines a sequence of iterates via the update rule $x_{t+1} = x_t + \tau \nabla_x \log p(x_t)$

From Optimization to Langevin Sampling

Gradient ascent finds a mode; Langevin dynamics adds noise so the process can sample from the whole distribution.

Start with maximization

$$x^\star = \arg\max_x \log p(x)$$

$$x_{t+1} = x_t + \tau \nabla_x \log p(x_t)$$

This is gradient ascent on the log-density. It is useful for finding high-probability points, but by itself it does not generate a distribution of samples.

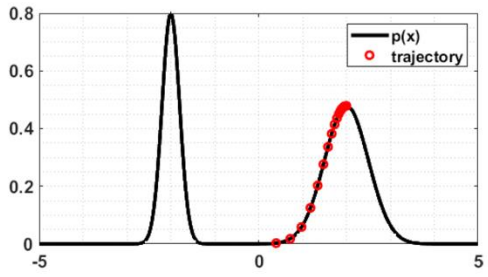
Definition: discrete-time Langevin equation

$$x_{t+1} = x_t + \tau \nabla_x \log p(x_t) + \sqrt{2\tau}\, z, \qquad z \sim \mathcal{N}(0, I)$$

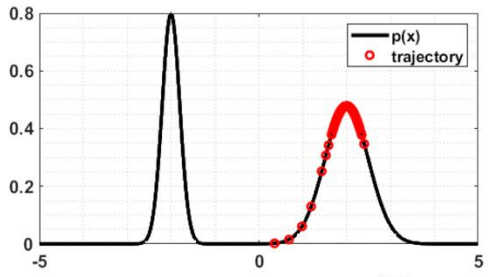
- **Drift term:** moves the point toward higher-density regions
- **Noise term:** prevents collapse to a single point and enables stochastic exploration
- Summary from the chapter: Langevin equation **= gradient descent + noise**.

➢ More in-depth discussions about the Langevin equation will be postponed to the **next section**.

1D Gaussian-mixture intuition



$\mathbf{x}_{t+1} = \mathbf{x}_t + \tau \nabla_{\mathbf{x}} \log p(\mathbf{x}_t)$



$\mathbf{x}_{t+1} = \mathbf{x}_t + \tau \nabla_{\mathbf{x}} \log p(\mathbf{x}_t) + \sqrt{2\tau}\mathbf{z}$

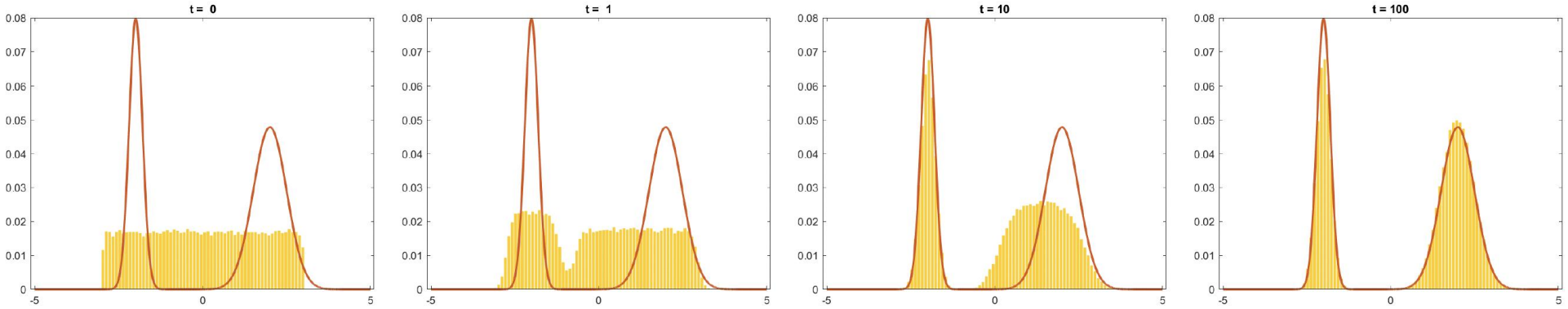
$$p(x) = \pi_1 \mathcal{N}(x \mid \mu_1, \sigma_1^2) + \pi_2 \mathcal{N}(x \mid \mu_2, \sigma_2^2)$$

- x0: initialized at x0=0, We choose τ = 0.05. We run the above gradient descent iteration for T = 500 times, and we plot the trajectory of p(xt) for t = 1, ..., T.
- As we can see in the figure, the sequence {x1, x2, . . . , xT } simply follows the shape of the Gaussian and climb to **one of the peaks**.
- What is more interesting is when we add the **noise** term. Instead of landing at the peak, the sequence xt moves around the peak and finishes somewhere near the peak.
- Gradient ascent: ends at a mode, while Langevin dynamics samples from a distribution

From Optimization to Langevin Sampling

Example. Following the previous example we again consider a Gaussian mixture

- Suppose we initialize M = 10000 uniformly distributed samples $x_0 \sim \text{Uniform}[-3, 3]$. We run Langevin updates for t = 100 steps. The histograms of generated samples are shown in the figures below.



- The figure shows the samples generated by Langevin dynamics. Initially the samples are uniformly distributed. As time progresses, the distribution of the samples become the **desired distribution**.

Why Langevin Dynamics Makes Sense

The Fokker–Planck view explains why the target density becomes a stationary distribution.

Theorem (Fokker–Planck equation)

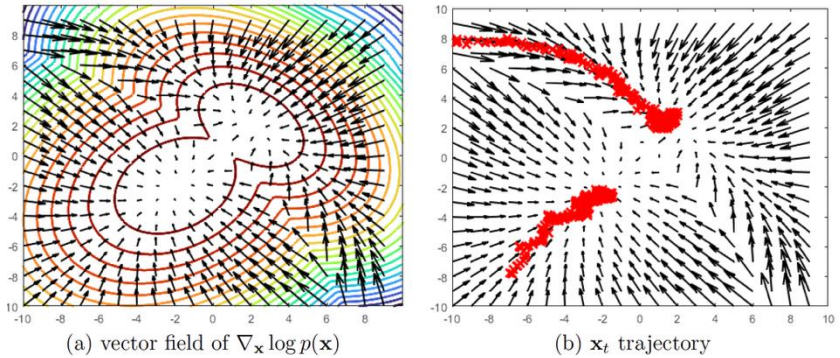
$$\partial_t p(x,t) = -\partial_x\{[\partial_x \log p(x)]\, p(x,t)\} + \partial_x^2 p(x,t)$$

- The random variable x_t has a time-dependent density p(x,t).
- If we plug the target density p(x) into the right-hand side, the PDE becomes zero at stationarity.
- So the desired target density is a stationary distribution of the Langevin process.

Stein's Score function $s_\theta(x) = \nabla_x \log p_\theta(x)$

Ordinary Score function $s_x(\theta) = \nabla_\theta \log p_\theta(x)$

Geometric intuition from the chapter



(a) vector field of $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ (b) $\mathbf{x}_t$ trajectory

- The score defines a vector field over the sample space.
- Vectors are strongest where the change in log p(x) is largest and weaker near peaks.
- A data point approximately follows the arrows toward a basin, while noise keeps the movement stochastic.

The Main Difficulty

SMLD solves this by learning a neural approximation to the score and plugging it back into Langevin dynamics.

Unknown quantity

$$\nabla_x \log p(x)$$

The target density p(x) is inaccessible, so the exact score is also inaccessible.

Learn a neural score model

$$s_\theta(x) \approx \nabla_x \log p(x)$$

Train s_θ directly from samples rather than modeling a normalized density explicitly.

Sampling recursion after training

$$x_{t+1} = x_t + \tau s_\theta(x_t) + \sqrt{2\tau}\, z$$

This is the practical SMLD pipeline: learn the score first, then sample by stochastic updates.

One-slide summary of the pipeline

Training samples from data distribution → **Train score network $s_\theta(x)$** → **Plug score into Langevin update** → **Generate samples from noise**

- It compares three routes: **explicit score matching**, **implicit score matching**, **and denoising score matching (DSM)**.
- The theorems later in the chapter explain why DSM becomes the most interpretable and practical route.

Theorem and Two Classical Routes: ESM vs. ISM

Explicit score matching estimates a surrogate density first; implicit score matching avoids KDE but introduces a Jacobian trace.

Kernel density estimate used in explicit score matching

$$q_h(x) = \frac{1}{M}\sum_{m=1}^{M}\frac{1}{h}K\left(\frac{x - x^{(m)}}{h}\right)$$

Idea: approximate the unknown density p(x) with a kernel density estimate q_h(x).

Theorem: explicit score matching (ESM)

$$J_{\mathrm{ESM}}(\theta) = \tfrac{1}{2}\,\mathbb{E}_{p(x)}\|s_\theta(x) - \nabla_x \log p(x)\|^2 \approx \tfrac{1}{2}\,\mathbb{E}_{q_h(x)}\|s_\theta(x) - \nabla_x \log q_h(x)\|^2$$

- Conceptually simple: estimate a surrogate density, then match its score.
- Weakness noted in the chapter: KDE is poor in high dimensions and with limited samples.
- After training, replace the true score in Langevin dynamics by $s_\theta(x)$.

$$J_{\mathrm{ESM}}(\boldsymbol{\theta}) \approx \frac{1}{M}\sum_{m=1}^{M}\int \|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}) - \nabla_{\mathbf{x}} \log q_h(\mathbf{x})\|^2 \frac{1}{h}K\left(\frac{\mathbf{x} - \mathbf{x}^{(m)}}{h}\right) d\mathbf{x}.$$

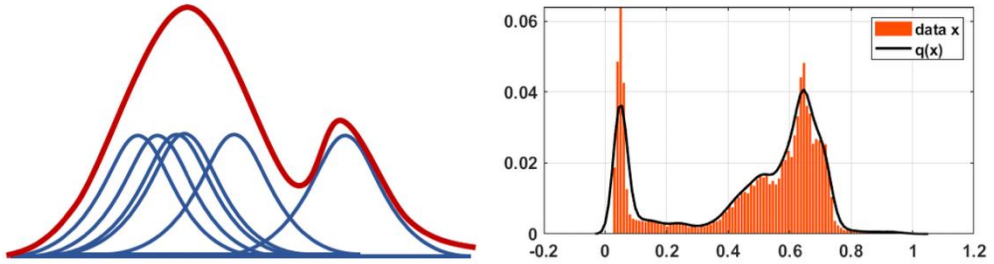


Figure 3.7: Illustration of kernel density estimation.

$$J_{\mathrm{ISM}}(\boldsymbol{\theta}) \approx \frac{1}{M}\sum_{m=1}^{M}\sum_{i}\left(\partial_i \mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}^{(m)}) + \frac{1}{2}|[\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}^{(m)})]_i|^2\right)$$

Implicit score matching (ISM)

$$J_{\mathrm{ISM}}(\theta) = \mathbb{E}_{p(x)}\left[\mathrm{Tr}(\nabla_x s_\theta(x)) + \tfrac{1}{2}\|s_\theta(x)\|^2\right]$$

- No explicit density estimate is required.
- But the trace / Jacobian term can be expensive for deep neural networks.
- The chapter therefore motivates a more practical alternative: DSM.

Denoising Score Matching (DSM)

DSM replaces “estimate an unknown density” by “use a known corruption model,” turning score learning into denoising.

Gaussian corruption model

$$q(x \mid x') = \mathcal{N}(x \mid x', \sigma^2 I),$$

$$x = x' + \sigma z,\ z \sim \mathcal{N}(0, I)$$

$$\nabla_x \log q(x \mid x') = -\frac{x - x'}{\sigma^2} = -\frac{z}{\sigma}$$

DSM objective before simplification

$$J_{\mathrm{DSM}}(\theta) = \mathbb{E}_{q(x,x')}\left[\tfrac{1}{2}\|s_\theta(x) - \nabla_x \log q(x \mid x')\|^2\right]$$

The target score now comes from a known conditional corruption distribution.

Theorem: practical DSM loss

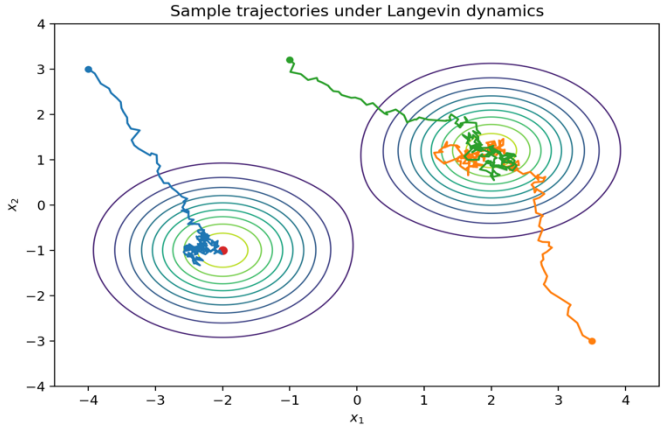
$$J_{\mathrm{DSM}}(\boldsymbol{\theta}) \overset{\mathrm{def}}{=} \mathbb{E}_{q(\mathbf{x},\mathbf{x}')}\left[\frac{1}{2}\left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}) - \nabla_{\mathbf{x}} \log q(\mathbf{x}|\mathbf{x}')\right\|^2\right]$$

$$= \mathbb{E}_{q(\mathbf{x}')}\left[\frac{1}{2}\left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}' + \sigma\mathbf{z}) + \frac{\mathbf{z}}{\sigma}\right\|^2\right].$$

- Input to the network: a noisy sample x+sigma z
- Target: the negative noise direction -z/sigma
- Thus, score learning becomes a denoising task

Interpretation in plain words

- Predicting noise is equivalent to denoising, because any denoised image plus the predicted noise will give us the noisy observation. Therefore, equation in Theorem is a denoising step.
- A noisy sample is displaced away from the data manifold; the learned score tells us how to move back toward high-density regions.
- This is why the chapter emphasizes that “predicting noise is equivalent to denoising.”
- Conceptually, DSM is the cleanest bridge from Chapter 3 to later diffusion-style training objectives.



Why DSM Is Legitimate

Vincent's equivalence theorem explains why the denoising loss can estimate the same score function as explicit score matching, up to a constant.

Theorem

$$J_{\mathrm{DSM}}(\theta) = J_{\mathrm{ESM}}(\theta) + C$$

for a constant C independent of θ

What this theorem buys us

- DSM is not merely a heuristic denoising trick.
- It is theoretically tied to score matching through an objective-equivalence result.
- So we gain a much more interpretable and practical loss without changing the learned optimum.

Comparing the three routes

ESM

- estimate a KDE surrogate
- simple conceptual story
- weak in high dimensions

ISM

- no KDE needed
- uses Jacobian trace
- often hard to scale

DSM

- uses known corruption model
- interpretable denoising target
- most practical in this chapter

Training and Inference

The training procedure in a score matching model is typically done by minimizing the denoising score matching loss function

Training procedure

$$\boldsymbol{\theta}^* = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \; \mathbb{E}_{p(\mathbf{x})}\left[\frac{1}{2}\left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}+\sigma\mathbf{z})+\frac{\mathbf{z}}{\sigma}\right\|^2\right]$$

$$\approx \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \; \frac{1}{M}\sum_{m=1}^{M}\frac{1}{2}\left\|\mathbf{s}_{\boldsymbol{\theta}}\left(\mathbf{x}^{(m)}+\sigma\mathbf{z}^{(m)}\right)+\frac{\mathbf{z}^{(m)}}{\sigma}\right\|^2,$$

Assumes a fixed noise level σ

$$\mathbf{z}^{(m)} \sim \mathcal{N}(0, \mathbf{I}).$$

noise conditioned score network (NCSN) by Song and Ermon

$$J_{\mathrm{NCSN}}(\theta) = \frac{1}{L}\sum_{i=1}^{L}\lambda(\sigma_i)\ell(\boldsymbol{\theta};\sigma_i)$$

where the individual loss function is defined according to the noise levels σ1, . . . , σL:

$$\ell(\boldsymbol{\theta};\sigma) = \mathbb{E}_{p(\mathbf{x})}\left[\frac{1}{2}\left\|\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}+\sigma\mathbf{z})+\frac{\mathbf{z}}{\sigma}\right\|^2\right]$$

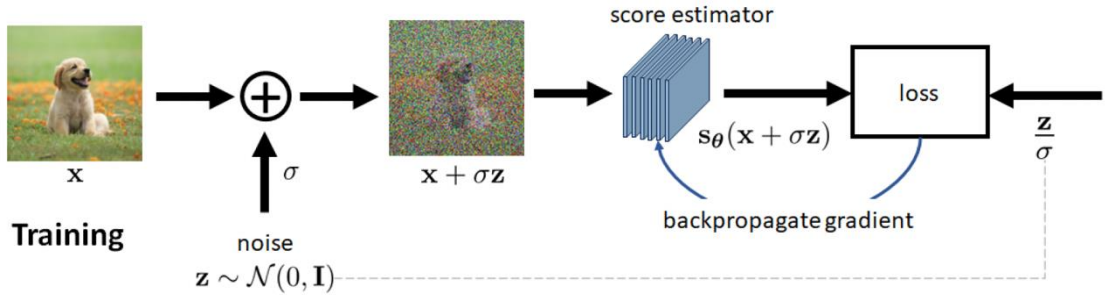
Inference:

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \frac{\alpha_i}{2}\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}_t, \sigma_i) + \sqrt{\alpha_i}\mathbf{z}_t,$$

$$\mathbf{z}_t \sim \mathcal{N}(0, \mathbf{I}),$$

- Assume that we have already trained the score estimator sθ. To generate an image, use the Langevin equation to iteratively draw samples by denoising the image.
- where $\alpha_i = \sigma_i^2/\sigma_L^2$ is the step size and sθ(xt, σi) denotes the score matching function for noise level σi.

Training of s_θ

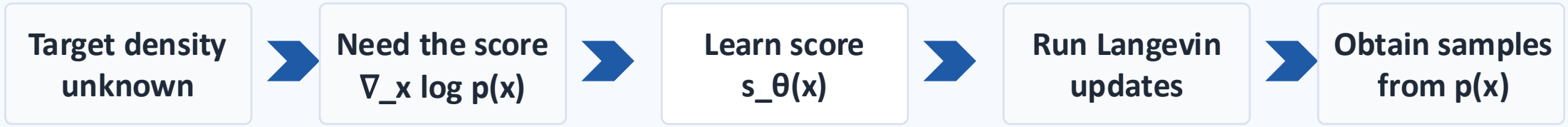


- Figure: Training of s_θ for denoising score matching.
- The network s_θ is trained to estimate the noise.

Summary

One compact storyline: learn the score, then sample by stochastic updates.

One Pipeline



Three takeaways worth saying out loud

- Theorem explains why Langevin dynamics is connected to the target distribution.
- The score function is the central quantity because it determines the drift direction.
- DSM is the most presentation-friendly result: it turns score learning into denoising.

Connection to later diffusion models

- It is a conceptual foundation rather than the final industrial algorithm.
- The key idea that survives: learn the score and use stochastic updates to generate data.
- That is why it is a natural bridge to score-based and diffusion models.

Content

8 Applications

What can diffusion do in bio research?

A short deck on where diffusion models help most: images, single-cell / multi-omics, molecular design, and patient-level simulation.

Generate | Denoise | Impute | Condition

Core idea A diffusion model learns how to reverse structured noise. In biology, that translates naturally into realistic data generation, missing-data completion, modality translation, and uncertainty-aware prediction.

Best use cases high-dimensional • noisy • multimodal • conditional generation • sparse observations

A useful mental model Diffusion is not just an image generator. In biology it is often best viewed as a flexible conditional sampler over complex scientific objects: images, cells, molecules, or patient trajectories.

Biological measurements are noisy

- Microscopy, MRI, CT, sequencing, and EHR-derived features all contain structured noise
- Diffusion models are trained precisely to reverse small noise increments.

Missingness is common

- Missing modalities, missing genes, sparse peaks, and incomplete images are routine.
- Conditional denoising naturally supports imputation and completion.

Biology is multi-scale

- Cell states, tissues, organs, and molecules interact across levels.
- Diffusion models can be conditioned on labels, covariates, time, or structure.

Biomedical imaging: where diffusion is already strongest

Image restoration

- Denoising for low-dose CT, accelerated MRI, and noisy microscopy.
-
- Super-resolution, deblurring, motion correction, and inpainting.

Cross-modality translation

- MRI→CT, synthesis of missing contrasts, or scanner harmonization.
-
- Useful when one modality is expensive, risky, or unavailable.

Data augmentation

- Generate realistic rare lesions, pathologies, or histology patterns.
-
- Can improve robustness when carefully validated.

Main benefit

Diffusion models are especially compelling when the task already looks like denoising or inverse reconstruction: start with a degraded observation and iteratively recover a plausible clean object.

Caution

Visual realism is not enough. In bioimaging, evaluation must also check anatomy, biomarkers, calibration, and out-of-site generalization.

Representative tasks reconstruction • segmentation assistance • synthesis • harmonization • uncertainty-aware restoration

MRI Image generation

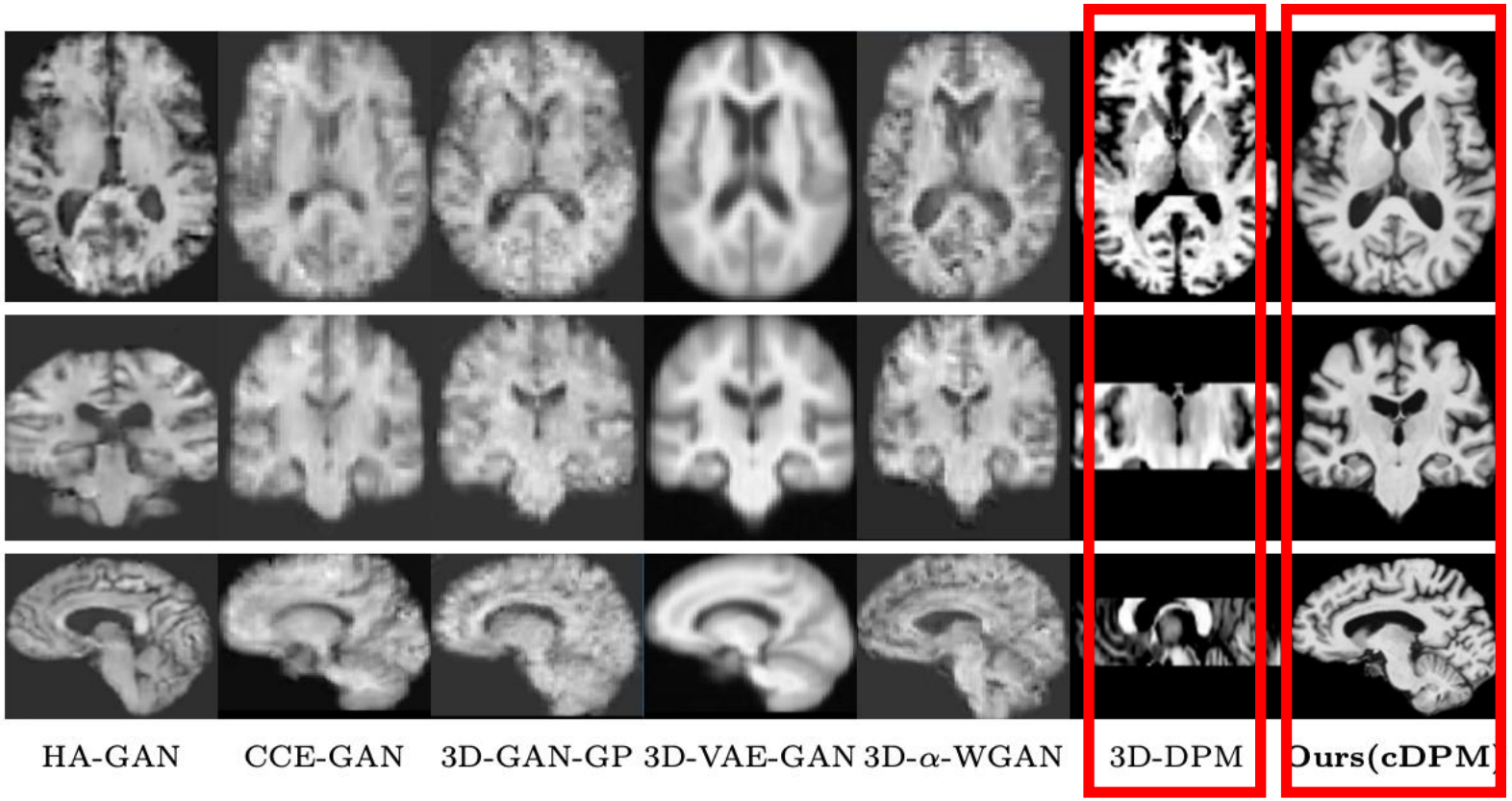


Fig. 4. 3 views of MRIs generated by 7 models. Compared to the MRIs produced by the other approaches, our cDPM model generates the most realistic MRI scans that provide more distinct gray matter boundaries and greater anatomical details.

MRI image reconstruction

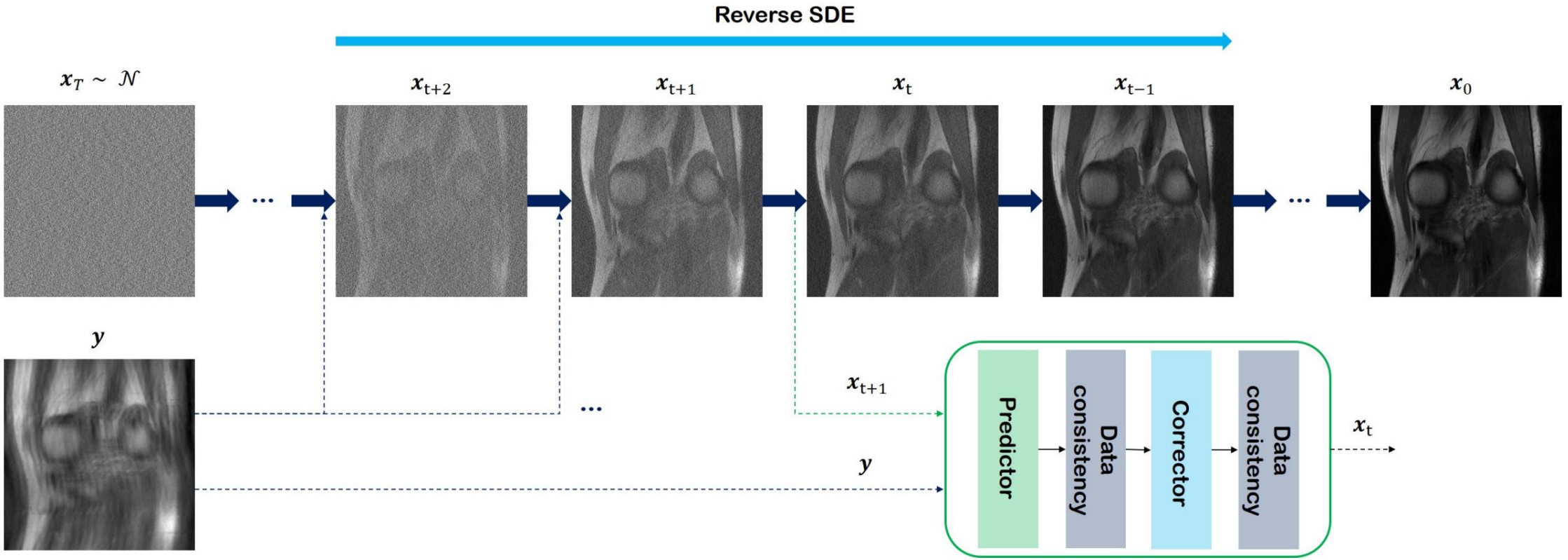


Figure 7: An overview of Score-MRI [48]. x_T is derived from the pre-trained prior distribution, and x_0 is retrieved by sequentially applying the predictor, data consistency, corrector, and data consistency steps, given the measurement through the reverse SDE procedure.

Single-cell and multi-omics: generation, imputation, trajectories

Synthetic cells

- Generate realistic scRNA-seq profiles for scarce cell types.
-
- Useful for augmentation, benchmarking, and rare-state discovery.

Missing-data completion

- Impute genes, chromatin accessibility, or protein abundances.
-
- Complete missing modalities in paired or partially paired designs.

Continuous trajectories

- Condition on pseudotime or covariates to model development and perturbation paths.
-
- Helpful for “what-if” simulation at the cell-state level.

Why it matters biologically

Single-cell data are sparse, compositional, and highly heterogeneous—exactly the setting where conditional generative modeling can reveal smooth latent structure while still preserving multimodality.

Good evaluation targets

cell-type fidelity
rare-cell coverage
trajectory smoothness
preservation of differential signal

Drug discovery and protein design

Small molecules

- Generate novel ligands conditioned on pocket geometry or target constraints.
-
- Optimize physicochemical properties while preserving 3D plausibility.

Proteins and peptides

- Design backbones, sequences, or therapeutic peptides with structural guidance.
-
- Useful for de novo design and structure-conditioned generation.

Docking / interactions

- Model protein–ligand or protein–protein configurations under constraints.
-
- Potential bridge from generative design to testable candidates.

The real opportunity

Diffusion models can sample from a chemically and structurally valid design space while being explicitly conditioned on function, structure, taxonomy, or target context.

The bottleneck

Not generation alone—scoring, synthesizability, stability, and experimental validation still decide whether a design matters.

Bottom line for bio research

Where diffusion helps most

1. Noisy inverse problems in imaging
2. Sparse, multimodal single-cell and omics settings
3. Conditional molecular and protein design

What to watch out for

- • Biological validity beyond realism
-
- • Leakage, privacy, and cohort bias
-
- • Compute cost and evaluation difficulty
-
- • Need for wet-lab or external validation

If the scientific question can be phrased as “generate / reconstruct / complete a complex biological object under constraints,” diffusion is often a strong candidate model.

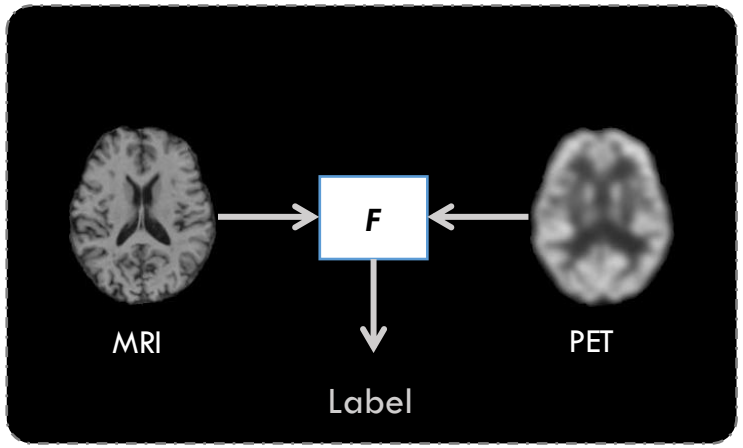
Cross-Modality Image Synthesis

Cross-Modal Image Synthesis via Deep Learning

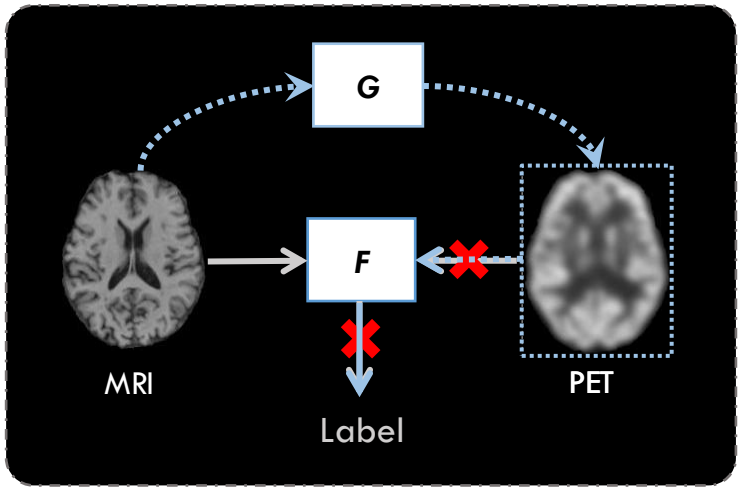
Y. Pan, M. Liu, et al. *MICCAI*, 2018
Y. Pan, M. Liu, et al. *IEEE Transactions on Medical Imaging*, 2020
Y. Pan, M. Liu, et al. *IEEE Trans. on Pattern Analysis and Machine Intelligence*, 2022

Cross-Modal Image Synthesis for Diagnosis

- Multi-modal imaging data for disease diagnosis (e.g., MRI and PET)
  - Providing complementary information of the brain
    Subjects usually have incomplete multi-modal data



Diagnosis with complete multi-modal images

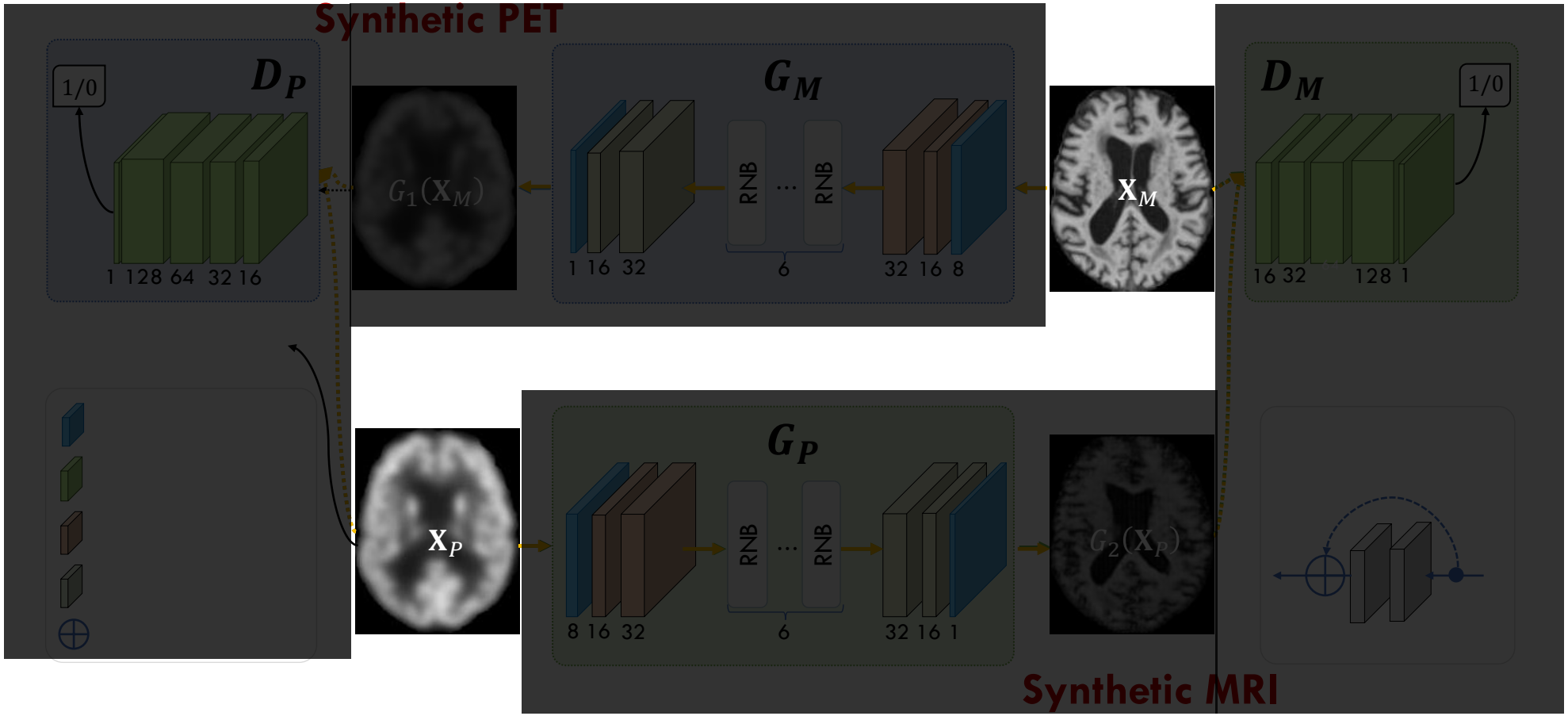


Diagnosis with incomplete multi-modal images

Y. Pan, M. Liu, C. Lian, Y. Xia, and D. Shen. *MICCAI*, 2019
Y. Pan, M. Liu, C. Lian, T. Zhou, Y. Xia, and D. Shen. *MICCAI*, 2018
Y. Pan, M. Liu, C. Lian, L. Yue, S. Xiao, Y. Xia, and D. Shen. *ISMRM*, 2019

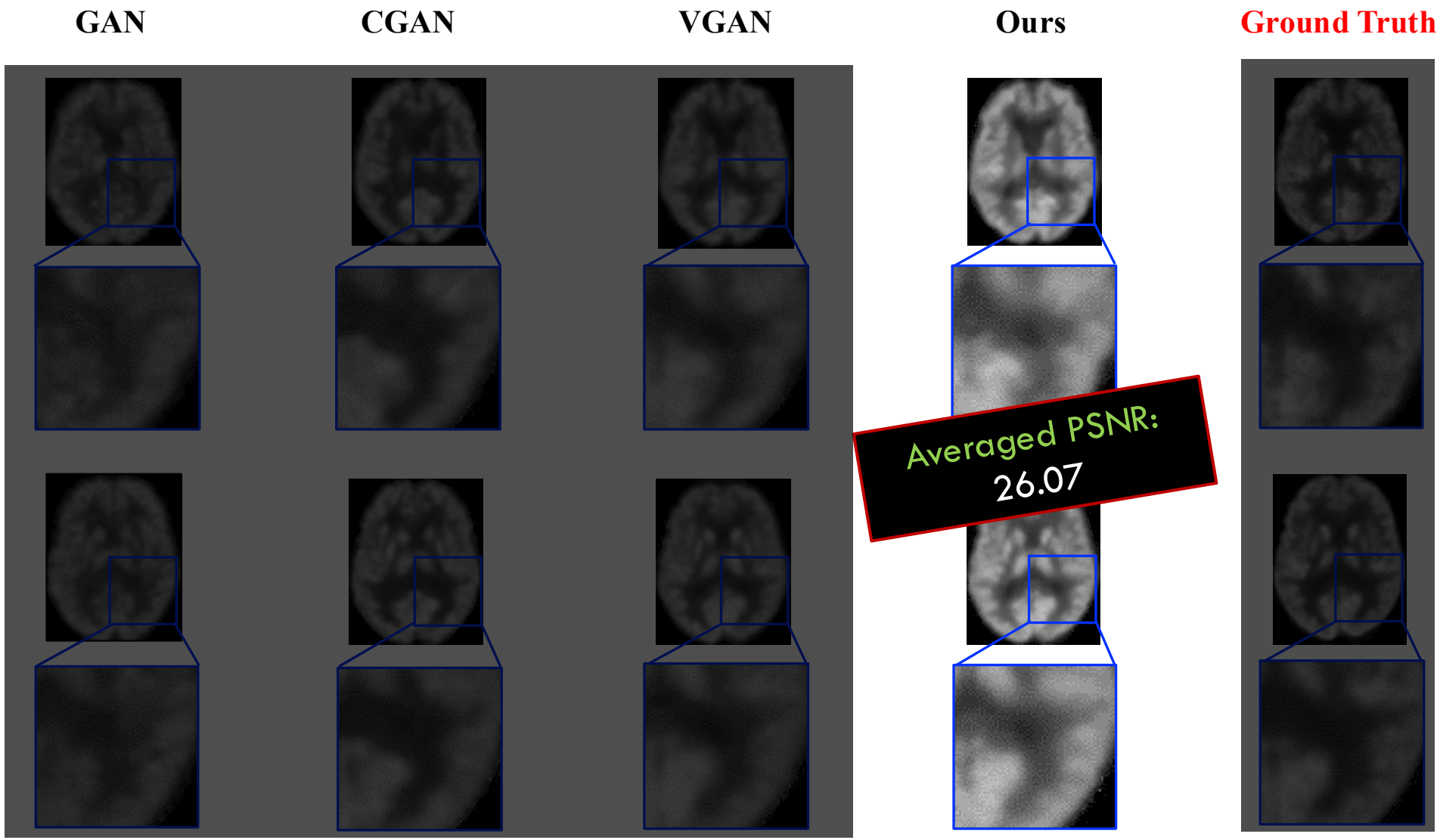
Cross-Modality Image Synthesis

- Generating missing PET/MRI scans for diagnosis



Y. Pan, M. Liu, C. Lian, T. Zhou, Y. Xia, and D. Shen. *MICCAI,* 2018
Y. Pan, M. Liu, C. Lian, L. Yue, S. Xiao, Y. Xia, and D. Shen. *ISMRM*, 2019

Synthetic PET Scans

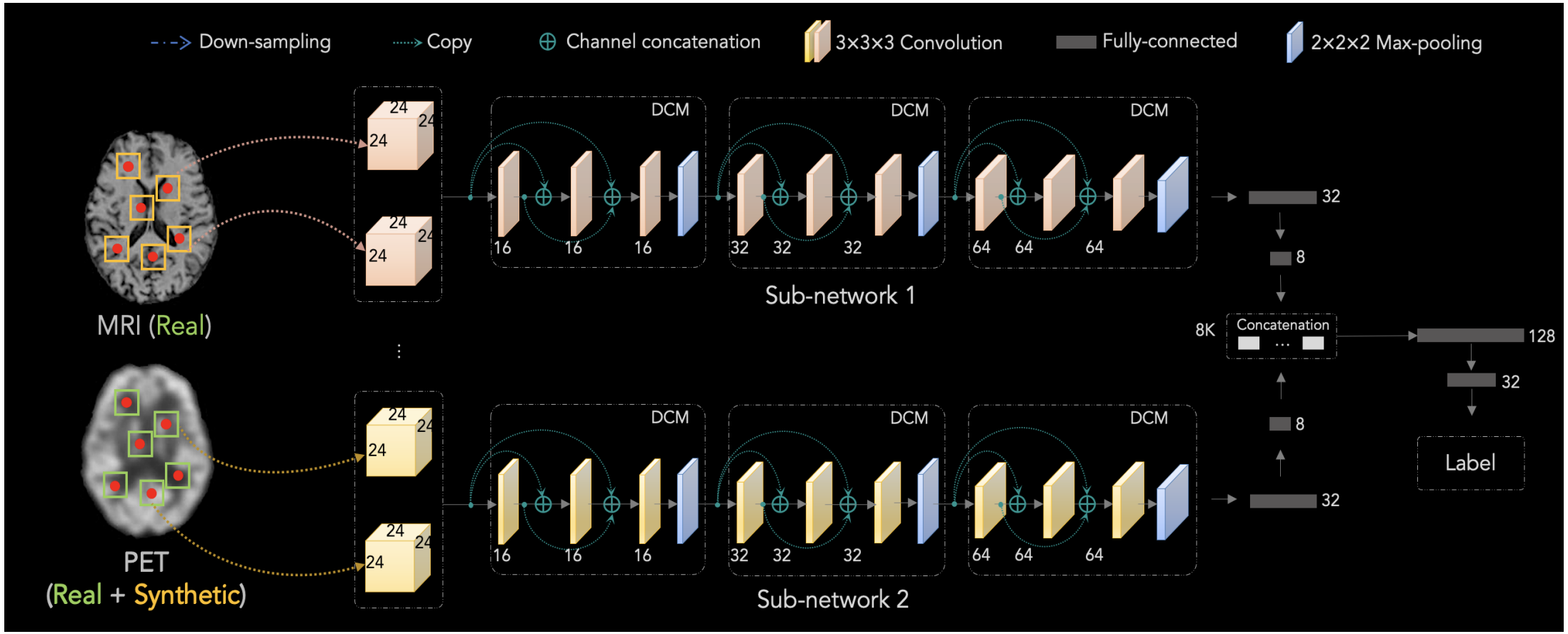


Y. Pan, M. Liu, C. Lian, T. Zhou, Y. Xia, and D. Shen. *MICCAI*, 2018
Y. Pan, M. Liu, C. Lian, L. Yue, S. Xiao, Y. Xia, and D. Shen. *ISMRM*, 2019

PSNR: Peak signal-to-noise ratio

Multi-Modal Classification

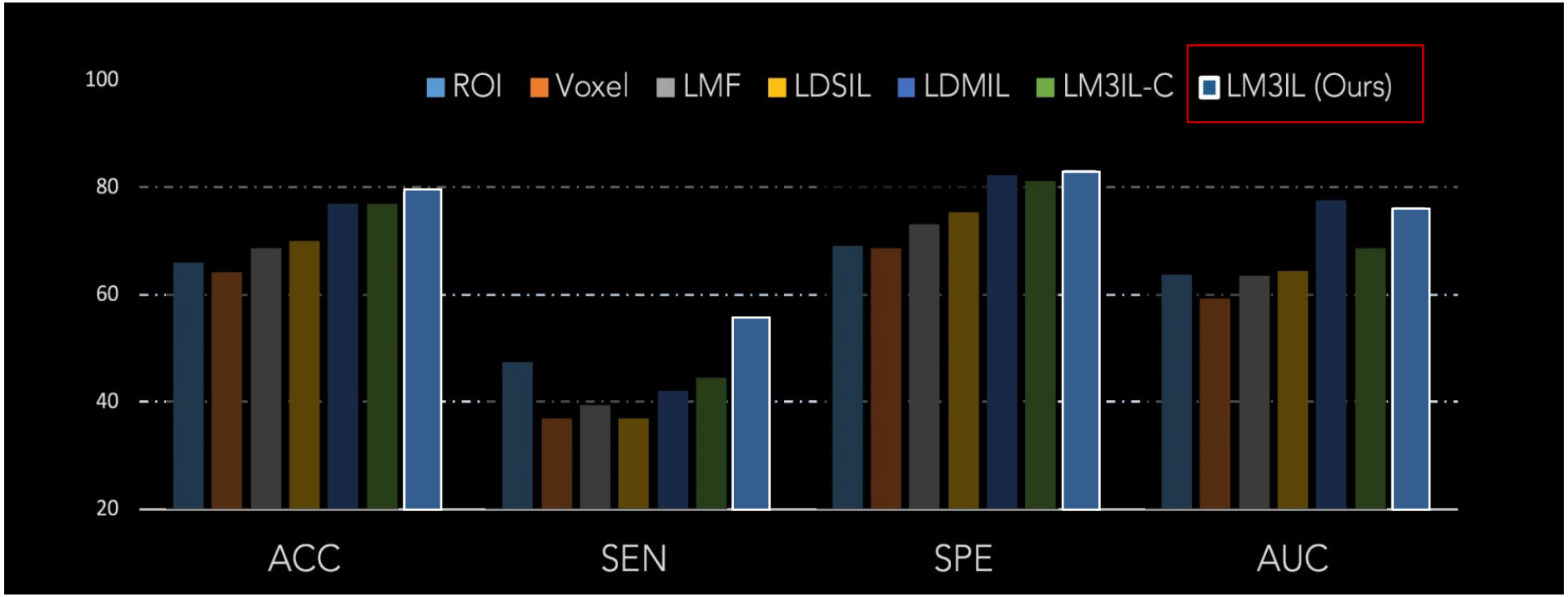
- Landmark-based multi-scale network for classification



Y. Pan, M. Liu, C. Lian, T. Zhou, Y. Xia, and D. Shen. *MICCAI*, 2018
Y. Pan, M. Liu, C. Lian, L. Yue, S. Xiao, Y. Xia, and D. Shen. *ISMRM*, 2019

Experiments

- Results of pMCI vs. sMCI with complete MRI and PET (after imputation)



Y. Pan, M. Liu, C. Lian, T. Zhou, Y. Xia, and D. Shen. *MICCAI*, 2018
Y. Pan, M. Liu, C. Lian, L. Yue, S. Xiao, Y. Xia, and D. Shen. *ISMRM*, 2019

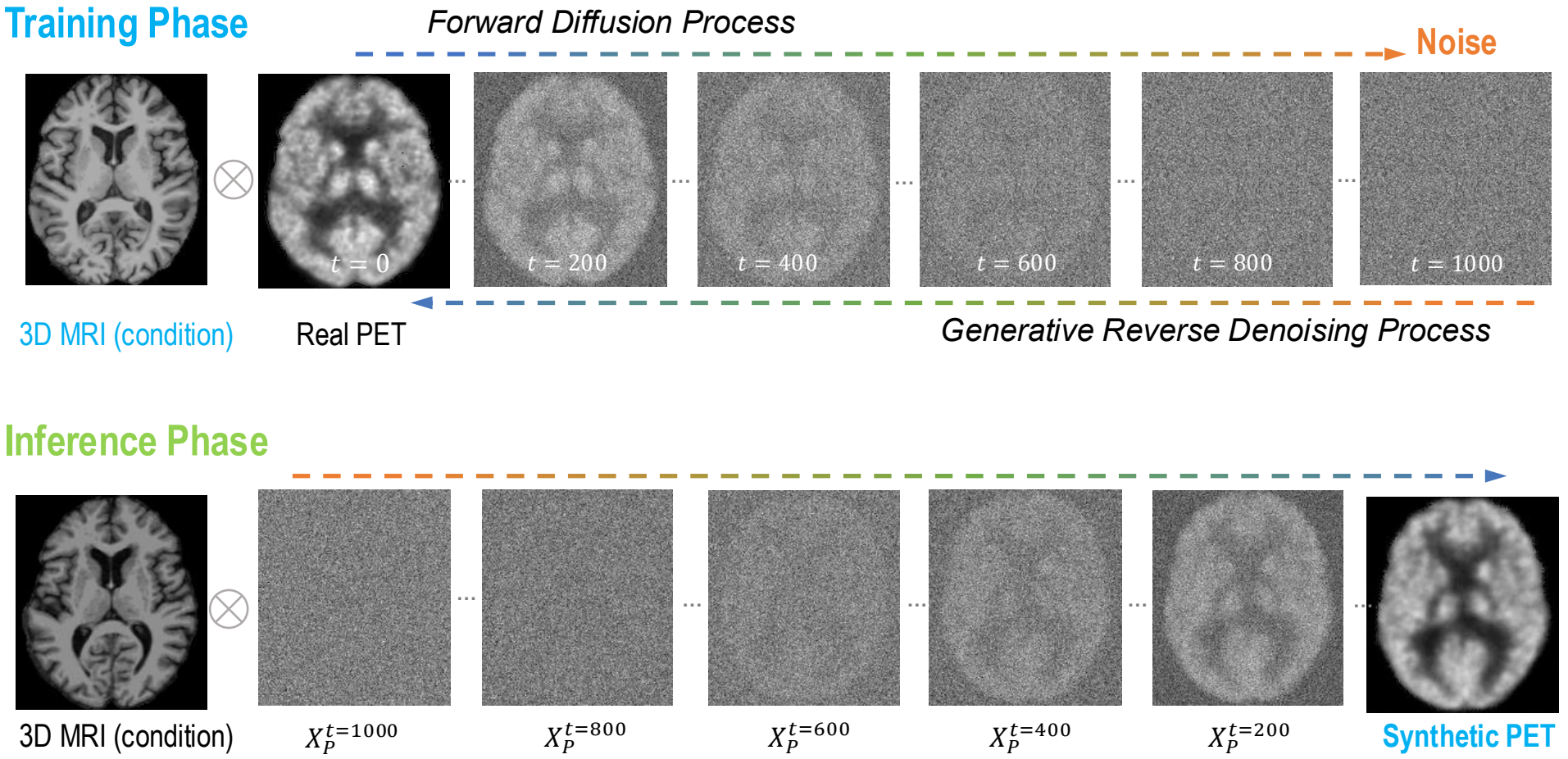
Diffusion-Based Image Synthesis

Image-Constrained Diffusion for MRI-to-PET Image Synthesis

M. Yu, M. Wu, L. Yue, A. Bozoki, and M. Liu. *Medical Image Analysis*, 2026
M. Yu, M. Liu. *ISMRM*, 2024

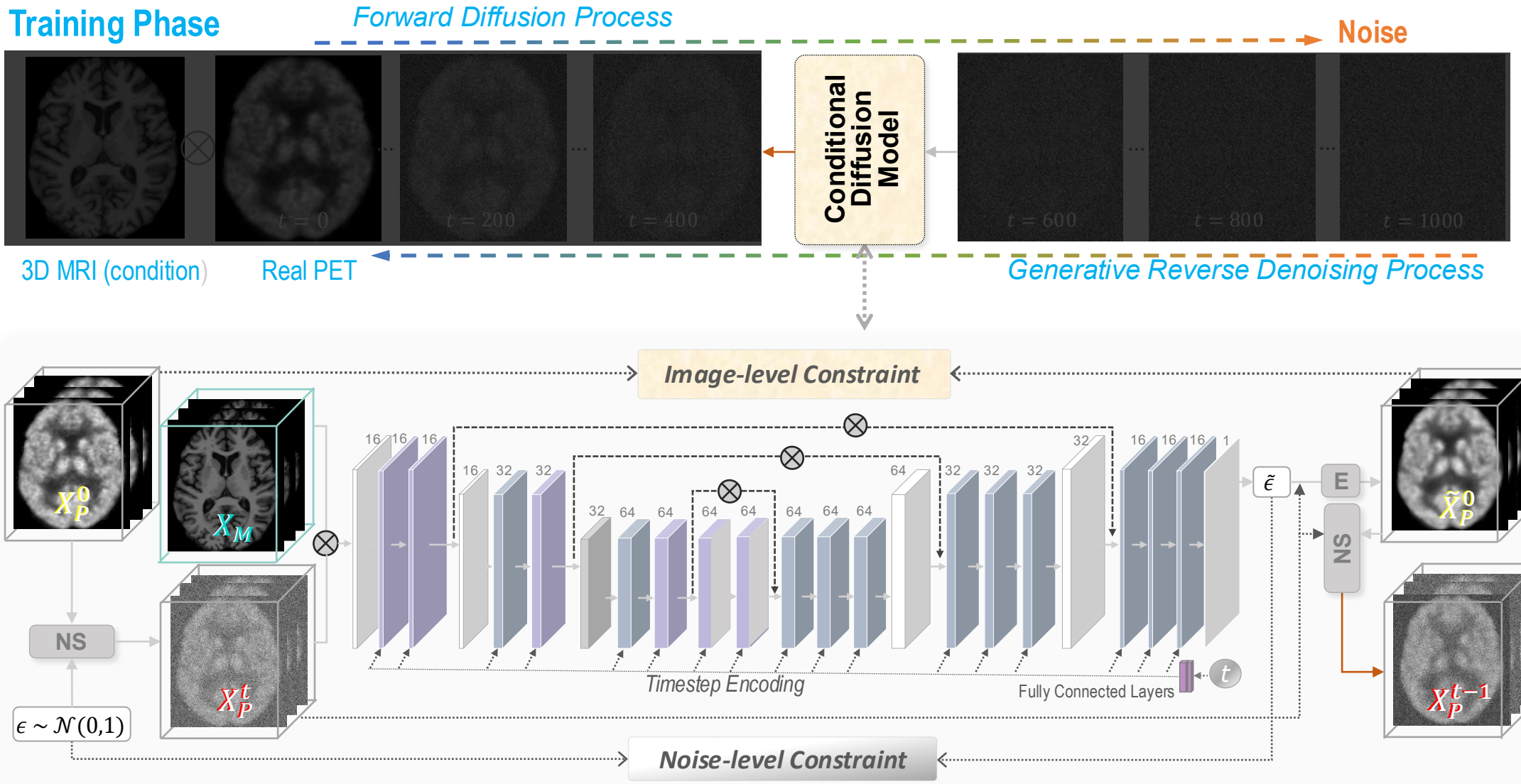
Deep Denoising Diffusion for PET Synthesis

- Generate PET scans from MRI

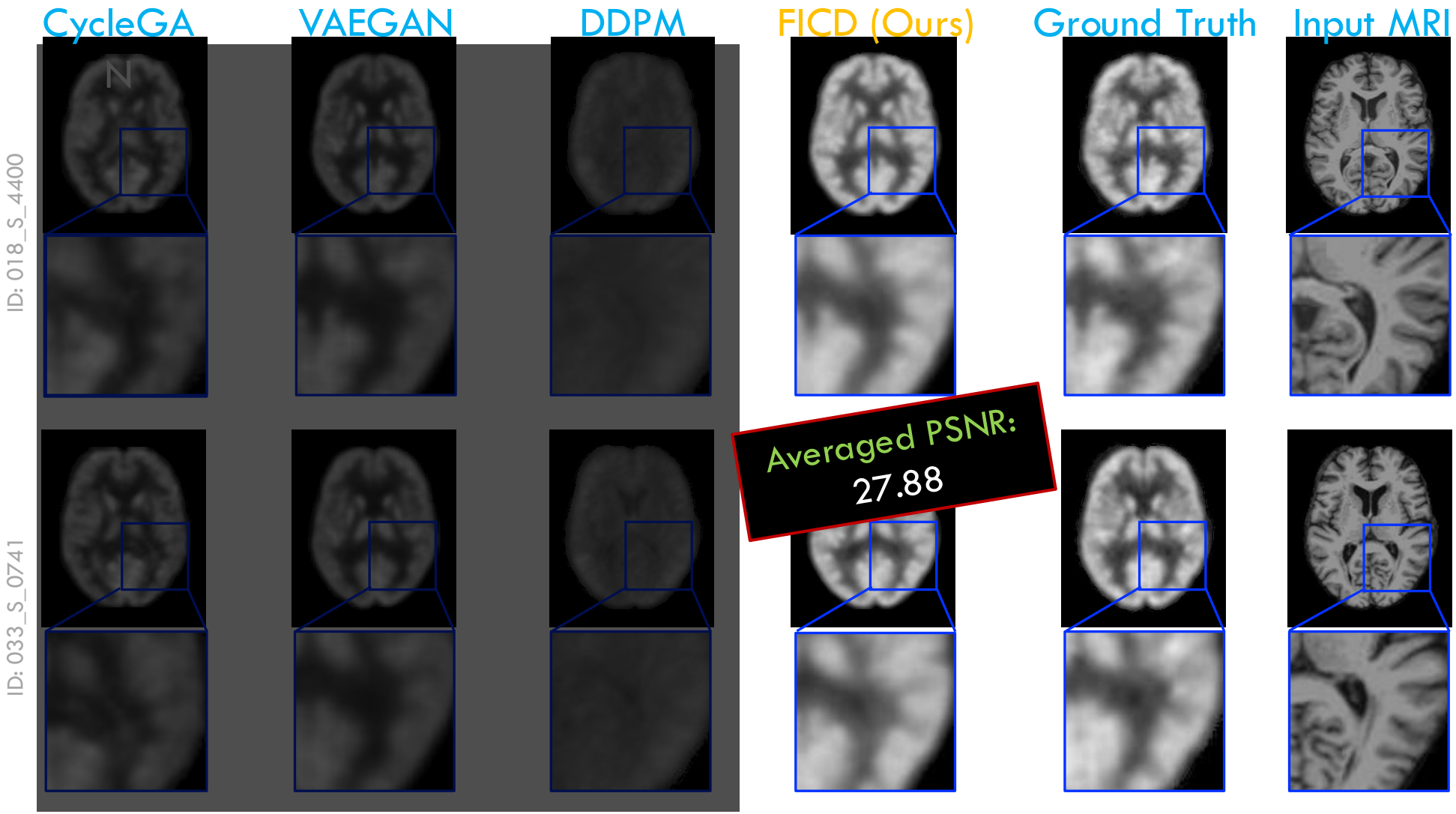


M. Yu, M. Wu, L. Yue, A. Bozoki, and M. Liu. *Medical Image Analysis*, 2026
M. Yu, M. Liu. *ISMRM*, 2024

Deep Denoising Diffusion for PET Synthesis

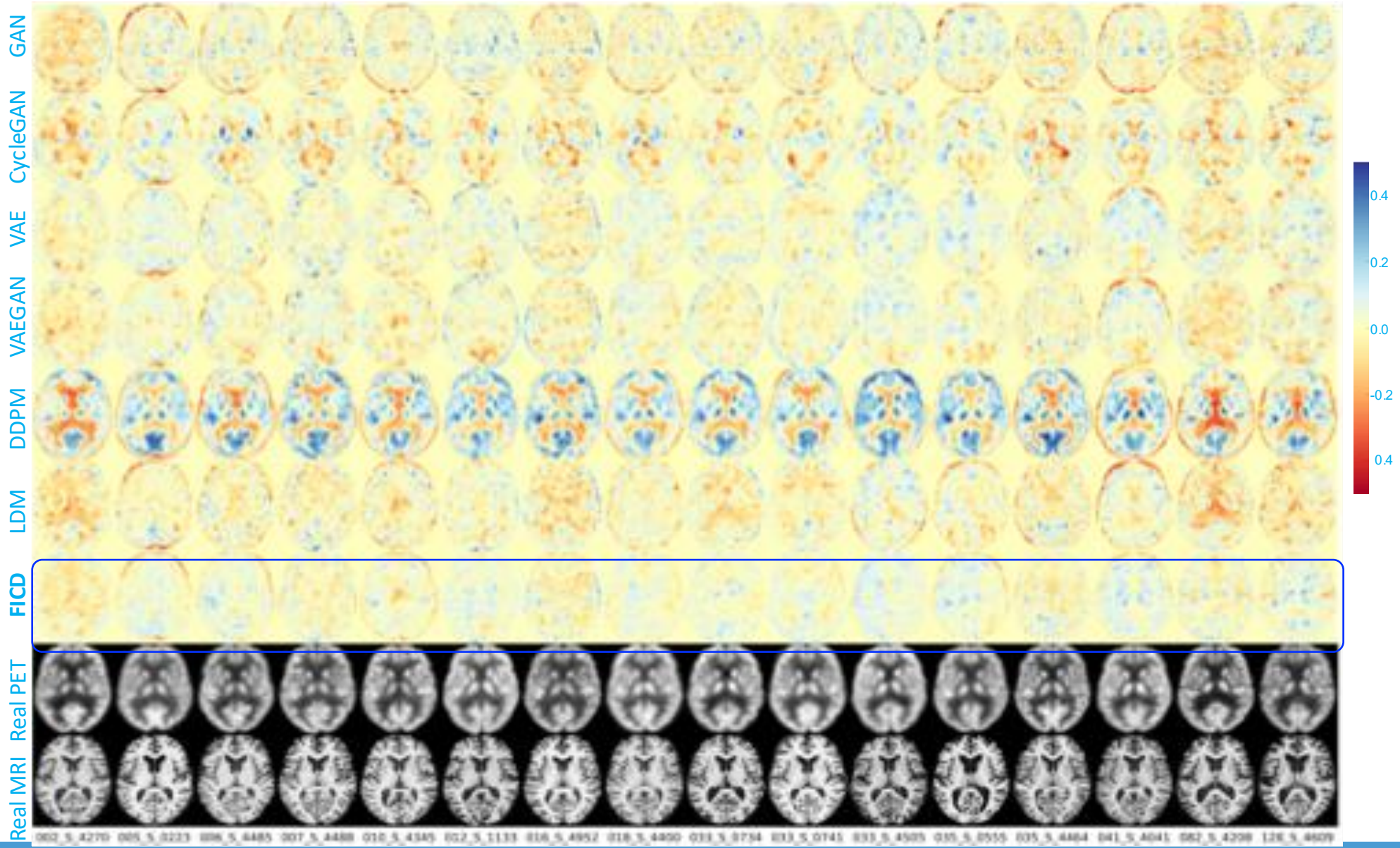


Synthetic FDG-PET Scans



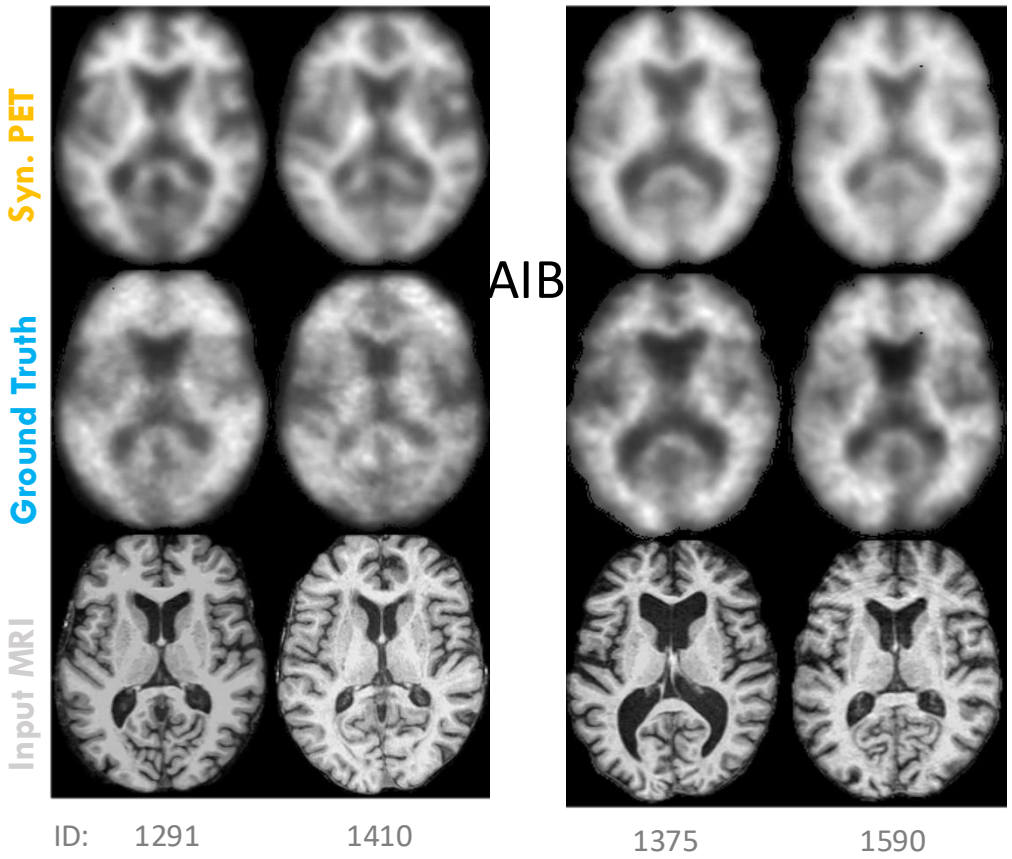
PSNR: Peak signal-to-noise ratio

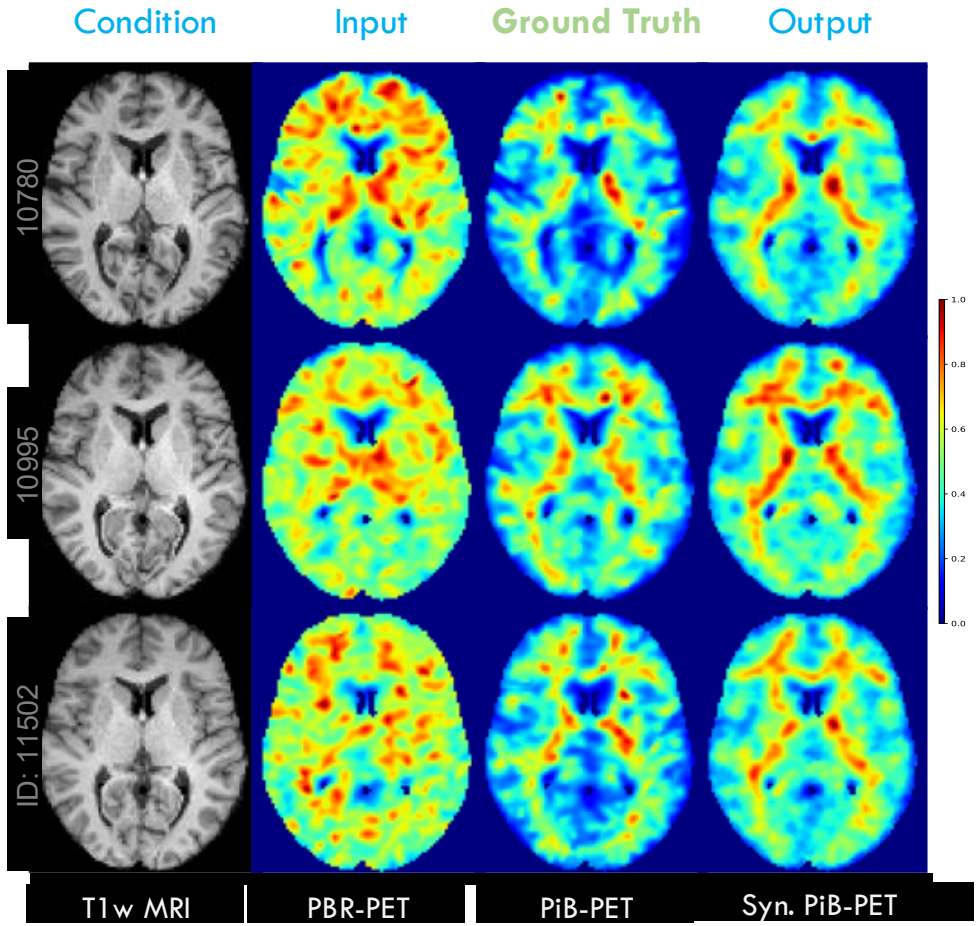
Synthetic FDG-PET Scan – Difference Map (Axial View)



Generalization to Synthesize Amyloid-PET

- Synthesize Amyloid PET on AIBL[1]





[1] Ellis, Kathryn A., et al. *International Psychogeriatrics* , 2009.
[2] B. L. Brett, et al. Journal of Neurology, Neurosurgery & Psychiatry, 2021.

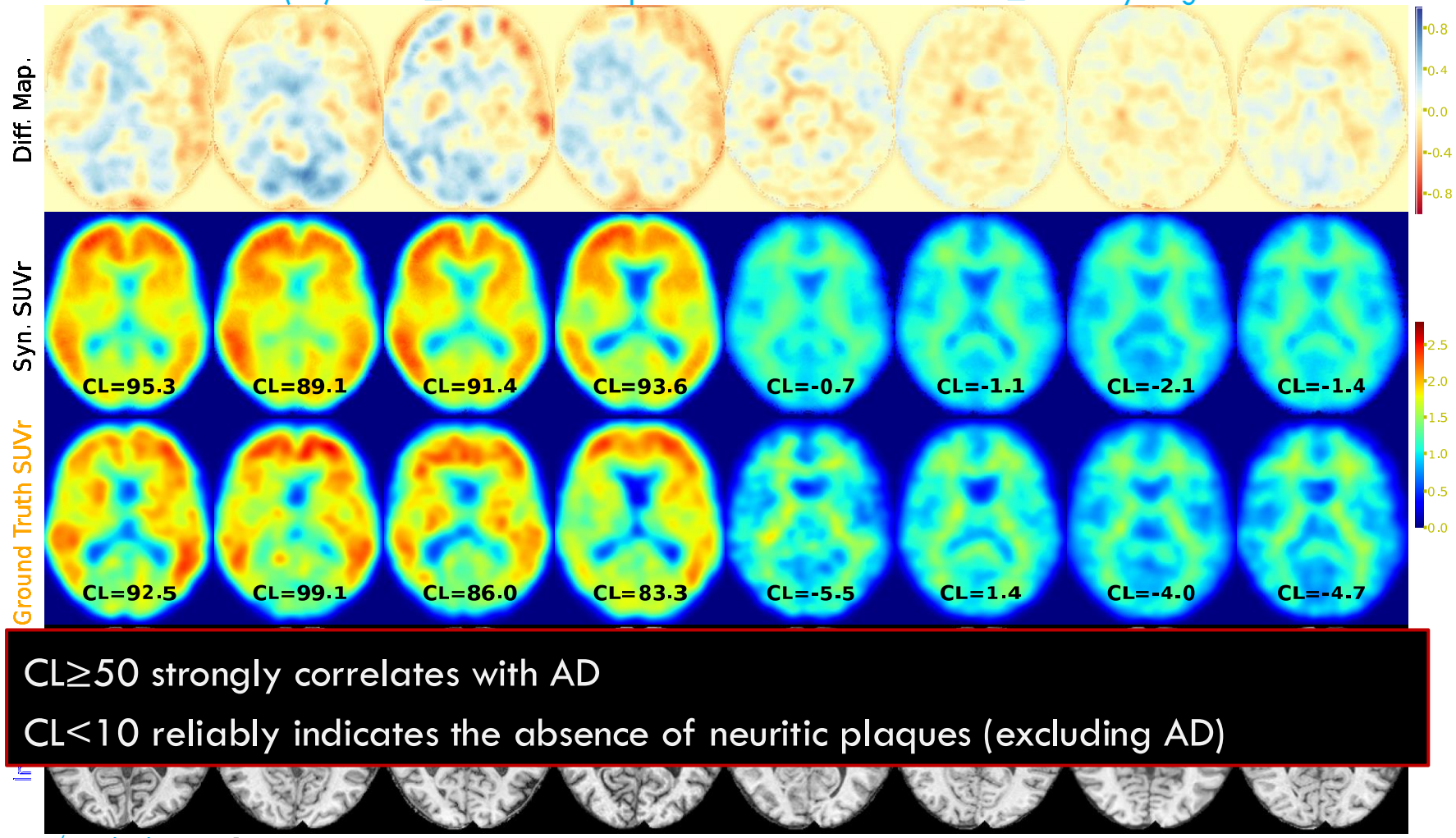
PBR: translocator protein (TSPO) tracer

Generated PET SUVr Maps

- Synthesize standardized uptake value ratio (SUVr) for PiB-PET in Centiloid Project[1]

Centiloid scale (CL): **100.9±5.8** for AD subjects

CL: **1.5±5.0** for young controls



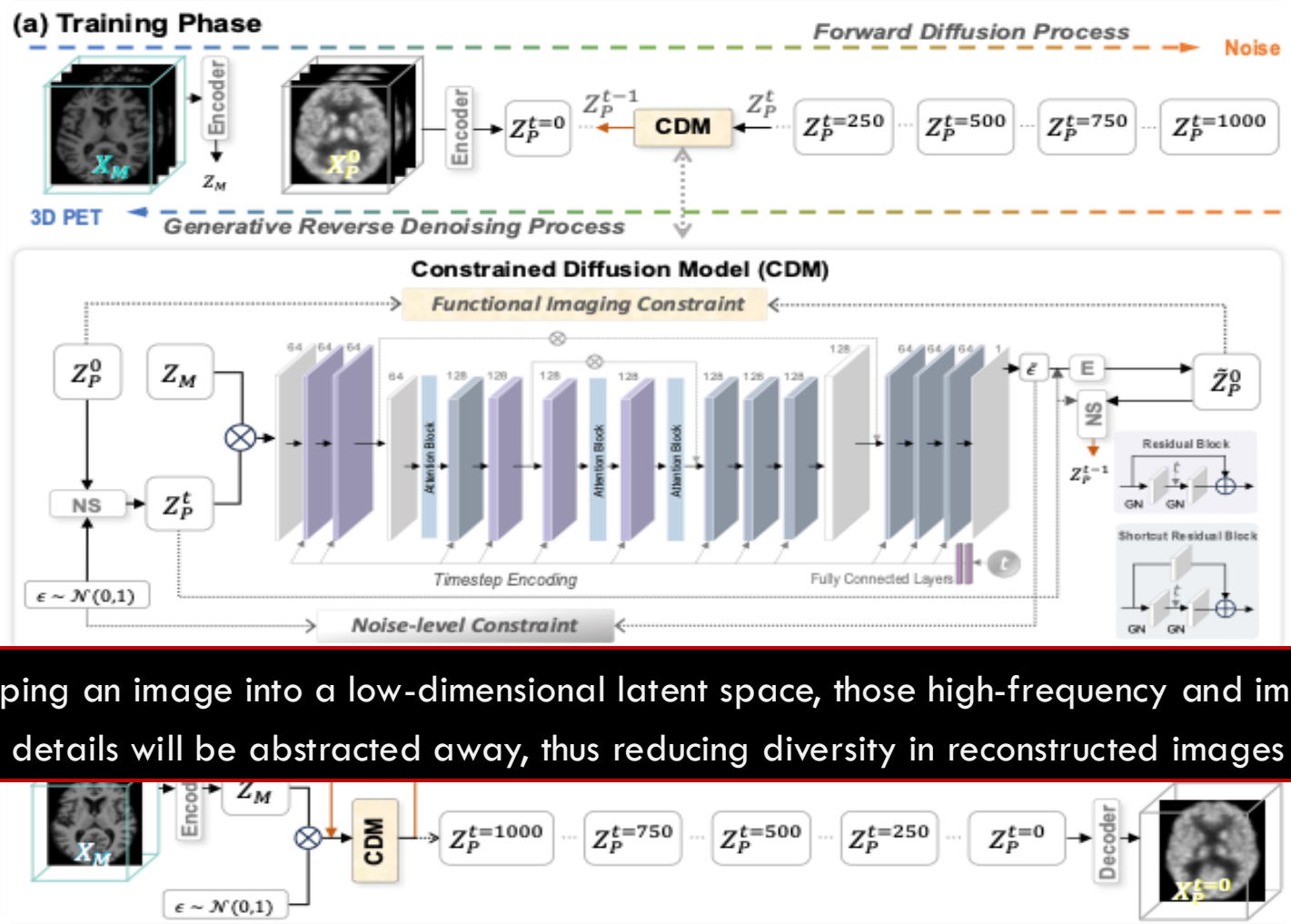
CL≥50 strongly correlates with AD

CL<10 reliably indicates the absence of neuritic plaques (excluding AD)

[1] https://www.gaain.org/centiloid-project[

Improve Output Uniqueness – Latent Diffusion

- Diffusion in latent space

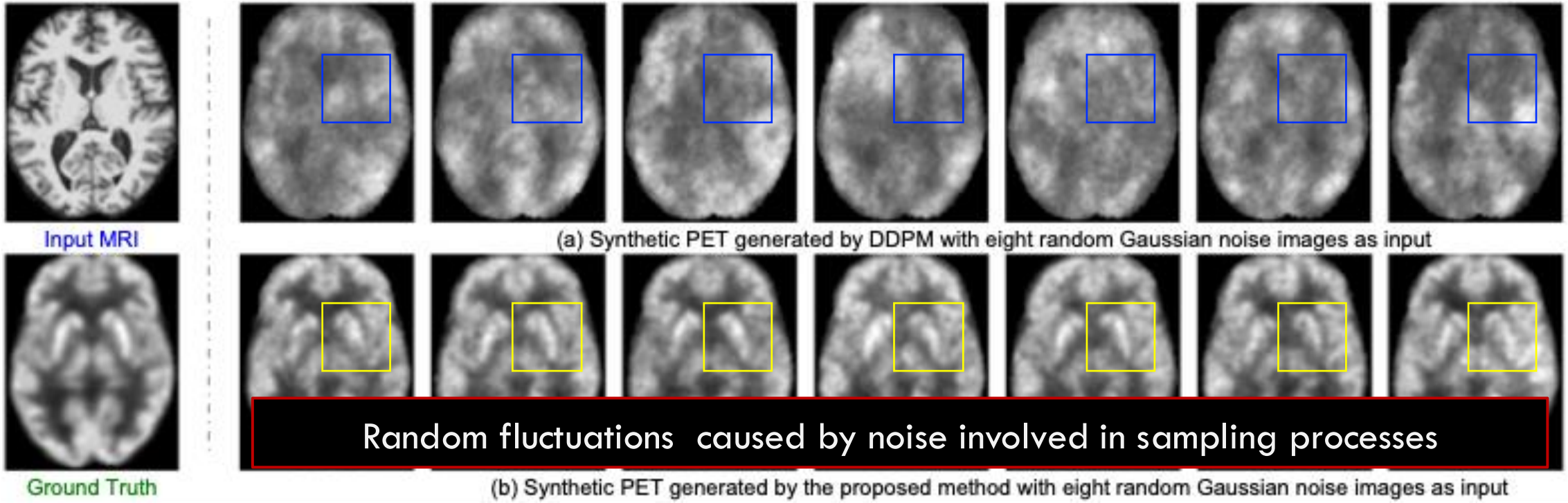


After mapping an image into a low-dimensional latent space, those high-frequency and imperceptible details will be abstracted away, thus reducing diversity in reconstructed images

Generalization to Synthesize Amyloid-PET

Radiotracer	PSNR↑	SSIM↑	MAE↓	NMI↑
AV45	22.8995±2.7352	0.8527±0.0643	0.0319±0.0102	0.7335±0.0861
PiB	21.4160±0.9953	0.8093±0.0096	0.0366±0.0069	0.7174±0.0107
FLUTE	26.9723±1.6334	0.9089±0.0148	0.0194±0.0040	0.8717±0.0542

Improve Output Uniqueness



eight separate sampling

iterations by (a) DDPM and (b) our FICD, with each iteration

Designing synthetic regulatory elements using the generative AI framework DNA-Diffusion

Lucas Ferreira DaSilva[1,2,27], Simon Senan[2,3,27], Judith F. Kribelbauer-Swietek[4,5,6], Zain Munir Patel[1,2,3], Lithin Karmel Louis[7], Aniketh Janardhan Reddy[8], Sameer Gabbita[2,9], Jonathan D. Rosen[10], Zach Nussbaum[11], César Miguel Valdez Córdova[12], Aaron Wenteler[13], Noah Weber[14], Tin M. Tunjic[15], Martino Mansoldo[16,24], Talha Ahmad Khan[17,25,26], Gue-Ho Hwang[1,2], Vincent Gardeux[5,6], David T. Humphreys[7,18], Cameron Smith[1,2,3], Matei Bejan[19], Peter Bromley[20], Will Connell[21], Bart Deplancke[5,6], Michael I. Love[10], Emily S. Wong[7,22], Wouter Meuleman[20,23] & Luca Pinello[1,2,3]

Motivation and Goal

Goal: Design short (~200 bp) DNA regulatory elements that are active in a chosen cell type.

Challenges:

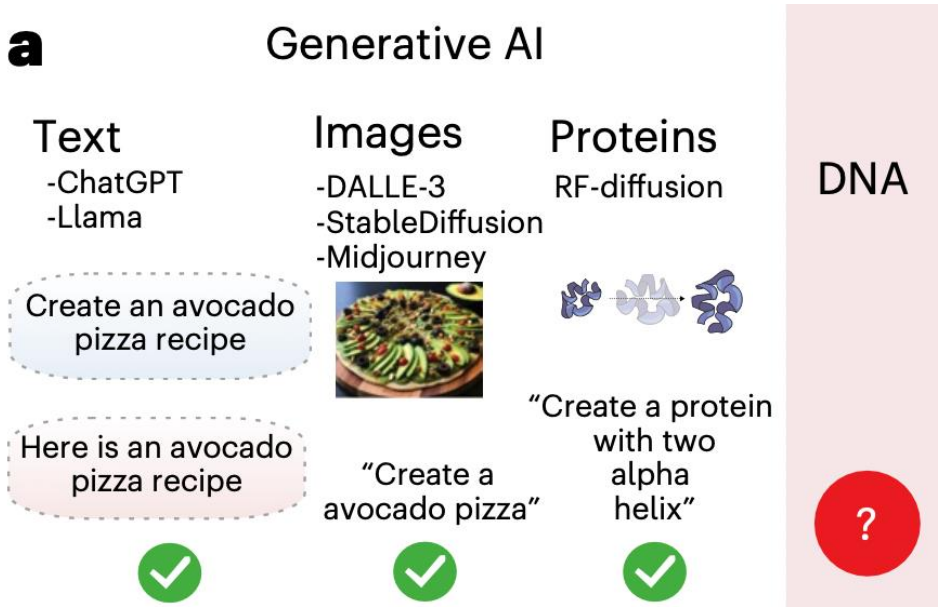
- DNA is easy to generate, but functional enhancers require correct TF motif "grammar"
- Balance: novelty + biological realism + cell-type specificity

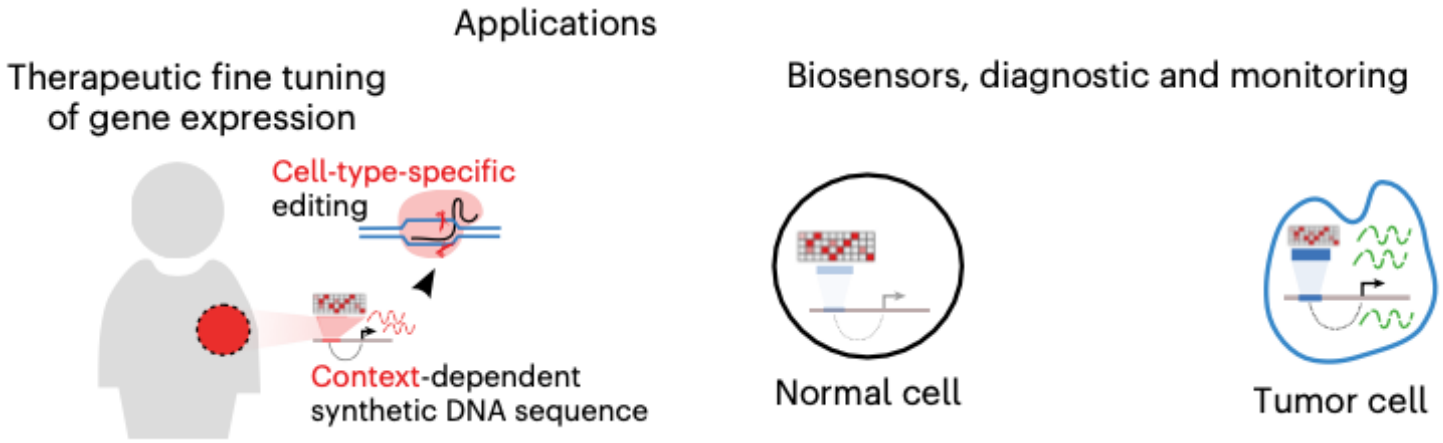
Contributions:

- A conditional diffusion model ("DNA-Diffusion") trained on cell-type-specific open chromatin (DHS) sequences
- Then in silico selection + wet-lab validation to find strong, specific enhancers

Applications:

- Context-dependent (cell type / chromatin / TF environment) synthetic DNA sequences
- Cell-type-specific editing / targeting
- Therapeutic fine tuning of expression





Data for Training

Training data = 200 bp DNA segments from DHS peaks

- Unique DNase I hypersensitive sites (DNase I) DNA sequences from different cell types, including K562, HepG2 and GM12878.
- For each cell type: take 200 bp around DHS summit

Train / validation / test split:

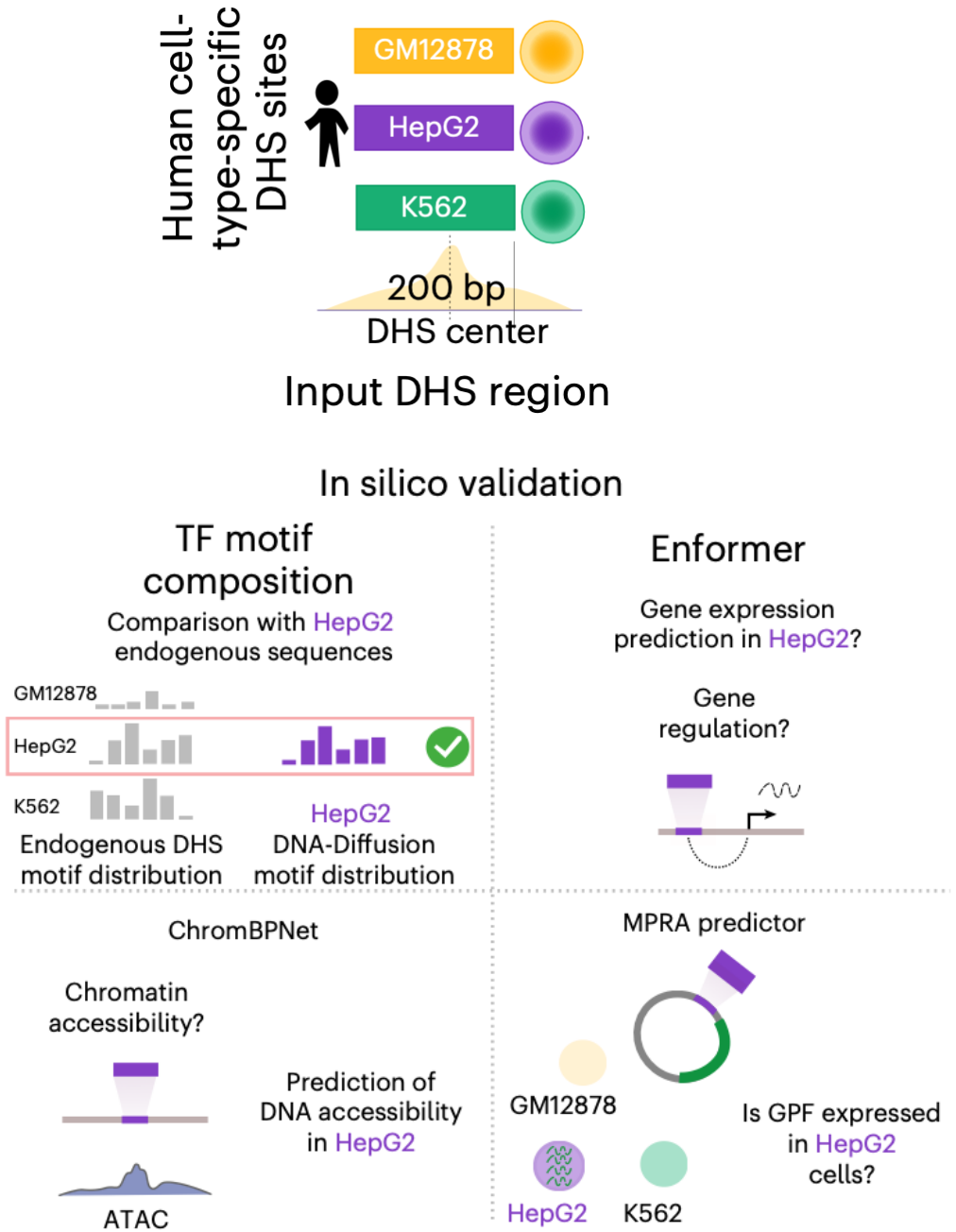
- Split by chromosome (e.g., hold out certain chromosomes for validation/test)

Generated:

- 100,000 synthetic 200 bp sequences per cell type (300k total).

Evaluate Generated Sequences:

- **In silico validation:**
  - ChromBPNet (accessibility),
  - MPRA predictor (reporter activity),
  - Enformer (DNase + CAGE expression tracks).
- **Wet lab validation:**
  - STARR-seq library (5,850 elements) across the 3 cell lines (episomal reporter assay).
  - EXTRA-seq endogenous locus experiment: replace enhancer region near AXIN2 and measure reactivation.



Conditional diffusion for DNA sequences

Diffusion Model:

- DDPM Learns $p(x \mid y)$:distribution of DNA sequences x given cell type y

Inputs to denoiser

- Noisy sequence x_t
- Diffusion timestep t
- Cell-type label y

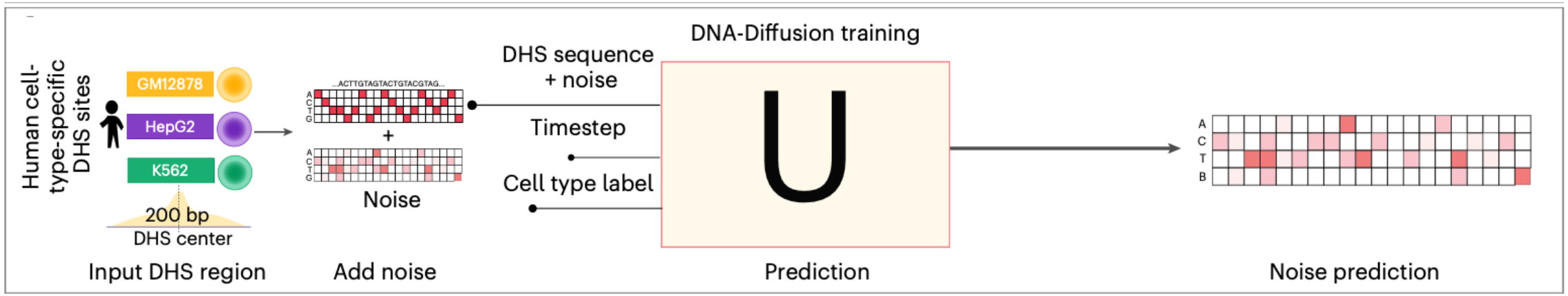
Architecture

- Denoising **U-Net** over a (4 × 200) one-hot-like DNA tensor
- Condition via **timestep embedding** + **cell-type embedding**

Training objective (core equation)

- Add noise: $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$
- Train network to predict noise:

$$\min_{\theta} \mathbb{E} \parallel \epsilon - \epsilon_\theta(x_t, t, y) \parallel^2$$



Classifier-Free Guidance: control specificity vs diversity

Classifier-Free Guidance (CFG)

During training, with some probability (~10%), replace the cell type y with a null token:

- Conditional: $\epsilon_\theta(x_t, t, y)$
- Unconditional: $\epsilon_\theta(x_t, t, \emptyset)$

This forces the model to learn both.

CFG at sampling time

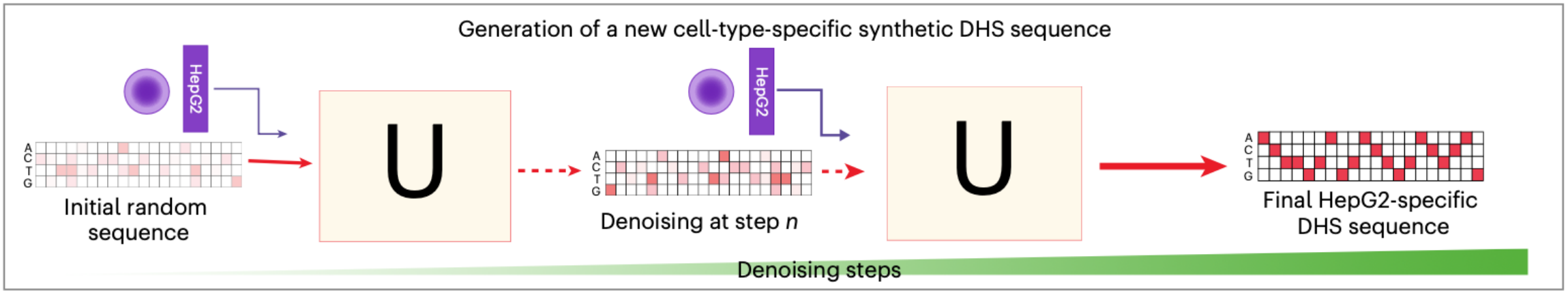
Compute:

$$\epsilon_{cond} = \epsilon_\theta(x_t, t, y), \epsilon_{uncond} = \epsilon_\theta(x_t, t, \emptyset)$$

Combine:

$$\epsilon_{CFG} = \epsilon_{uncond} + s(\epsilon_{cond} - \epsilon_{uncond})$$

- s =guidance scale.
- Larger $s$pushes the sample harder toward the cell-type distribution (more specificity, less diversity).



Experiment Results

Evaluation Metrics

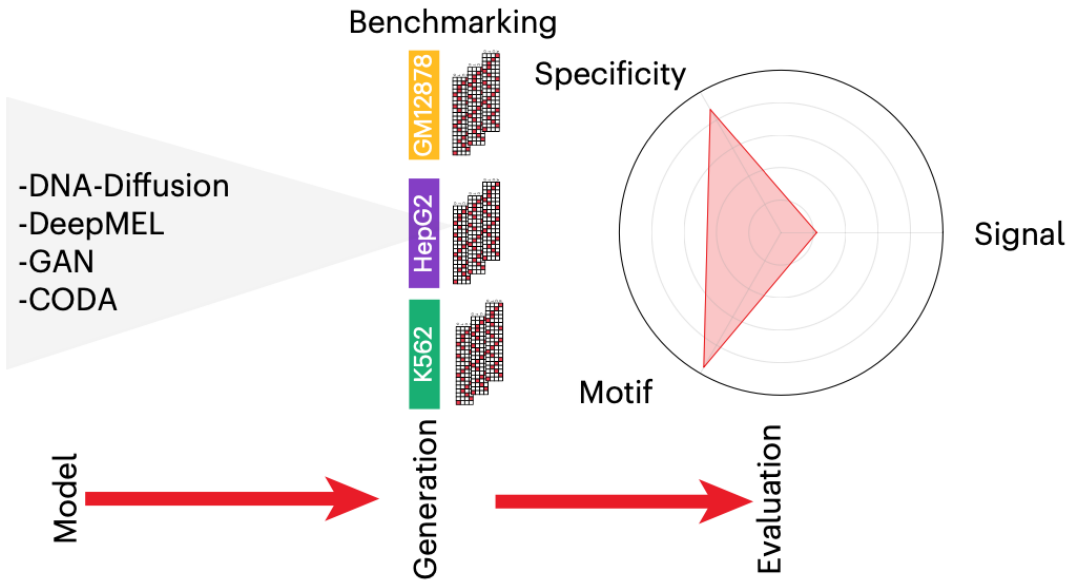
- **Novelty**: % of generated 200-bp sequences with long exact matches to training DHS
- **Motif realism**: Similarity of TF motif and motif-pair distributions (Jensen–Shannon distance)
- **Signal intensity**: how strong the predicted regulatory signal is (e.g., accessibility or CAGE level)
- **Signal specificity**: how much the signal is restricted to the target cell type (vs present in all cell types)

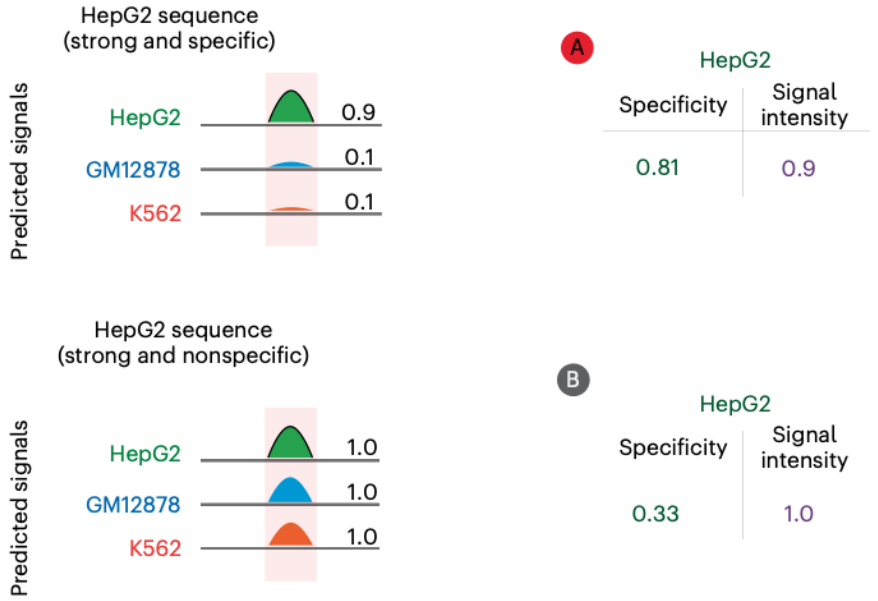
In silico validation:

- Motif grammar: are key TF motifs enriched appropriately?
- Predictive “oracles” (sequence → function proxy):
- Chromatin accessibility predictor
- MPRA / enhancer-activity predictor
- Enformer-like multi-track predictor

Wet-lab validation:

- STARR-seq: do sequences drive transcription in the target cell type?
- Endogenous locus test: does a designed sequence work in genomic context?





Content

9 Theoretical Properties

Theoretical Properties

Lemma 1 (Noise-Outsourcing Lemma)

1. *Let (X, Y) be a random pair taking values in $\mathcal{X} \times \mathcal{Y}$ with joint distribution $P_{X,Y}$.*
2. *Suppose $\mathcal{Y}$ is a standard Borel space.*

Then there exists a random variable $\eta \sim$ Uniform$[0,1]$ and a Borel-measurable function $G : [0,1] \times \mathcal{X} \to \mathcal{Y}$ such that η is independent of X and

$$(X, Y) = (X, G(\eta, X)) \quad \text{almost surely.} \tag{2}$$

GANs aim to approximate complex probability distributions through adversarial training.

Key theoretical questions:

- How to mathematically characterize the distribution of imaging and text datasets (e.g., ImageNet)?
- Why do we think that a specific GAN (loss function and architecture) can learn the key complexity of certain distributions?
- Can GANs converge to the true data distribution of some complex datasets?
- What distance measure is optimized in training?
- How to rigorously evaluate GANs in complex scenqrios?
- How do GANs generalize to unseen data?

- **What is the Manifold Hypothesis?**
  - High-dimensional data (e.g., images, text) often lies on a lower-dimensional manifold embedded in a higher-dimensional space.
  - Learning this low-dimensional structure is crucial for improving generative models.
- **Why Study Generative Models from this Perspective?**
  - Understanding DGMs through this lens helps explain their strengths and weaknesses.
  - Provides insights into why certain models (e.g., diffusion models, GANs) outperform others (e.g., VAEs, normalizing flows)

References

Aggarwal, A., Mittal, M., & Battineni, G. (2021). Generative adversarial network: An overview of theory and applications. *International Journal of Information Management Data Insights*, *1*(1), 100004.

Arjovsky, M., Chintala, S., and Bottou, L. (2017). Wasserstein generative adversarial networks. In International Conference on Machine Learning.

Brophy, E., Wang, Z., She, Q., & Ward, T. (2023). Generative adversarial networks in time series: A systematic literature review. *ACM Computing Surveys*, *55*(10), 1-31.

Chan, S. H. (2024). *Tutorial on Diffusion Models for Imaging and Vision.* arXiv:2403.18103.

Creswell, A., White, T., Dumoulin, V., Arulkumaran, K., Sengupta, B., & Bharath, A. A. (2018). Generative adversarial networks: An overview. *IEEE signal processing magazine*, *35*(1), 53-65.

Croitoru, F.-A., Hondru, V., Ionescu, R. T., & Shah, M. (2023). *Diffusion Models in Vision: A Survey.* IEEE Transactions on Pattern Analysis and Machine Intelligence, 45(9), 10850–10869.

Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. (2014). Generative adversarial nets. In Advances in neural information processing systems, pages 2672{2680.

Gui, J., Sun, Z., Wen, Y., Tao, D., & Ye, J. (2021). A review on generative adversarial networks: Algorithms, theory, and applications. *IEEE transactions on knowledge and data engineering*, *35*(4), 3313-3332.

Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., and Courville, A. (2017). Improved training of wasserstein gans. arXiv:1704.00028.

Higham, C. F., Higham, D. J., & Grindrod, P. (2025). *Diffusion Models for Generative Artificial Intelligence: An Introduction for Applied Mathematicians.* SIAM Review, 67(3), 607–623.

Ho, J., Jain, A., & Abbeel, P. (2020). Denoising Diffusion Probabilistic Models. *Advances in Neural Information Processing Systems (NeurIPS)*, 33, 6840–6851.

Hyvärinen, A. (2005). Estimation of non-normalized statistical models by score matching. Journal of Machine Learning Research, 6, 695–709.

Loaiza-Ganem, Gabriel, Brendan Leigh Ross, Rasa Hosseinzadeh, Anthony L. Caterini, and Jesse C. Cresswell. "Deep generative models through the lens of the manifold hypothesis: A survey and new connections." *arXiv preprint arXiv:2404.02954* (2024).

Kazerouni, A., Khodapanah Aghdam, E., Heidari, M., Azad, R., Fayyaz, M., Hacihaliloglu, I., & Merhof, D. (2023). *Diffusion models in medical imaging: A comprehensive survey.* Medical Image Analysis, 88, 102846.

References

Kingma, D.P. and Welling, M. (2019). An Introduction to variational autoencoders. Foundations and Trends® in Machine Learning 12 (4), 307-392

Mirza, Mehdi, and Simon Osindero. "Conditional generative adversarial nets." *arXiv preprint arXiv:1411.1784* (2014).

Nichol, A. Q., & Dhariwal, P. (2021). Improved Denoising Diffusion Probabilistic Models. In *Proceedings of the 38th International Conference on Machine Learning (ICML 2021)*, *Proceedings of Machine Learning Research*, 139, 8162–8171.

Pan, Z., Yu, W., Yi, X., Khan, A., Yuan, F., & Zheng, Y. (2019). Recent progress on generative adversarial networks (GANs): A survey. *IEEE access*, *7*, 36322-36333.

Papamakarios, G., Nalisnick, E., Rezende, D. J., Mohamed, S., & Lakshminarayanan, B. (2021). *Normalizing Flows for Probabilistic Modeling and Inference*. Journal of Machine Learning Research, 22(57), 1–64.

Song, J., Meng, C., & Ermon, S. (2021). Denoising Diffusion Implicit Models. In *International Conference on Learning Representations (ICLR 2021)*.

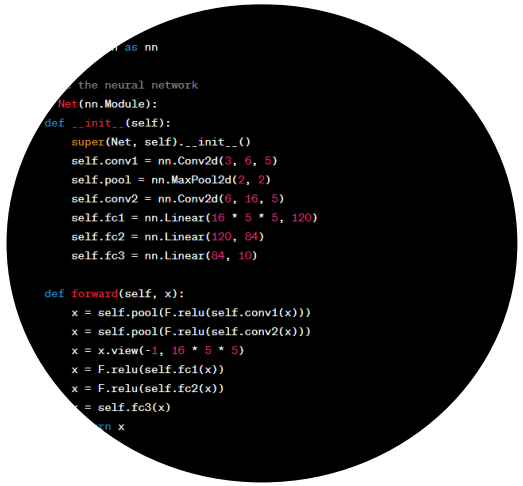
Xu, Lei, Maria Skoularidou, Alfredo Cuesta-Infante, and Kalyan Veeramachaneni. "Modeling tabular data using conditional gan." *Advances in neural information processing systems* 32 (2019).

Yi, Xin, Ekta Walia, and Paul Babyn. "Generative adversarial network in medical imaging: A review." *Medical image analysis* 58 (2019): 101552.

Zhan, Z., Chen, D., Mei, J.-P., Zhao, Z., Chen, J., Chen, C., Lyu, S., & Wang, C. (2024). *Conditional image synthesis with diffusion models: A survey*. *arXiv preprint* arXiv:2409.19365.

Zhou, X., Jiao, Y., Liu, J., and Huang, J. (2023). A deep generative learning approach to conditional sampling. Journal of the. American Statistical Association, 118(543):1837-1848.

How to succeed in this course?



Practice

Explore

Visualize

Discuss

Ask